
UNIT 13 TOOLS FOR DATA SCIENCE – AN INTRODUCTION

- 13.1 Introduction
- 13.2 Objectives
- 13.3 Tools for Data Science
- 13.4 Datasets and File Formats
- 13.5 Basics of Tableau
- 13.6 Basics of Power BI
- 13.7 Basics of Python
- 13.8 Basics of R-STUDIO
- 13.9 Summary
- 13.10 Solutions/Answers
- 13.11 Further Readings

13.1 INTRODUCTION

In today's digital era, data is often considered as the new oil—an invaluable resource powering decision-making across industries like healthcare, finance, retail, education, and manufacturing. Yet, the true power of data lies not in its raw form, but in “how effectively we can interpret and apply it?”. That's where data science steps in: a multidisciplinary domain blending mathematics, statistics, computer science, and domain expertise to transform chaotic data into meaningful knowledge.

At the heart of this transformation from data to useful information, there exists a powerful suite of tools—technologies that enable data professionals to collect, process, analyze, visualize, and communicate insights with efficiency and precision. These tools form the foundation of the data science workflow, streamlining everything from basic data handling to complex predictive modelling and real-time reporting.

This Unit, serves as your foundational guide to four of the most widely adopted tools in the data science pipeline:

- **Tableau** – An industry-leading platform for visual storytelling and dashboard creation.
- **Power BI** – Power BI – A robust business intelligence tool from Microsoft that lets you create interactive reports and easily connects with other Microsoft products for smooth data analysis.
- **Python** – A versatile and easy-to-learn programming language, Python is widely favoured by beginners and professionals alike for data tasks—thanks to powerful libraries like Pandas, NumPy, Matplotlib, and Scikit-learn that simplify data handling and machine learning.

- **R** – A statistical powerhouse designed for analytical rigor, favoured in academia and research-heavy environments.

The sections covered in this unit, introduces not just the technical skills needed to work with these tools, but also offers practical exposure to their user interfaces, dataset handling methods, and data visualization techniques. The goal is to equip students, aspiring data scientists, and professionals with both the confidence and competence to begin their journey in data science.

Whether you're analysing trends, forecasting outcomes, or simply making smarter decisions, mastering these tools is your first step into a data-driven future. Let's begin.

13.2 OBJECTIVES

Upon completing this unit, learners will be able to:

1. To understand the core functionalities of Tableau, Power BI, Python, and R.
2. Identify and work with various dataset formats such as CSV, Excel, and JSON.
3. Upload, preprocess, and clean datasets for analysis.
4. Design and interpret meaningful data visualizations.

These objectives are designed to empower learners to not only master the basics of data tools, but also apply their skills across academic, research, or industry settings with clarity and confidence.

13.3 TOOLS FOR DATA SCIENCE

In the Unit 9 and 10 we had already explored MS-Excel as a tool for Data Science, but today in the rapidly evolving world of data, many other tools are available, these tools are essential for converting raw information into actionable insights. Apart from MS Excel, the following tools for Data science assist in collecting, cleaning, analyzing, and visualizing data across various domains.

Below are four foundational tools widely used in the industry and academia:

- **Tableau:** A powerful and easy-to-use visualization tool that lets you build interactive reports and dynamic dashboards—no coding required. It's perfect for turning data into clear stories and sharing insights with people who may not have a technical background.
- **Power BI:** Created by Microsoft, Power BI is a strong business intelligence tool that works smoothly with Excel and allows users to build real-time dashboards—making it ideal for enterprise-level data analysis and reporting.
- **Python (Anaconda/Google Colab):** A general-purpose programming language with extensive libraries like Pandas, NumPy, and Matplotlib

for data manipulation, analysis, and visualization. Anaconda simplifies local setup, while Colab provides a free cloud-based environment.

- **R (RStudio):** R is a programming language tailored for statistical analysis and data visualization. It's widely used in academic research and data-centric projects for its strong capabilities in handling complex data and producing high-quality graphics. RStudio makes coding, visualizing, and managing data efficient and user-friendly.

These tools are essential to today's data science process, and mastering them equips students and professionals with the skills to tackle real-world challenges efficiently and confidently

13.4 DATASET AND FILE FORMATS

A dataset is an organized set of data used for analysis and building models. It's usually laid out like a table, where rows show individual entries and columns represent different features or variables. Depending on the type and use of the data, datasets can be stored in different formats—one of the most common being CSV (Comma-Separated Values), which is widely used for storing tabular data; Excel files (.xls, .xlsx), which allow multiple sheets and more formatting options; JSON and XML, which are suitable for storing nested or hierarchical data; SQL databases, used for managing structured data; Parquet and ORC, which are efficient columnar storage formats often used in big data systems; and TXT files, which store plain text with custom delimiters.

Format	Description / Strengths / Use Cases
CSV (Comma-Separated Values)	A simple, plain-text format where each row is a new line and columns separated by commas. Very widely supported by spreadsheets (Excel, LibreOffice), data-analysis tools, databases. Great for tabular data, easy to read and edit.
Excel (.xls / .xlsx)	Spreadsheet format allowing multiple sheets, richer metadata / formatting than CSV. Handy if you want human-friendly editing, multiple tables in one file, or need features like formulas or embedded metadata. (Often exported to CSV / other formats for ML workflows.)
JSON (JavaScript Object Notation)	A text-based format supporting nested / hierarchical data (objects, arrays). Useful for semi-structured or hierarchical data (e.g., configuration data, nested records, API outputs). Readable by humans and easily parsed by most programming languages.
XML (eXtensible Markup Language)	Another text-based, hierarchical format. Sometimes used when data has complex nested structure or needs to follow a schema. Less common in modern ML tasks than JSON, but still used in legacy systems or data exchange.
SQL Databases / Relational	For structured datasets that need efficient querying, relationships, updates — more dynamic than flat files. Useful

Format	Description / Strengths / Use Cases
Databases	when data is large, relational, or needs concurrent access/updates.
Parquet, ORC, and other columnar / big-data formats	Designed for large-scale data (big data / analytics), these formats store data in a columnar fashion — which can lead to better compression, faster read/write especially when only some columns are needed. Well-suited for large datasets, distributed systems, data warehouses, and big-data frameworks.
Plain text (.txt) / delimiter-separated (custom delimiters)	Sometimes used for very simple or custom data, logs, semi-structured text. Flexible but requires agreed upon structure or parsing logic; often not ideal for complex datasets.

Datasets can be sourced from various platforms depending on the need and application. Popular sources include data-sharing platforms like **Kaggle** and the **UCI Machine Learning Repository**, which host numerous datasets for academic and practical use. **Government portals** such as **Data.gov** provide open-access datasets for public use. Additionally, **APIs** from platforms like Twitter or Google Maps offer real-time and structured data. Other methods include **web scraping**, where data is extracted from websites, and data collected through **IoT devices or sensors**, which continuously generate data in various domains. Some of the useful links to gather the datasets are given below:

- Kaggle — <https://www.kaggle.com/datasets>
- UCI Machine Learning Repository — <https://archive.ics.uci.edu/datasets>
- Google Dataset Search — (search via <https://datasetsearch.research.google.com/>)
- Open Government Data Platform India (data.gov.in) — <https://data.gov.in/>
- Data.gov (US) — <https://data.gov>
- Community/curated lists like “Awesome Public Datasets” on GitHub or data.world

We will discuss, the usage of datasets along with the utilization of various tools mentioned above. Let’s begin with Tableau first, in the next section.

13.5 TABLEAU

Tableau is a user-friendly and a powerful tool for creating interactive data visualizations. It connects easily to a wide range of data sources and lets users explore and present their data through clear, engaging visuals with its simple drag-and-drop interface, even beginners can build charts, graphs, and dashboards—no coding needed. Widely used by data analysts and scientists, Tableau is popular in industries like healthcare, tech, and e-commerce to support smart, data-driven decisions. Once installed, users can get started by activating their license or signing in with their Tableau account.

Tableau Installation: Tableau is a free, easy-to-use tool that lets users build engaging and interactive visualizations—all without writing any code. It helps transform raw data into a more readable and insightful format. Tableau has become a widely used tool in the business analytics space.

Users can connect with others across the globe, share dashboards and data visualizations, and even publish their work on websites, blogs, or social media platforms.

Here in this section, we will try to cover most of the features of Tableau in brief. To begin with the Prerequisites of Using Tableau.

As a Prerequisites for Using Tableau, it only requires Basic computer skills, such as running programs and navigating software interfaces, along with Familiarity with spreadsheet applications, lastly your Willingness to explore and learn new tools.

Firstly, Let's learn how to Install Tableau, in order to **Get Started with Installation**, simply go to the official website at <https://www.tableau.com> and follow the step-by-step instructions provided there.

The stepwise process is also given here, the Fig.1: Tableau Installation shows the interface available at official website at <https://www.tableau.com>

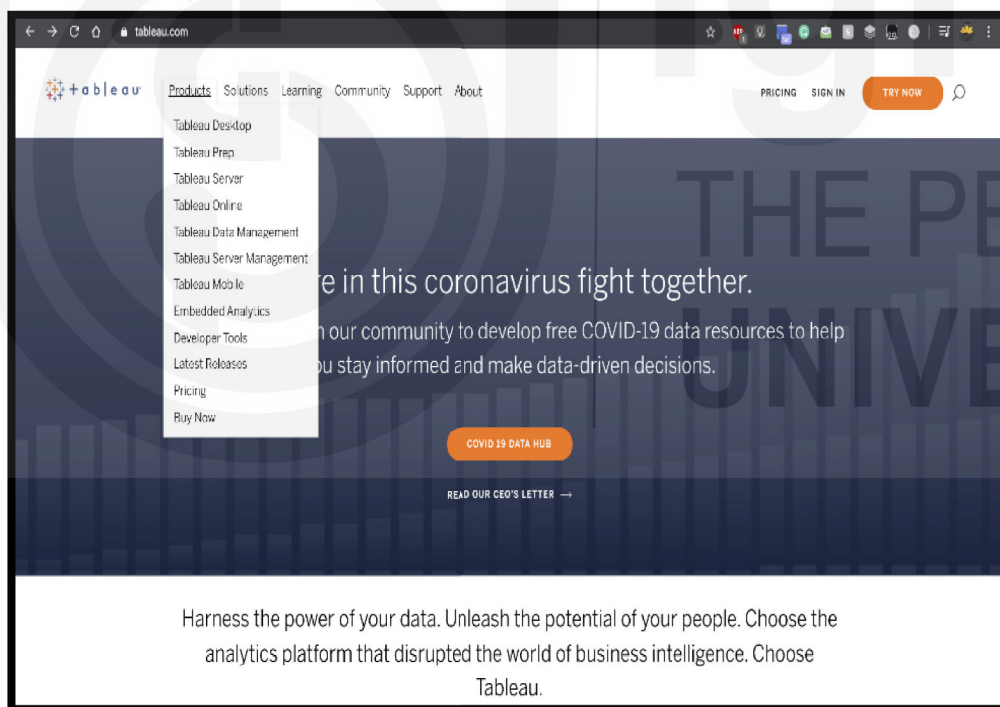


Fig.1: Tableau Installation

At official website at <https://www.tableau.com>, Navigate to the **Products** section, where you'll find four main options as shown in Fig.2: Tableau Desktop Products:

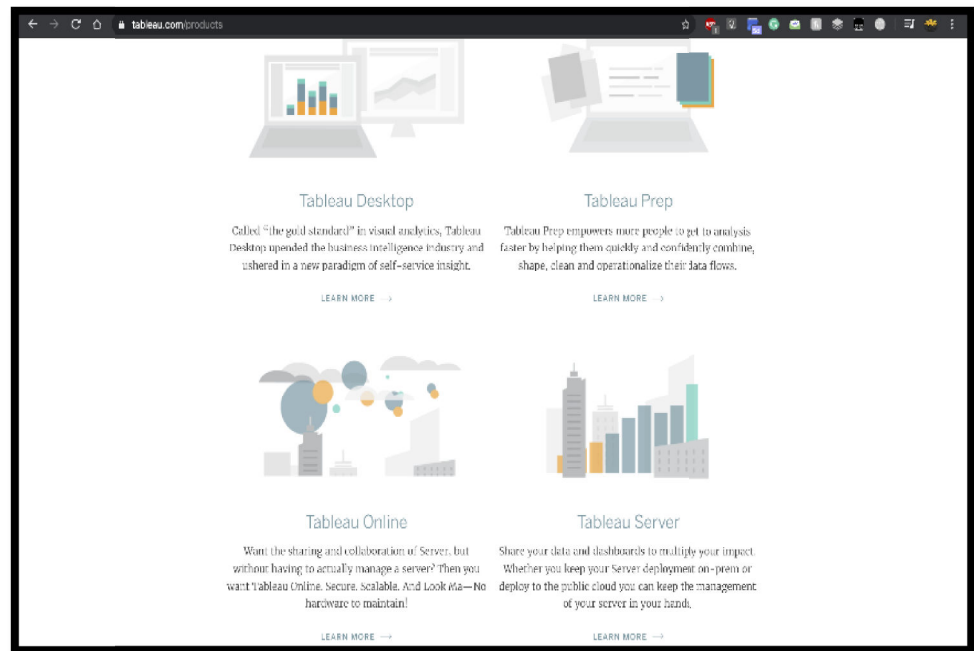


Fig.2: Tableau Desktop Products

Refer to Fig.2: Tableau Desktop Products, it covers **Tableau Desktop** and **Tableau Prep**, **Tableau Server** and **Tableau Online** as its major components, a brief description of these components is as follows:

- **Tableau Desktop:** A standalone desktop application designed for individual users to create and analyze data visualizations.
- **Tableau Prep:** This includes two components—**Tableau Prep Builder**, is used to design and build data preparation workflows, while **Tableau Prep Conductor** allows organizations to schedule, manage, and track these workflows efficiently across teams.

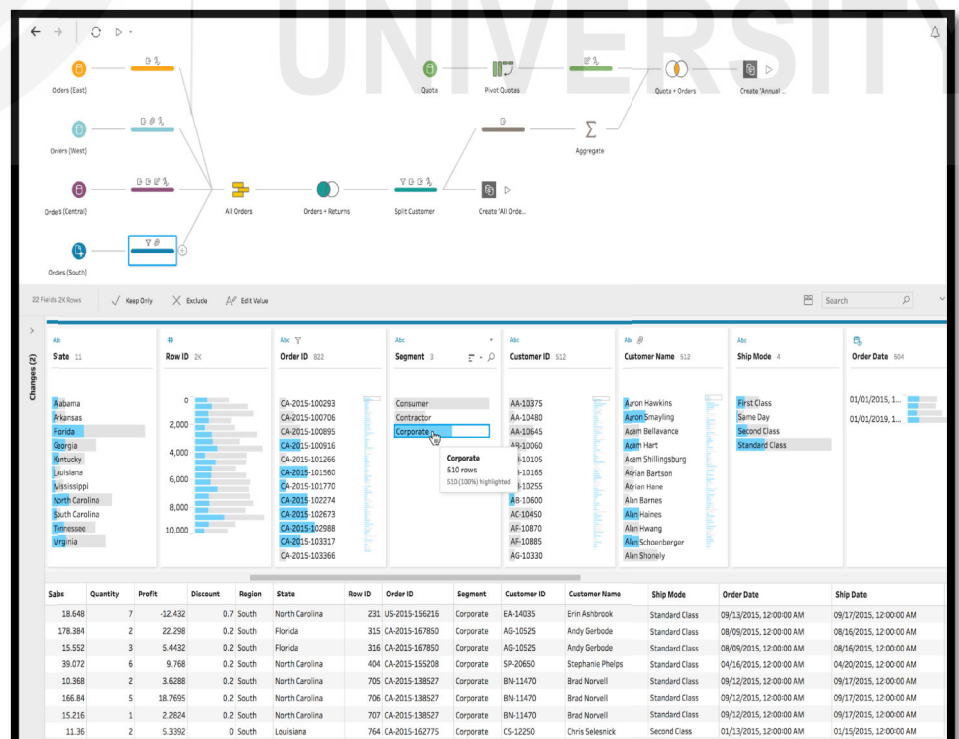
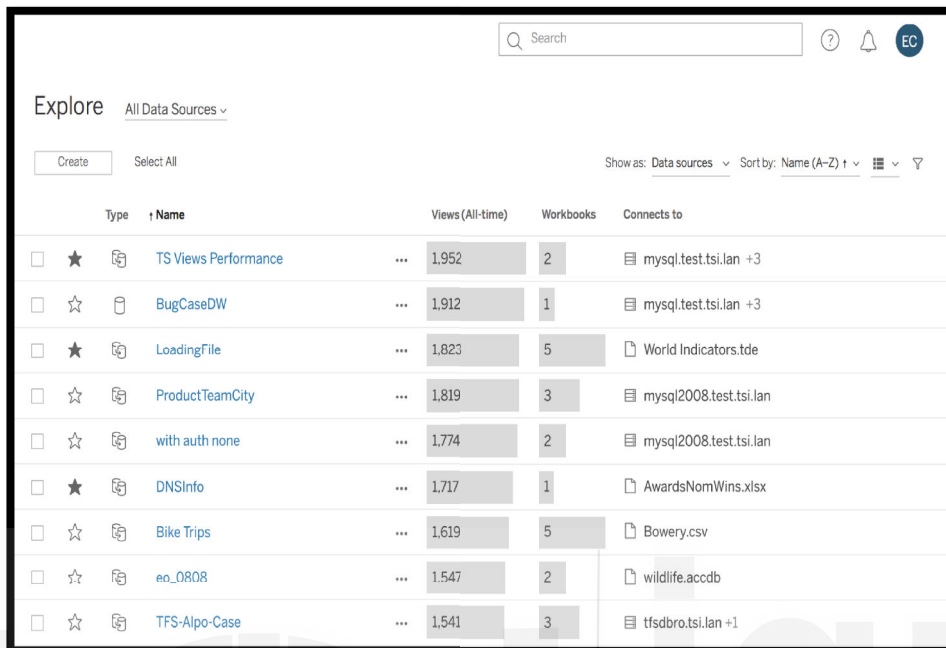


Fig.3: Tableau Prep

- **Tableau Server:** This is primarily designed for enterprise-level use, enabling organizations to securely share, manage, and collaborate on data visualizations across teams.



The screenshot shows the 'Explore' page in Tableau Server. At the top, there is a search bar and navigation links. Below, a table lists various data sources with columns for Type, Name, Views (All-time), Workbooks, and Connects to. The table is sorted by Name (A-Z).

Type	Name	Views (All-time)	Workbooks	Connects to
★	TS Views Performance	1,952	2	mysql.test.tsi.lan +3
☆	BugCaseDW	1,912	1	mysql.test.tsi.lan +3
★	LoadingFile	1,823	5	World Indicators.tde
☆	ProductTeamCity	1,819	3	mysql2008.test.tsi.lan
☆	with auth none	1,774	2	mysql2008.test.tsi.lan
★	DNSInfo	1,717	1	AwardsNomWins.xlsx
☆	Bike Trips	1,619	5	Bowery.csv
☆	eo_0808	1,547	2	wildlife.acddb
☆	TFS-Alpo-Case	1,541	3	tfsdbro.tsi.lan +1

Fig.4: Tableau Server

- **Tableau Online** is a public platform, so any content you upload can be accessed by other users of the software. As a result, your work is no longer private or confidential.

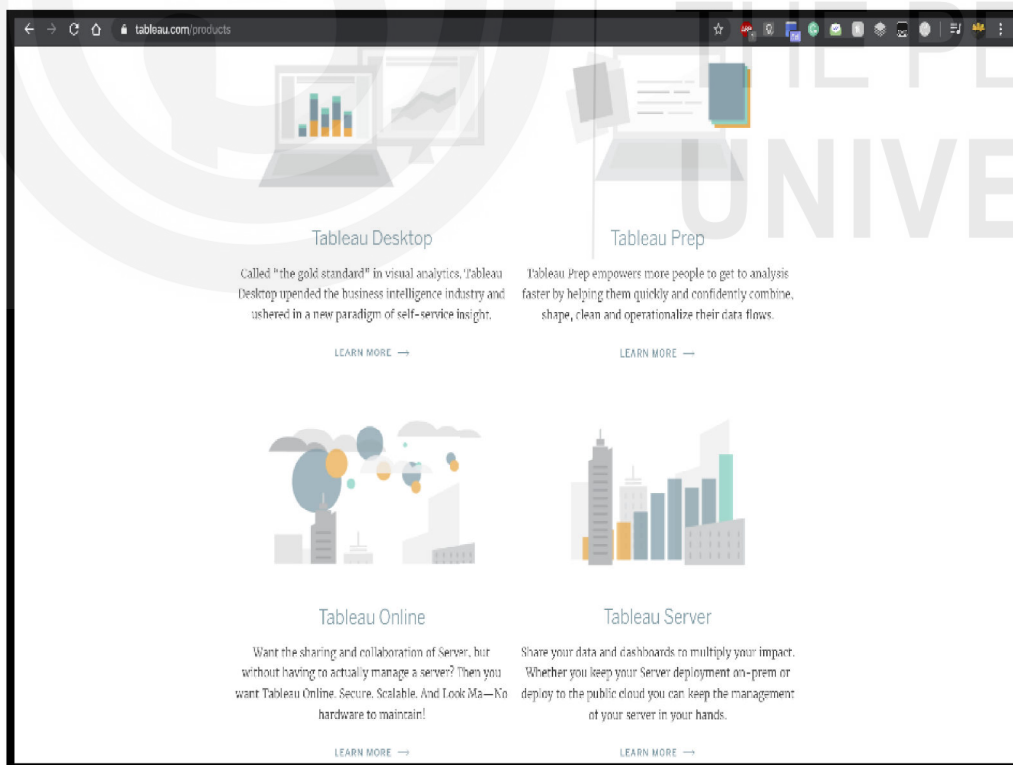


Fig.5: Tableau Online

You can Choose Tableau Desktop and start your free trial, The interface of Tableau Desktop is shown below in Fig.6.

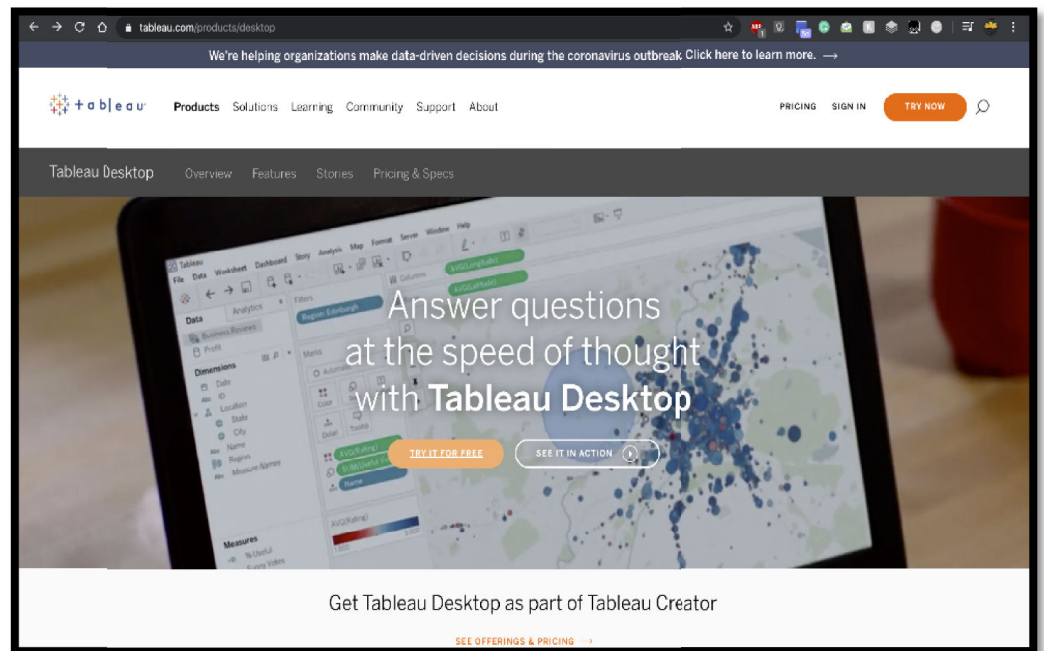


Fig.6: Tableau Desktop

Choose Tableau Desktop and to start your free trial, perform the following steps:

- Choose the software version that matches your system requirements and download the free trial.
- If you prefer not to use the trial, you can download Tableau Public instead. Once done, Tableau will be installed on your system.

Once Tableau is installed you will see the user interface as shown in Fig. 7, below

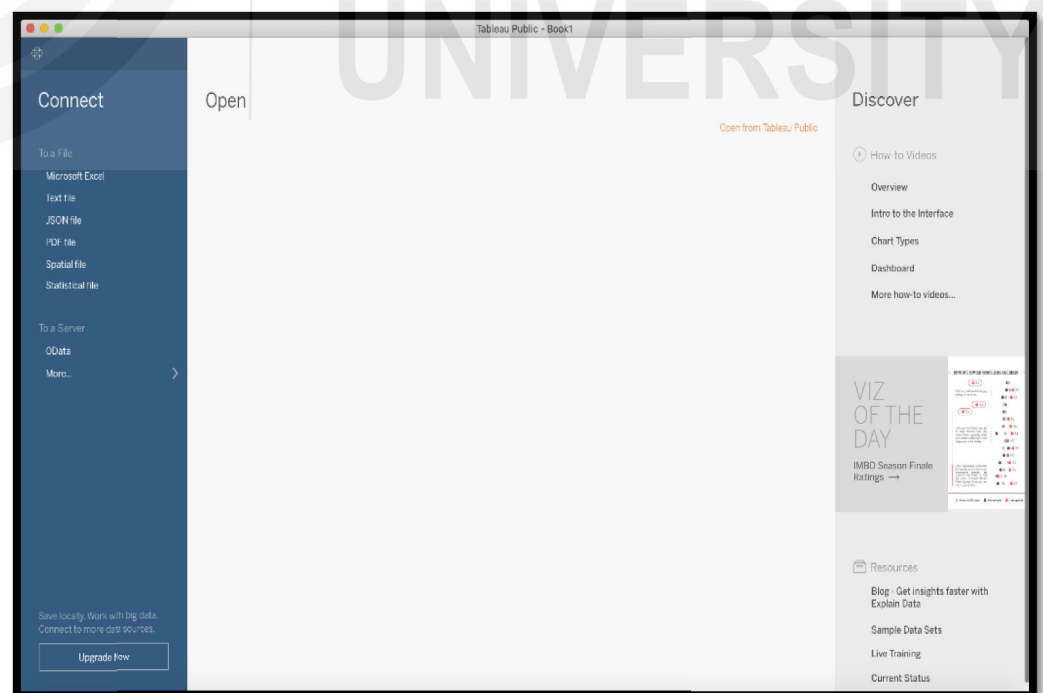


Fig.7: Tableau Installed

13.5.1 Interface of Tableau - Tableau's main interface window

The Tableau workspace consists of several components, including menus, a toolbar, the Data pane, shelves, cards, and one or more sheets. These sheets may be worksheets, dashboards, or stories as shown in Fig.8 below, but for detailed guidance on creating dashboards or working with stories, you can explore the sections dedicated to dashboards and the Story workspace.

If you're working with Tableau in a web browser, check out “Creators: Get Started with Web Authoring” and “Tour Your Tableau Site” for guidance.



Fig.8: Interface of Tableau

The components of the interface of Tableau are shown in Fig. 8 numbered from A to I, their respective elaboration and utility is given below:

- A. **Workbook Title** –A workbook contains multiple sheets, which may include worksheets, dashboards, or stories. For more information, see the section on *Workbooks and Sheets*.
- B. **Cards and Shelves** – These are areas where you can drag data fields to create and personalize your visualizations.
- C. **Toolbar** – The toolbar offers quick access to various commands, as well as tools for analysis and navigation.
- D. **View Area** – This is the primary workspace where your visualization, or "viz," is displayed.
- E. **Home Icon** – Click this button to go back to the Start page, where you can connect to new data sources. For more information, refer to the Start Page section [1].
- F. **Side Bar** –Located within worksheets, this section contains both the Data pane and the Analytics pane.
- G. **Data Source Tab** – Click here to navigate to the Data Source page, where you can view, explore, and manage your data. For more details,

refer to the Data Source Page section.

- H. **Status Bar** – Shows details and status of the current visualization or view.
- I. **Sheet Tabs** – Each tab represents a different sheet in the workbook, such as a worksheet, dashboard, or story. More info is available in *Workbooks and Sheets*.

13.5.2 Overview of Tableau Toolbar Buttons

While building or modifying a view, the toolbar located at the top provides quick access to frequently used tools and commands. In Tableau Desktop, you can choose to show or hide the toolbar by going to **Window > Show Toolbar**.

The table (Table 1: Tableau Toolbar) shown below describes what each button on the toolbar does. Note that certain buttons may not appear in every version of Tableau. You can also refer to *Visual Cues and Icons in Tableau Desktop* for additional details.

Toolbar Button	Description
Tableau Icon	Opens the Start Page (Desktop only).
Undo	Reverses the last action. Multiple undos are allowed.
Redo	Restores the last undone action.
Save	Saves the workbook (File > Save in Server/Cloud).
New Data Source	Connects to a new or existing data source.
Pause Auto Updates	Stops automatic updates when changes are made.
Run Update	Manually refreshes data (Desktop only).
New Worksheet	Adds a new worksheet, dashboard, or story.
Duplicate	Creates a copy of the current sheet.
Clear	Removes filters, formatting, or entire view elements.
Swap	Switches Rows and Columns fields.
Sort Ascending	Sorts data in increasing order.
Sort Descending	Sorts data in decreasing order.
Totals	Adds or removes grand totals or subtotals.
Highlight	Enables value highlighting in the sheet.
Group Members	Combines selected data into a group.
Show Mark Labels	Toggles visibility of data labels.
Fix Axes	Locks or unlocks axis range (Desktop only).
Format Workbook	Adjust workbook-wide formatting settings.
Fit	Controls how the view fits in the window.
Show/Hide Cards	Shows or hides worksheet cards.
Presentation Mode	Hides all except the view (Desktop only).
Download	Exports view/data in image, PDF, or CSV (Server/Cloud only).
Share	Publishes workbook to Server or Cloud (Desktop only).
Show Me	Suggests best visualizations for selected data.

Table 1: Tableau Toolbar

After understanding the user interface of Tableau, let's understand how to begin the work of data analysis, wherein the first step is to understand "How to upload any dataset?", next section relates to this only.

13.5.3 How to upload any dataset ?

The stepwise process of uploading any dataset, in Tableau is as follows:

Step – 1: Start Tableau on your system, you will see an interface as shown in Fig 9 below



Fig 9: Tableau Front Page Interface

Step – 2: "Navigate to the 'Connect' section, choose 'To a File', then click on file format you want to upload for an instance 'Microsoft Excel'.", as shown in Fig.10 below.

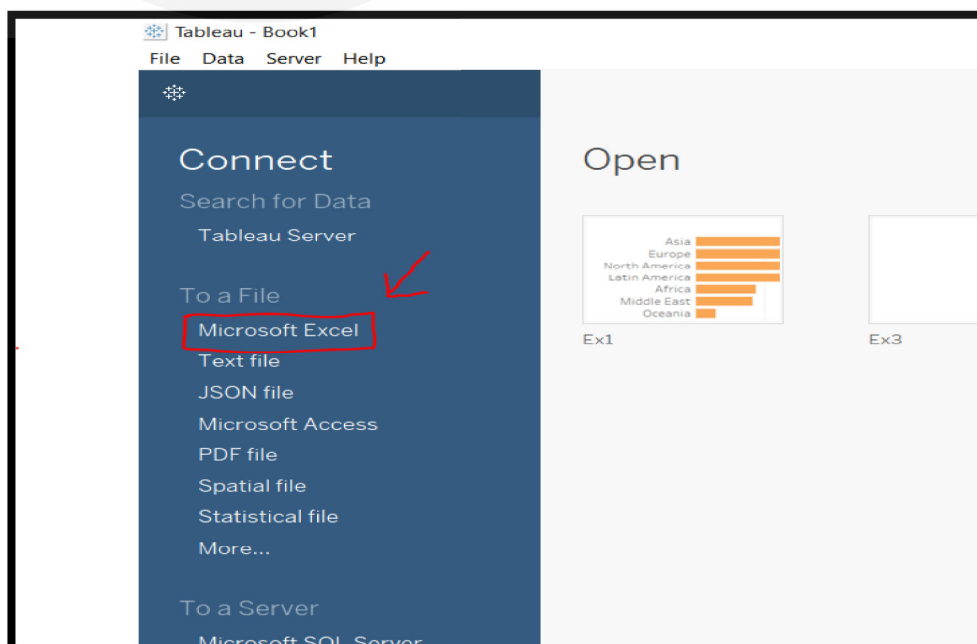


Fig 10: File Upload Interface in Tableau

Step – 3: Clicking this opens a file browser window, allowing you to navigate through folders and choose the file you wish to open, as shown in Fig. 11.

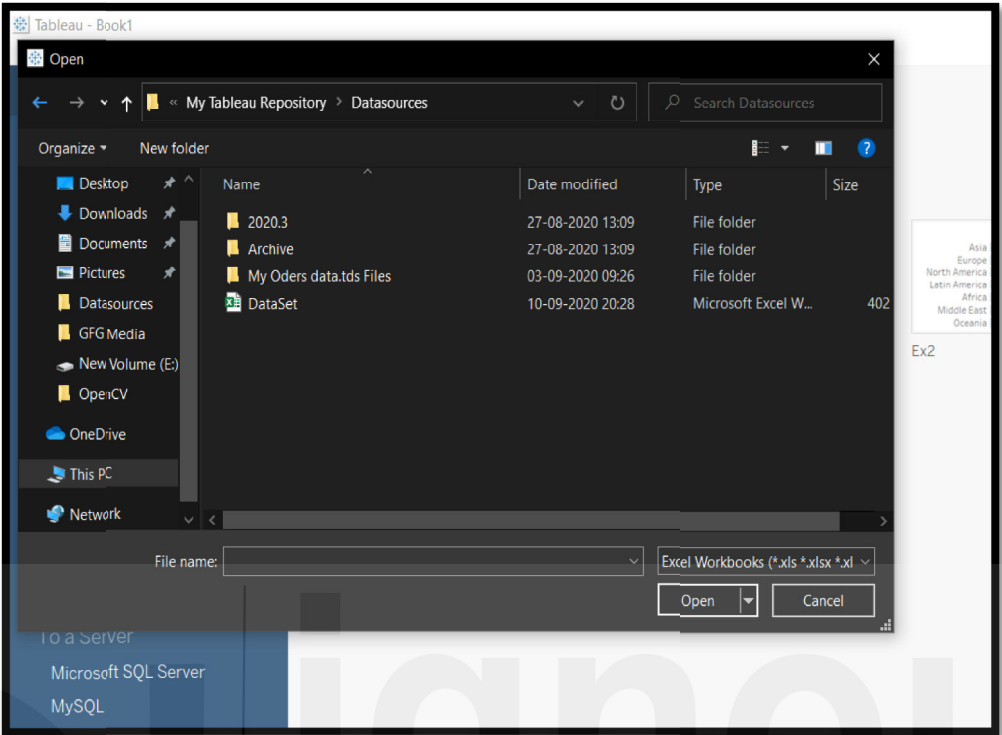


Fig 11: File Browser Interface in Tableau

Now, the sub-steps to be followed for the upload of dataset are as follows:

Step – 3(a): Select an appropriate file and open it, as shown in Fig. 12

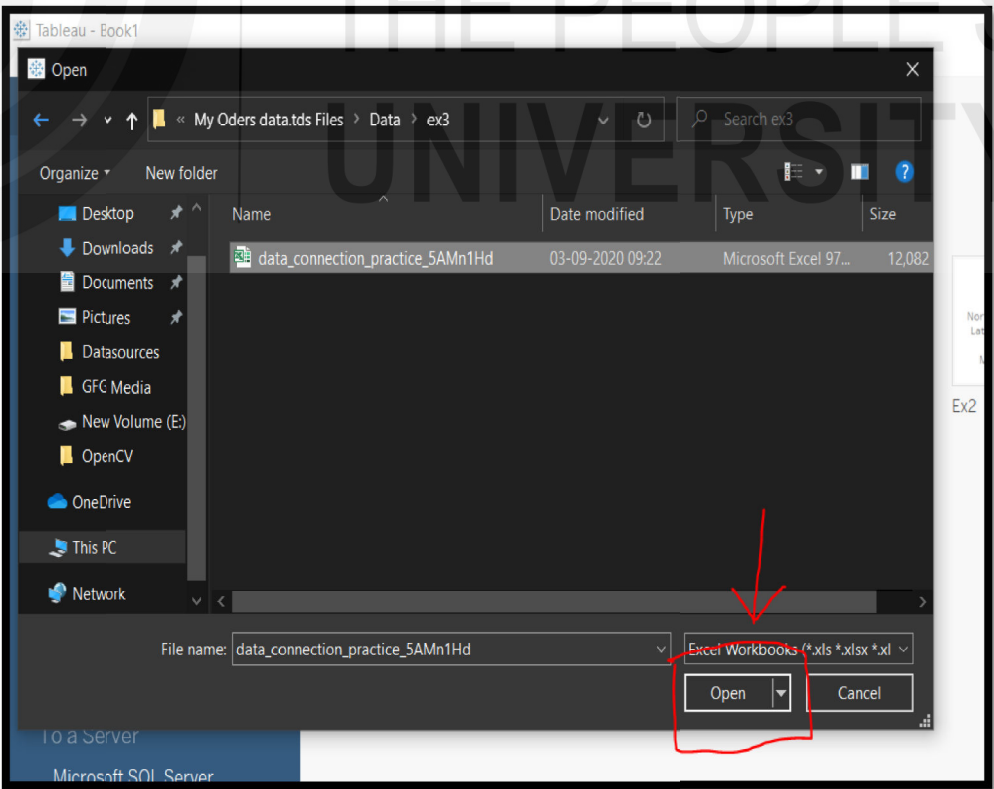


Fig 12: File upload in Tableau

Step – 3(b): Once the file is opened in Tableau, you'll see a screen displaying the connected data, as shown in Fig 13 below

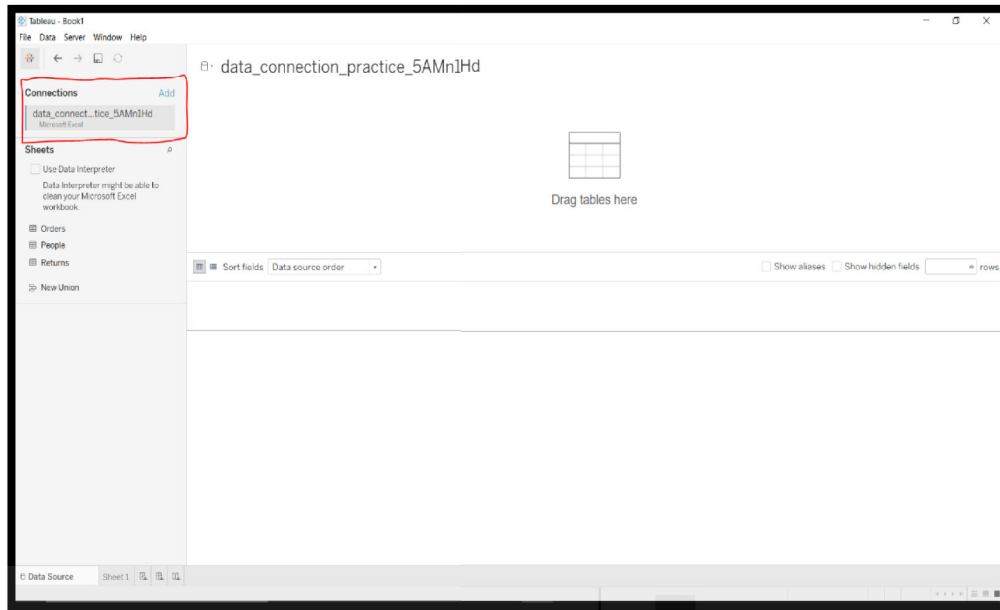


Fig 13: Data Connection in Tableau

Finally, Tableau displays the contents of your Excel file along with all its available sheets as shown in Fig 14,



Fig 14: Data Connection sheets in Tableau

By dragging and dropping a sheet, you'll be able to view its data and proceed with further tasks, like generating descriptive statistics and many more. The subsequent section relates to the generation of descriptive statistics.

13.5.4 Generating Descriptive Statistics - Statistical Analysis and Tableau

Statistical analysis is essential in business intelligence, as it enables organizations to explore data, identify trends, and gain meaningful insights. One key area—**descriptive statistics**—centers on summarizing data through metrics such as measures of central tendency (like mean and median) and variability (such as standard deviation).

Descriptive statistics form the starting point for understanding any dataset. It summarizes the main characteristics of numerical values by providing measures such as averages, ranges, and levels of variation. Tableau includes several tools that allow users to produce these statistics quickly and interpret them either numerically or visually. The following steps explain how descriptive statistics can be generated within Tableau in a practical and intuitive way, by using the Summary card.

A Summary Card in Tableau is a feature used to display quick statistical information about the data in a worksheet or about a selected field. It automatically shows key measures such as minimum, maximum, average, total (sum), and count of values, helping users quickly understand the overall distribution and range of the data. Summary cards can be applied to the entire view or to specific dimensions and measures, making them useful for quick data validation, comparison, and insight generation without performing manual calculations.

Let's learn how to use the Summary Card for generating descriptive statistics, Following are the steps to generate descriptive statistics in Tableau:

- 1) Load your dataset into Tableau and open a new worksheet.
- 2) Choose the numerical field you want to analyze and drag it to either the Rows or Columns shelf to create a basic view.
- 3) On the menu bar, select **Worksheet** → **Show Summary**.
- 4) After opening your worksheet, navigate to the *Worksheet* menu in the top-left corner and select **Show Summary**, as shown in Fig. 15 below.

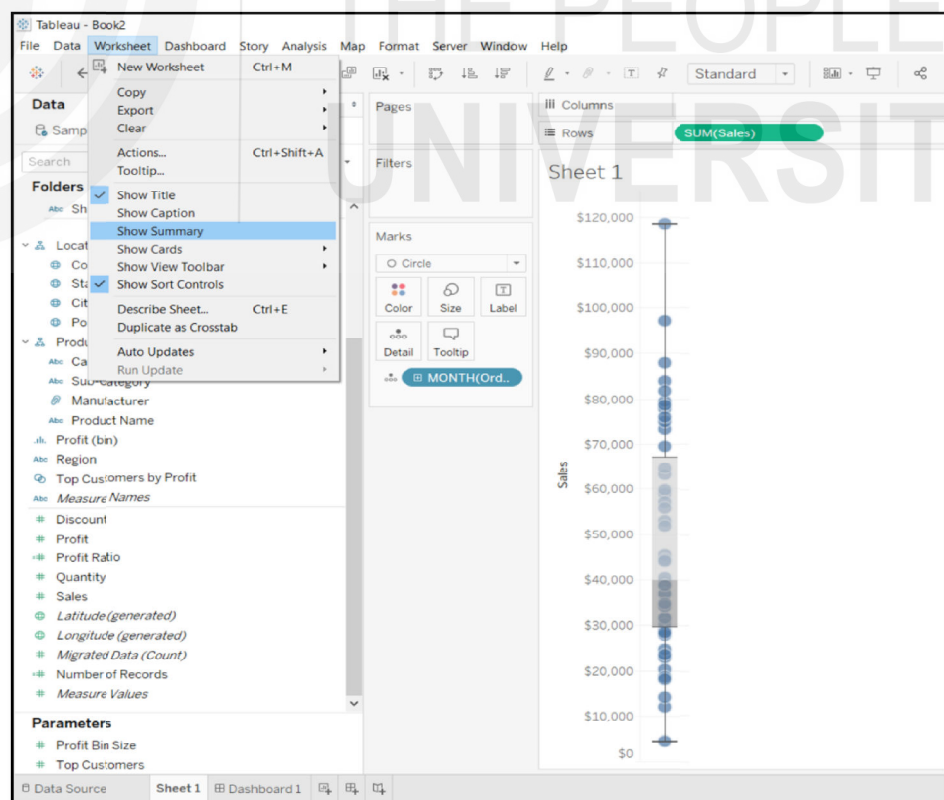


Fig 15: Descriptive Statistics in Tableau

Note: It is to be noted that, once enabled, the Summary Card automatically appears on the right side of the visualization, as shown in Fig. 16 below.

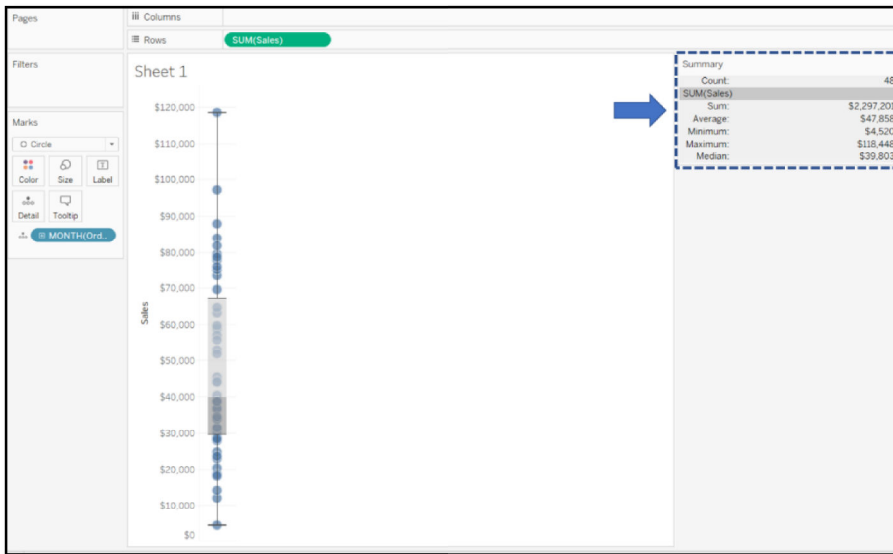


Fig 16: Summary of Descriptive Statistics in Tableau

Note: The Summary Card can be repositioned anywhere on the worksheet—whether to the right, left, top, or bottom—simply by dragging and dropping it as needed, as shown in fig. 17 below.

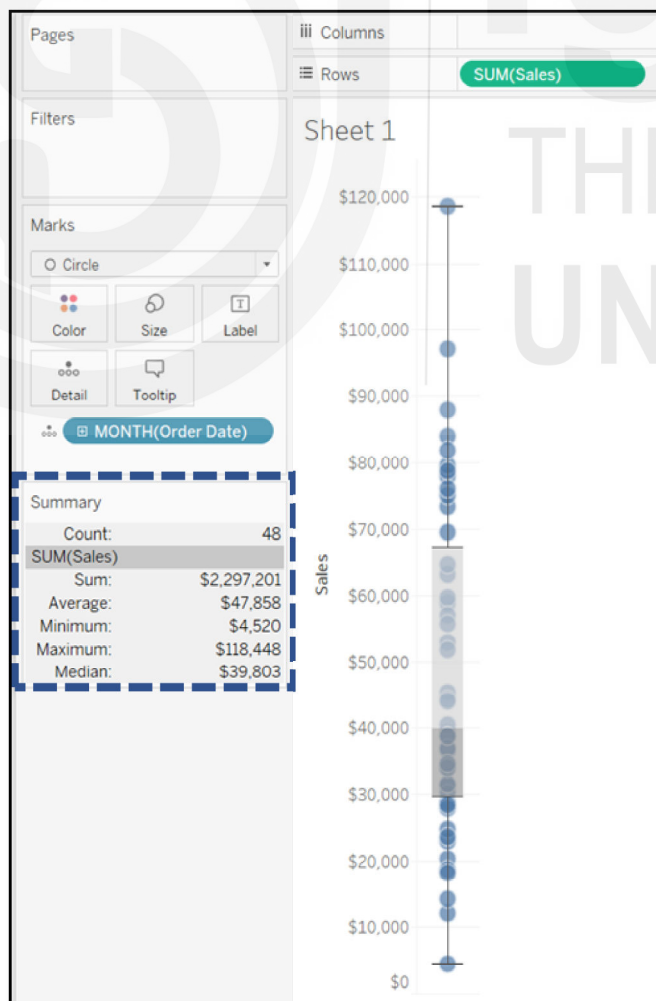


Fig. 17: Summary in Tableau

In short, Tableau provides an efficient way to generate descriptive statistics using the Summary Card. It offers clear, quick insights into key measures such as count, sum, average, minimum, and maximum values. By using the Summary Card, users can easily understand data behavior and obtain essential statistical information. This makes Tableau not only a visualization tool but also a practical platform for basic descriptive analysis.

Till the time we learned, how to upload the data set? and how to generate descriptive statistics? of the available dataset. Now, let's extend our discussion to the plotting of graphs and its analysis. The same is performed in next section.

13.5.5 Graph plotting and its analysis

Once the data has been uploaded, it can be used for visualization, which is the primary goal of using Tableau. This section focuses on demonstrating how to build a series of visualizations in Tableau. To perform visualization we need to **Connect to the database** in Tableau, in this case the referred data source contains **three separate sheets—Product, OrderDetails, and PropertyInfo**—all included within a single Excel file.

Below are the Steps to follow for visualization:

A) Opening Tableau and Connecting to Dataset

1. Launch **Tableau Desktop**.
2. On the **Start Page**, click on **Text file / Excel** depending on the file format.
3. Browse and select the **Product dataset file**.

Below is the structure of product table (Table-2):

Table 2: Product Table

ProductID	ProductName	ProductCategory	Price
1	Large Towel	Housekeeping	9
2	Hand Towel	Housekeeping	5
3	Washcloth	Housekeeping	3
4	Shampoo	Housekeeping	40
5	Moisturizer	Housekeeping	40
6	Conditioner	Housekeeping	40
7	Hand Soap	Housekeeping	35
8	Bath Soap	Housekeeping	35
9	Tissues	Housekeeping	14
10	Toilet Paper	Housekeeping	19
11	Shower Cap	Housekeeping	10
12	Bed Sheet (King)	Housekeeping	29
13	Bed Sheet (Double)	Housekeeping	20

4. Click **Open** → Tableau loads the data into the **Data Source window**.
5. Verify all column names and data types.

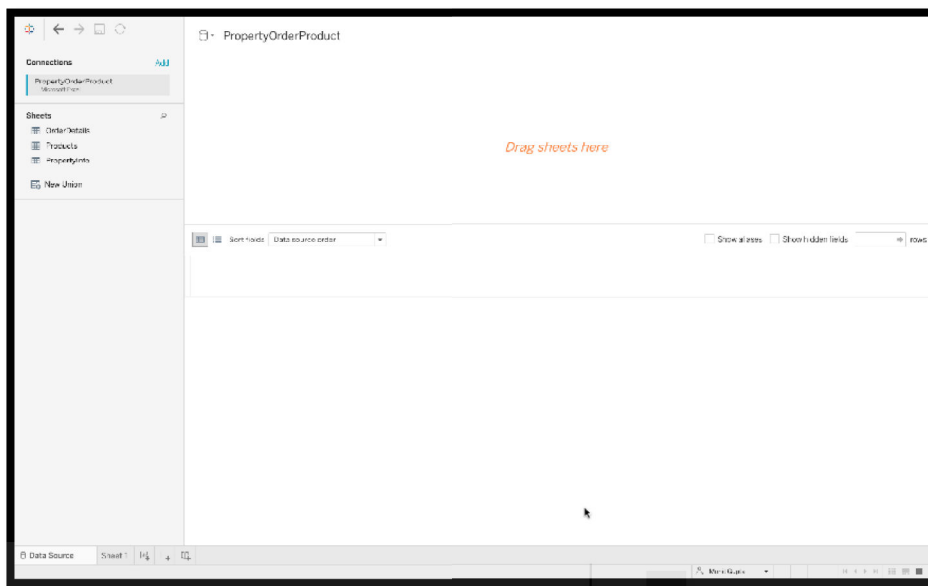


Fig. 18: Dataset Connection with Tableau

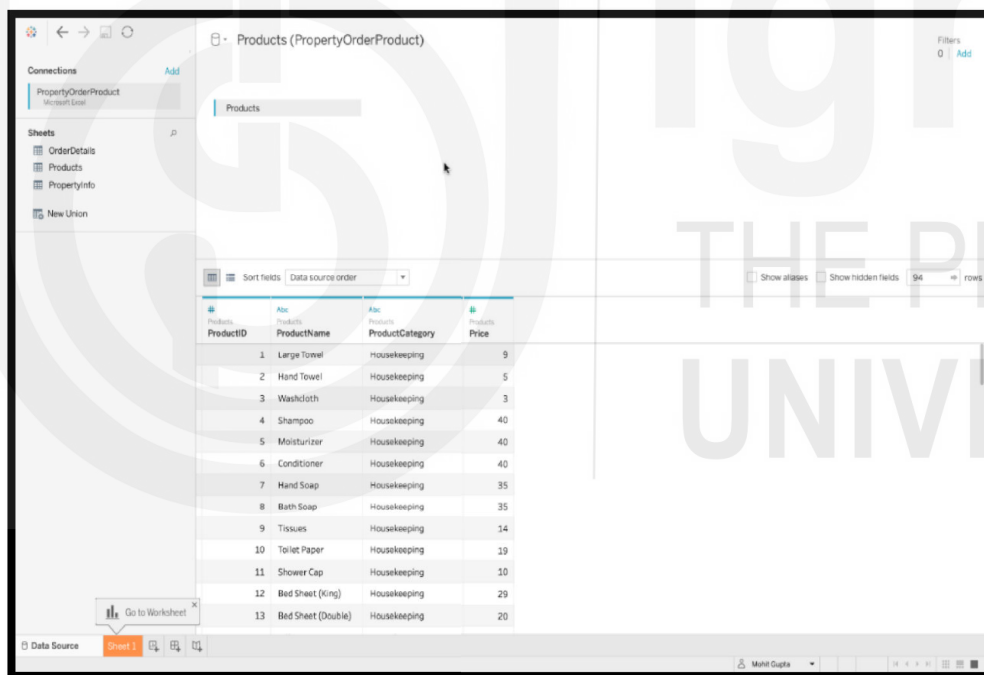


Fig. 19: Open Dataset Connection with Tableau

6. Click on **Sheet 1** to begin visualization.

B) Creating a Graph (Bar Chart: Category vs Price)

Step-by-Step Graph Plotting:

1. Drag **Product_Category** to the **Columns shelf**.
2. Drag **Price** to the **Rows shelf**.
3. From the **Marks card**, select **Bar**.
4. Tableau automatically plots a **bar graph showing the total price for each category**.

5. Drag **ProductName** to the **Color** option in the Marks card to differentiate products.
6. Click on **Show Mark Labels** to display price values on the bars.
7. Add a title such as: “*Category-wise Product Price Distribution*”

13.5.6 Dashboard Creation in Tableau is an important and interesting task, and to create a dashboard, at least two charts are required.

Here, we create the worksheets for the creation of Dashboard:

Worksheet 1: Category-wise Total Price (Bar Chart)

1. Drag Product_Category → Columns shelf.
2. Drag Price → Rows shelf.
3. From the Marks card, select Bar.
4. Click Show Mark Labels.
5. Rename the sheet as: “*Category-wise Price*”

Worksheet 2: Product-wise Price Comparison (Bar Chart)

1. Drag ProductName → Columns shelf.
2. Drag Price → Rows shelf.
3. Select Bar from Marks.
4. Click on Sort (Descending) by price.
5. Enable Show Mark Labels.
6. Rename the sheet as: “*Product Price Comparison*”

Creating the Dashboard

1. Click on the **New Dashboard** icon at the bottom of Tableau.
2. A blank dashboard page opens.
3. From the **Dashboard Pane (left side)**:
 - Drag “*Category-wise Price*” sheet onto the dashboard.
 - Drag “*Product Price Comparison*” sheet below or beside the first chart.
4. Adjust the size and alignment of both charts.
5. Set **Dashboard Size** to **Automatic** for responsive display.
6. Add a **Dashboard Title**, for example: “*Product Pricing Analysis Dashboard*”

Further, as an alternative you can create from a Dashboard in Tableau. On using following these steps.

- Open Tableau → Connect to Excel / CSV files
- Import the three tables into Data Source Product, OrderDetails, and PropertyInfo—all included within a single Excel file.
- Create manual joins using ProductID

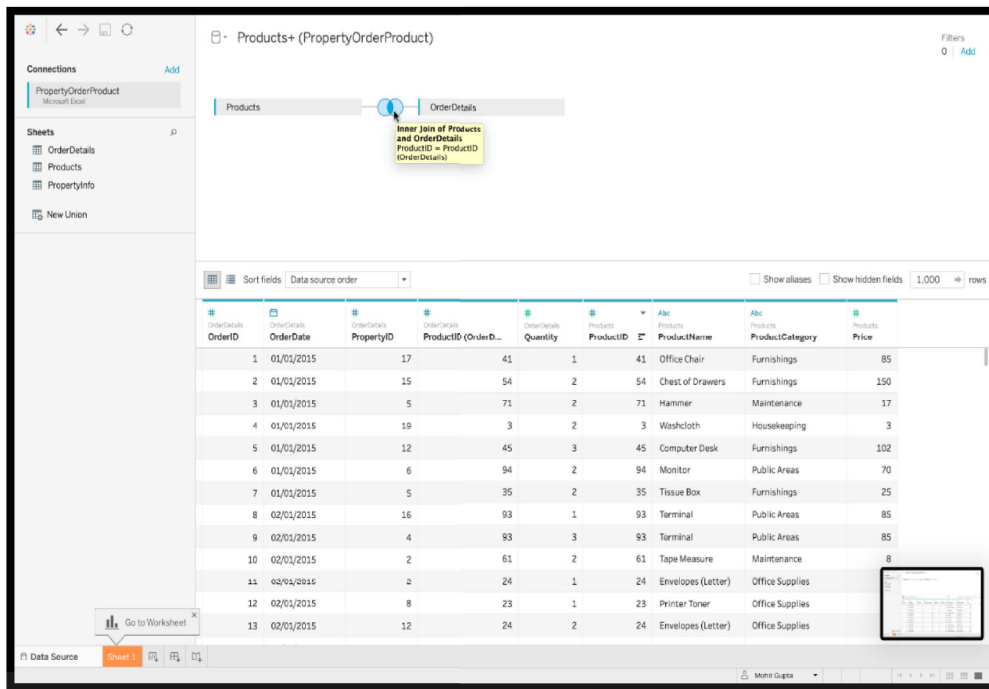


Fig. 20: Two Dataset Connection with Tableau

If you hover over the join, you'll see that an inner join has been created using a common field, namely *Product ID*. An inner join means that the two tables share a common column, allowing their data to be combined seamlessly. Next, add the *PropertyInfo* sheet, and you'll notice that it also gets joined automatically.

Note:

Tableau automatically detects relationships between tables based on matching field names. Since fields like **Product ID** share the same name across the connected sheets, Tableau recognizes them as common keys and creates the join without requiring manual input. This automatic behavior occurs only when field names are consistent across tables.

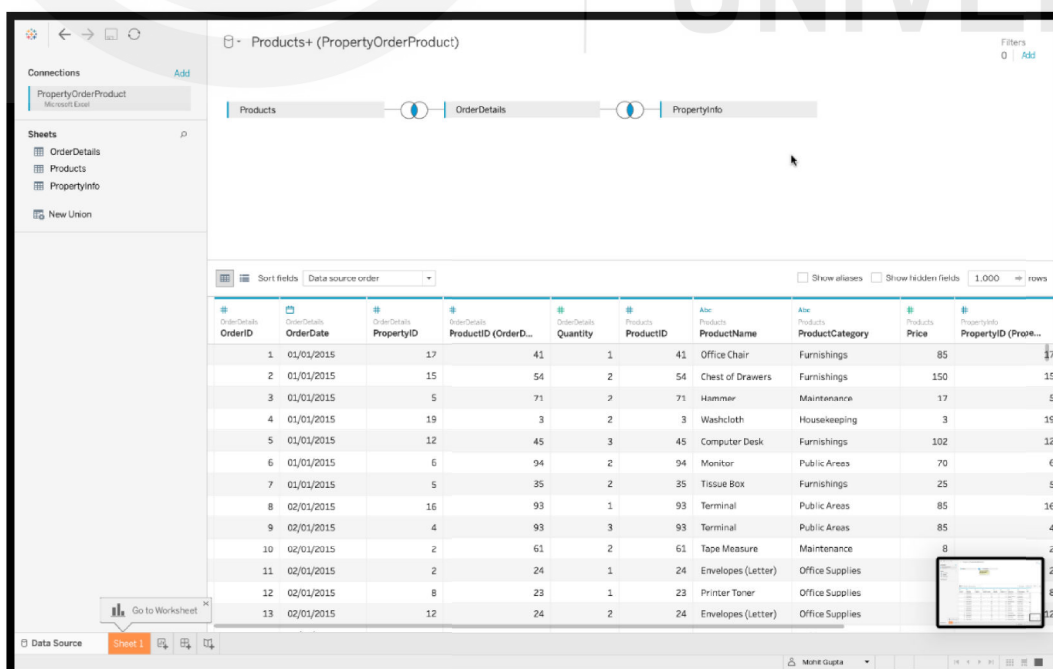


Fig. 21: ExcelSheet Dataset Connection with Tableau

Hovering over the join reveals that an inner join has been established between *OrderDetails* and *PropertyInfo* using *PropertyID* as the common key

OrderID	OrderDate	PropertyID	ProductID (OrderD...	Quantity	ProductID	ProductName	ProductCategory	Price	PropertyID (Prope...
1	01/01/2015	17	41	1	41	Office Chair	Furnishings	85	
2	01/01/2015	15	54	2	54	Chest of drawers	Furnishings	150	
3	01/01/2015	5	71	2	71	Hammer	Maintenance	17	
4	01/01/2015	19	3	2	3	Washcloth	Housekeeping	3	
5	01/01/2015	12	45	3	45	Computer Desk	Furnishings	102	
6	01/01/2015	6	94	2	94	Monitor	Public Areas	70	
7	01/01/2015	5	35	2	35	Tissue Box	Furnishings	25	
8	02/01/2015	16	93	1	93	Terminal	Public Areas	85	
9	02/01/2015	4	93	3	93	Terminal	Public Areas	85	
10	02/01/2015	2	61	2	61	Tape Measure	Maintenance	8	
11	02/01/2015	2	24	1	24	Envelope (Letter)	Office Supplies	24	
12	02/01/2015	8	23	1	23	Printer Toner	Office Supplies	85	
13	02/01/2015	12	24	2	24	Envelope (Letter)	Office Supplies	24	

Fig. 22: ExcelSheet Dataset Connection with Tableau

Now, that data is completely ready for data visualization through dashboard

➤ Go to **Sheet 1** → Create charts:

Histogram / Bar chart for Sales Distribution

Box Plot via **Analytics Pane**

- Create additional visuals if required (e.g., Region-wise sales)
- Select **Dashboard** → **New Dashboard**
- Drag visualizations into the layout and adjust formatting
- Add filters or legends for interactivity

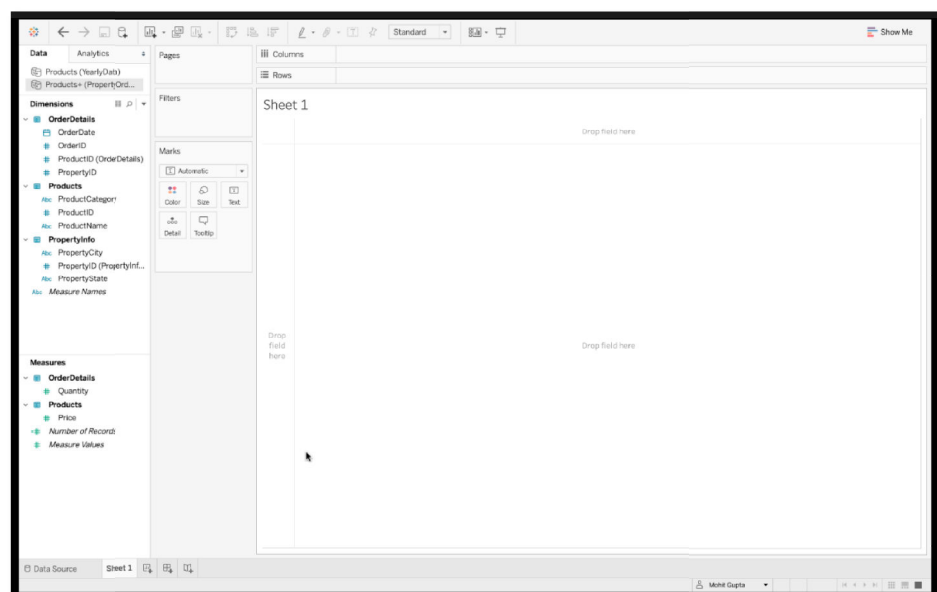


Fig. 23: Excel Sheet Dataset Connection with Tableau

Choose **Quantity**, then navigate to the top-right corner of the toolbar and click the **Show Me** button to generate a visualization.

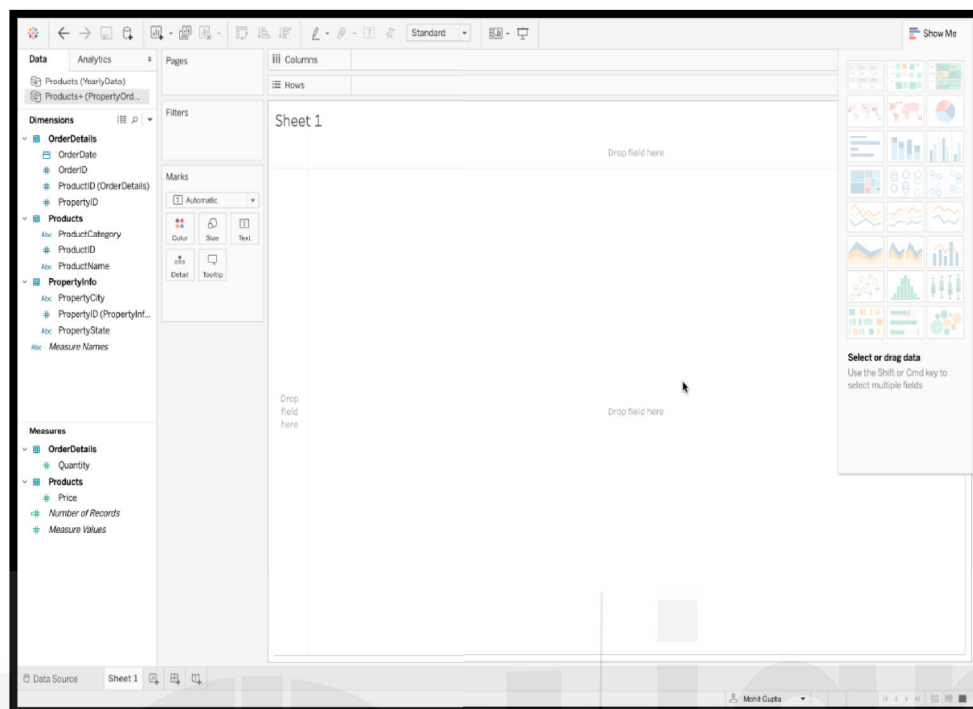


Fig. 24: Excel Sheet Quantity Dataset Connection with Tableau

Go ahead and click on **Text Table** to display the data in a tabular format.

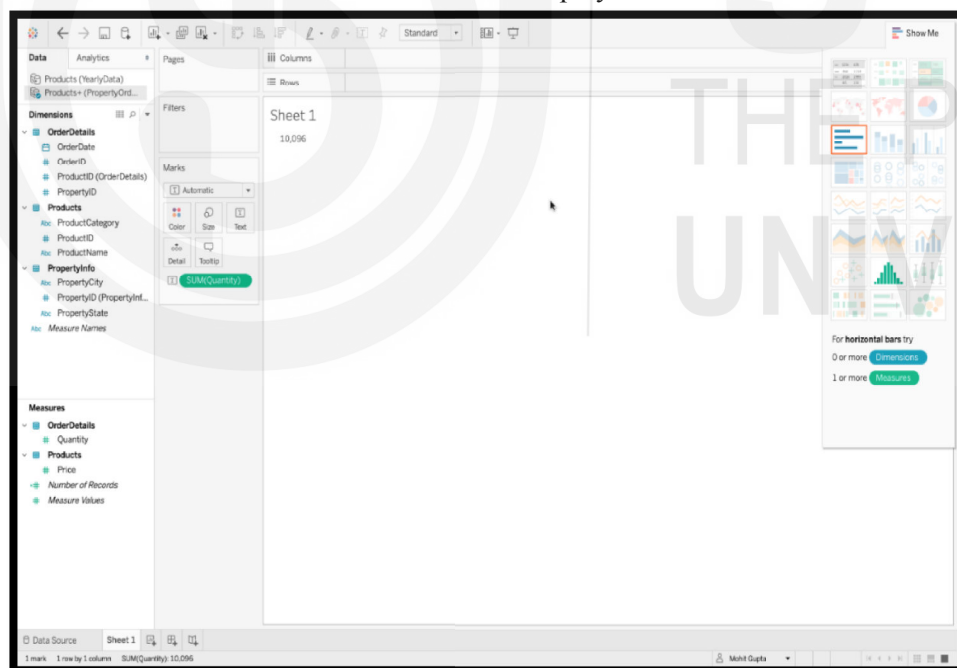


Fig. 25: Text table Dataset Connection with Tableau

The current view displays only one measure (the total quantity), which adds up to 10,096. With no dimensions or categories to break it down, the visualization isn't very insightful. Let's enhance it by adding additional fields to generate a more detailed and interesting chart.

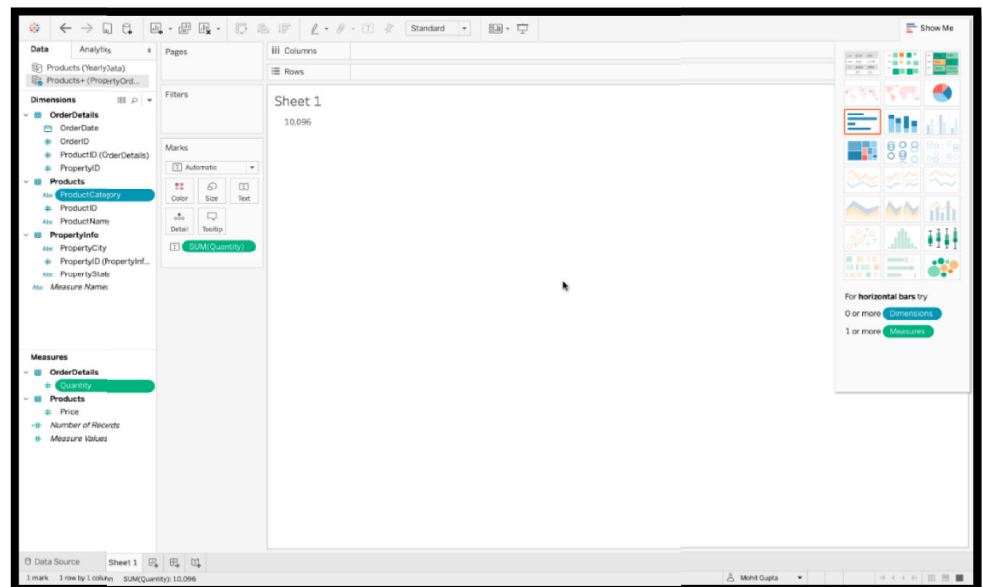


Fig. 26: Text table Dataset Connection with Tableau

To select multiple non-adjacent fields, press and hold the Ctrl key (or Command key on Mac), then click on each one individually. For example, select both *Quantity* and *Product Category*. With these fields selected, Tableau will now suggest a broader range of visualization options.

Now that both Product Category and Quantity are in the view, head to the Show Me panel and select the Tree Map option. This chart uses both color and size to visually represent values, making trends easy to identify.

The Tree Map clearly shows that Furnishings has the highest number of items ordered. Public Areas and Housekeeping follow closely, with Maintenance next, and Office Supplies having the lowest quantity.

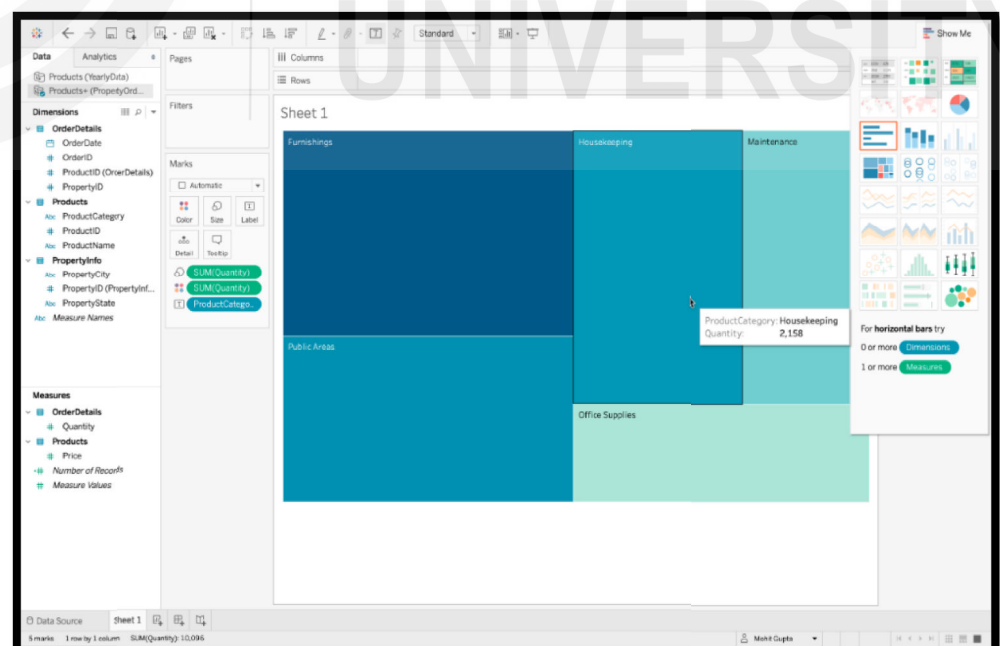


Fig. 27: Tree Map Dataset Connection with Tableau

At first glance, *Maintenance* and *Office Supplies* appear to be nearly the same size in the Tree Map. However, when you revisit the **Show Me** tab and

explore the visual more closely, it becomes clear that *Maintenance* has a **slightly higher quantity compared to Office Supplies**.

This illustrates the fundamental steps for quickly creating visualizations in Tableau using the options provided in the **Show Me** panel.

13.5.7 SUM UP: Tableau - Data handling and User Interface

1. Introduction to Tableau Interface

- **Workspace:** Consists of Data Pane, Shelves (Rows, Columns), and Worksheets.
- **Marks Card:** Customize visuals (color, size, label).
- **Dashboard Tab:** Combine visuals.
- **Connect Pane:** Connect to CSV, Excel, SQL, etc.

2. Uploading the Dataset

- Launch Tableau Desktop.
- Under "Connect", select CSV/Excel/Database.
- Browse and upload your dataset.
- Drag tables into the workspace if needed.

3. Handling Data

- Use **Data Interpreter** for auto-cleaning.
- Create **Calculated Fields** for new variables.
- Use **Filters**, **Groups**, and **Bins** to organize data.

4. Plotting Graphs

- Drag fields to Rows and Columns.
- Use Show Me panel to choose chart types (bar, line, scatter, pie, maps).
- Build dashboards combining multiple visuals.

5. Summary : Generating Descriptive Statistics

Operation	Tableau Feature Used
Median, Quartiles & Outliers	Box Plot from Analytics Pane
Summary Statistics	Show Summary (Worksheet tab)
Distribution Pattern	Histogram + Distribution Bands
Quick statistical insights	Summary Card (can be dragged anywhere on screen)

Table 3: Brief of generative statistics

These components help quickly identify:

- Data spread and center

- Skewness in sales
- Outlier products with extreme performance

6. Final Output (Expected)

A **dashboard** visually showing:

- Sales distribution chart
- Box plot with median & quartiles
- Optional supplier/region filter
- Summary card displaying statistics like:
 - ✓ Count
 - ✓ Mean
 - ✓ Minimum
 - ✓ Maximum

☞ Check Your Progress - 1

1. What are the key components of the Tableau workspace?

.....

.....

.....

.....

2. How do you connect to a Microsoft Excel file in Tableau?

.....

.....

.....

.....

3. What is the purpose of the 'Show Me' panel in Tableau?

.....

.....

.....

.....

4. How can you calculate the median in Tableau?

.....

.....

.....

.....

5. How are outliers identified using Tableau?

.....

.....

.....

13.6 POWER BI

Power BI is a Microsoft application built to convert raw data into valuable insights.

It enables users to create interactive dashboards, reports, and charts to simplify data analysis and understanding. Whether you're in business, research, or any data-driven field, Power BI helps you identify patterns, track trends, and make informed decisions more efficiently.

The process involves three main steps:

1. **Power Query Editor** – Prepare and clean your data.
2. **Data Modeling and Relationships** – Establish connections between data tables and organize your data.
3. **Visualization** – Build interactive graphs and charts to explore and showcase your data efficiently.

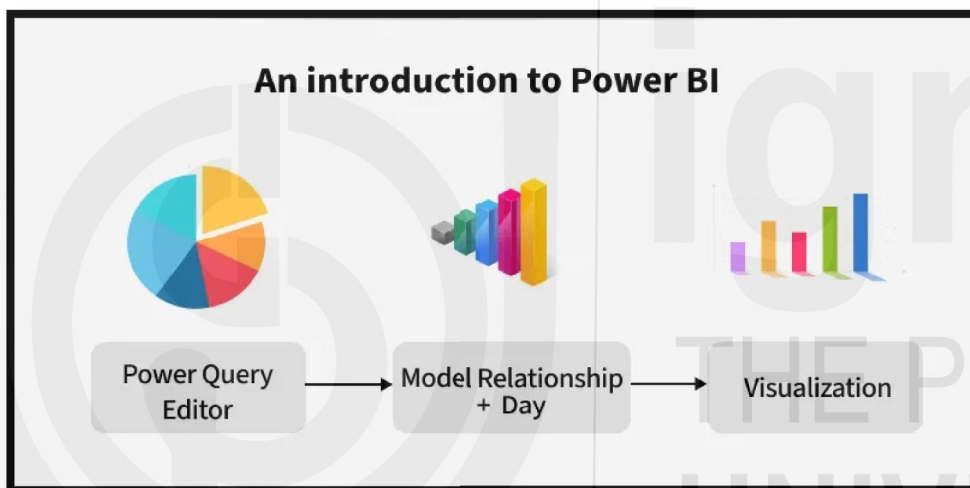


Fig.28: Getting Started with Power BI

Working with Power BI is straightforward, it follows the easy steps, shown in Fig. 29 below

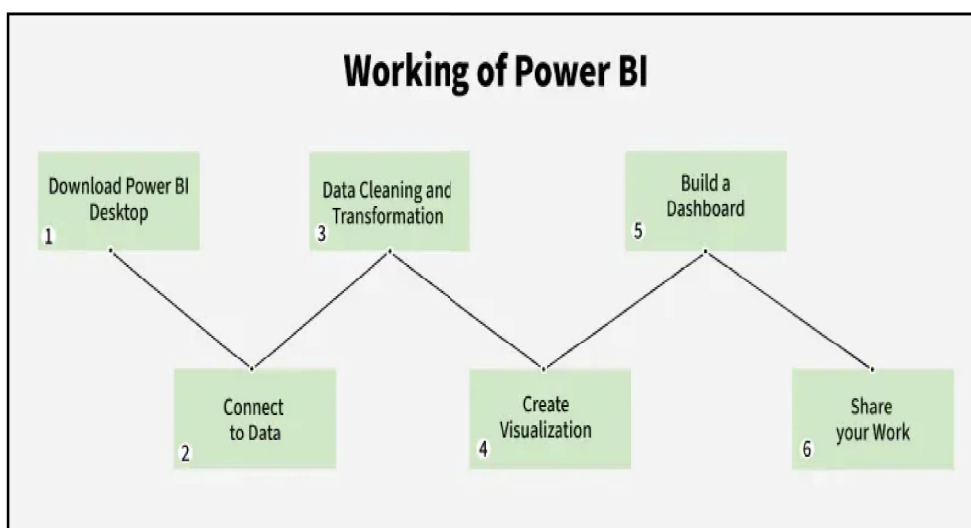


Fig.29: Working of Power BI

Step-by-Step procedure to work by Using Power BI is as follows :

Step 1: Download and Install Power BI Desktop: Go to Microsoft's official website and download Power BI Desktop for free. Install it on your Windows system. This version enables you to create dashboards and reports. After installation, you'll get a clean interface to start working with your data.

Step 2: Connect to Your Data: Open Power BI and click on “Get Data.” You can connect to a variety of data sources such as Excel files, SQL databases, or cloud services like Google Analytics and Salesforce. Select your desired source and load the data. You can also combine data from multiple sources into one report.

Step 3: Clean and Prepare Your Data: Data often needs cleaning before analysis. Use **Power Query** to remove unnecessary rows, correct data formats, and handle missing values. This ensures your dataset is accurate and ready for analysis.

Step 4: Create Visualizations: Drag data fields into the report canvas to start building visualizations. Power BI will automatically generate charts, tables, or graphs based on the data. Choose visual types that best represent your insights. Visuals help in identifying patterns and key metrics quickly.

Step 5: Build a Dashboard: Arrange multiple visuals into a single-page dashboard. Dashboards allow users to explore different data perspectives by interacting with the visuals. You can customize layouts to highlight important information and enhance the user experience.

Step 6: Share Your Work: After completing your report or dashboard, publish it to the **Power BI Service** (online platform). This allows you to share your insights with others, enabling collaboration and easy access from anywhere others can access and view it at any time. You can also set up automatic data refreshes to keep your reports up to date, ensuring they always display the most recent information.

Using Power BI - Power BI is mainly used to create interactive reports and dashboards that provide deeper insights into your data. After publishing your report to the Power BI service, both you and other users can explore and interact with it while tracking its performance. This guide highlights the essential steps for building and modifying reports and dashboards.

Essential Steps:

1. **Extract the Data** – Begin by importing your data from various sources.
2. **Transform the Data** – Use Power Query to clean, modify, and prepare your data, as well as to manage and adjust relationships between tables as needed.
3. **Apply Calculations with DAX** – Use Data Analysis Expressions (DAX) to perform calculations on your dataset.
4. **Start Visualizing** – Once the data is prepared, proceed to build your visualizations.

5. **Create and Customize Visuals** – Add elements like charts, graphs, and cards, and modify them for clarity and effectiveness.
6. **Publish to the Cloud** – Upload your dashboard to the Power BI service so your team can access and interact with it.

Loading Data - To start handling data in Power BI, select the **‘Get Data’** option under the **Home** tab. This allows you to connect to a wide range of data sources, including:

- **Files** – Import data from formats like Excel or CSV.
- **Databases** – Establish connections to databases like SQL Server, Oracle, or MySQL.
- **Direct Query** – Access data directly from the source without importing it.
- **Online Services** – Connect with platforms like Google Analytics, Salesforce, or SharePoint.
- **Live Connection** – Establish a real-time link to data sources for always up-to-date information.

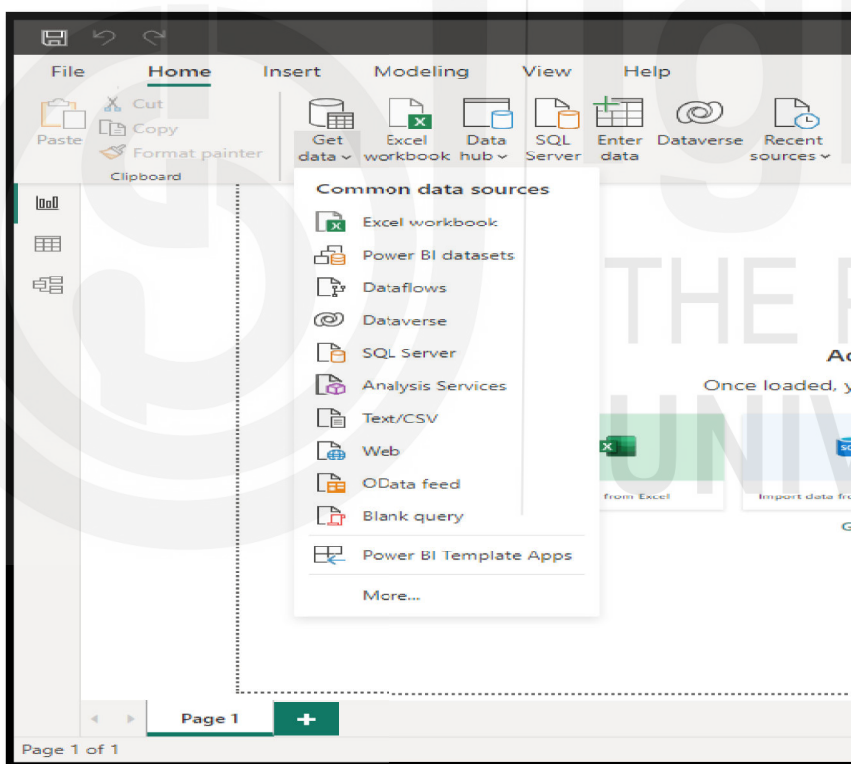


Fig.30: Using Power BI

After importing the data into the application, you can begin transforming and modeling it. You can create new columns with custom configurations and define your own calculated fields.

Before building meaningful dashboards and analytical reports, it is essential to prepare and structure the imported data. Raw data often contains missing fields, formatting inconsistencies, or lacks the calculated values required for analysis. Power BI provides powerful tools for transforming, enriching, and organizing data so that it aligns with analytical and business requirements.

Power BI enables users to enhance the existing dataset by creating **calculated columns** and **measures**. These user-defined fields help implement business logic, perform advanced calculations, and support accurate data insights that may not be directly available from the source. This process forms the core of **data modeling**, which ensures that the data is structured correctly for visualizations and interactions. The steps to create **calculated columns** and **measures** are listed below:

Step 1: Begin by clicking on the **"Data"** icon from the three options available in the left-hand panel.

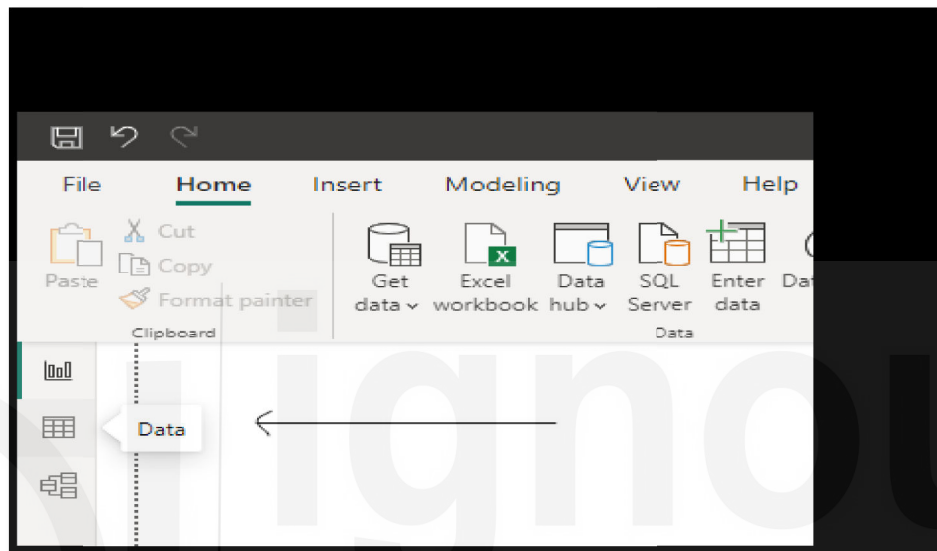


Fig.31: Data Tab Power BI

Step 2: In the **"Table Tools"** tab, go to the **"Calculation"** group and choose **"New Measure"** to create a custom calculation.

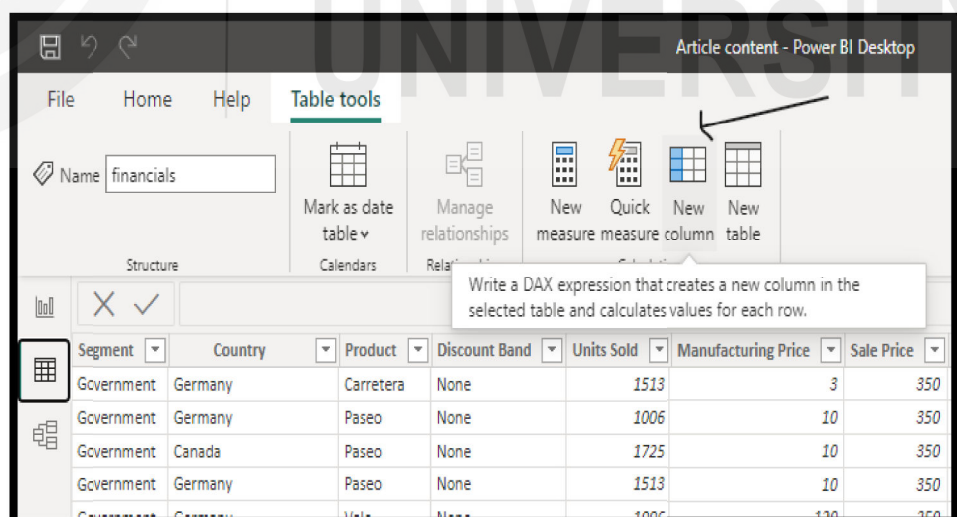


Fig.32: Table Tools Power BI

Step 3: Next, use the desired functions along with the table field names to define your logic and generate a new column.

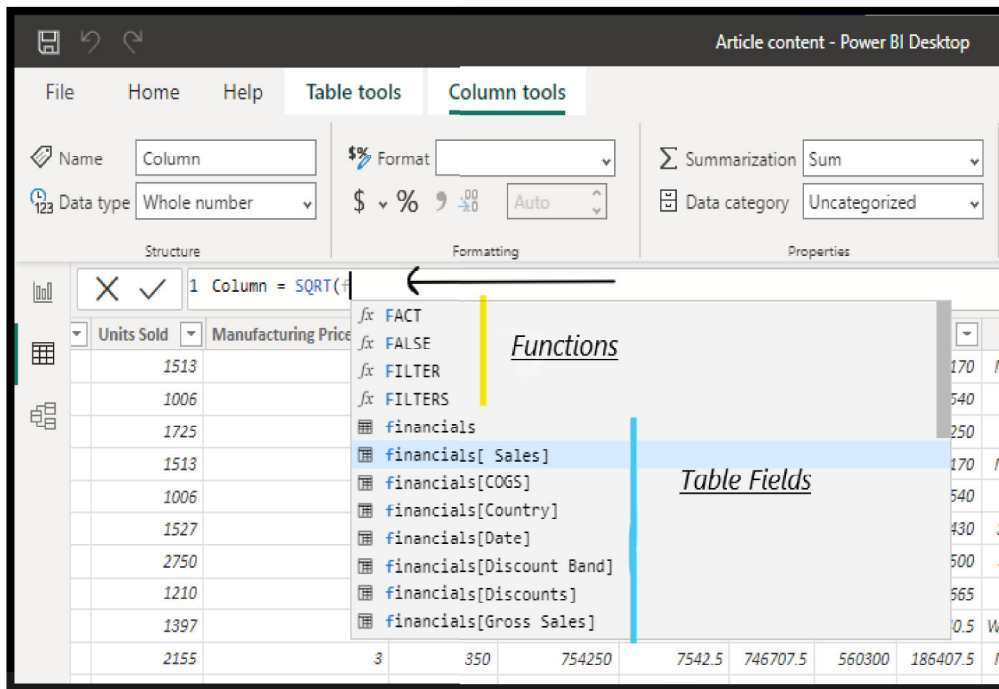


Fig.33: Column Tools Power BI

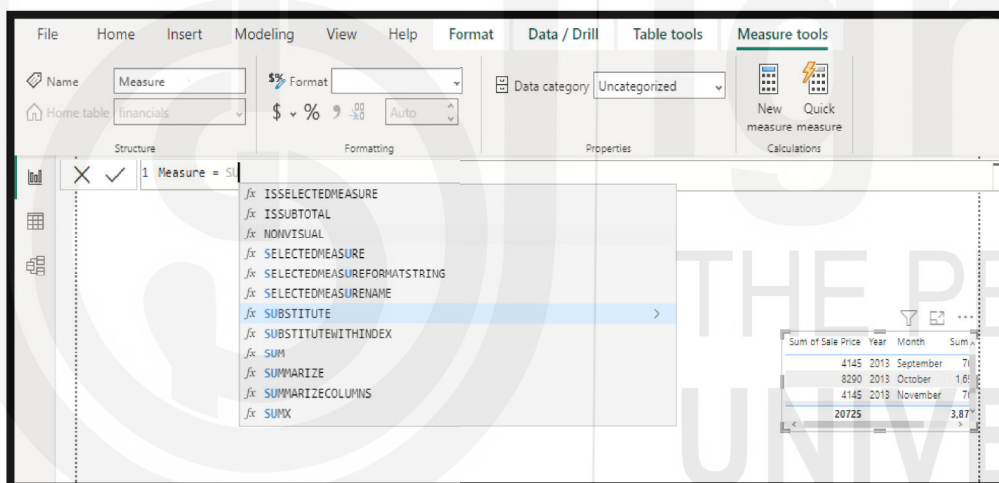


Fig.34: Measure Tools Power BI

Step 4: Once this step is finished, your required columns or measures will be set up. Use the **"New Column"** option when you need the column to remain part of the table and update automatically during data refreshes, as it becomes a permanent part of the data model. On the other hand, **measures** are calculated dynamically and function more like real-time filters—making them more efficient since they aren't stored in memory.

Power BI also offers a built-in **Power Query Editor** that supports advanced data cleaning and transformation, ensuring your data is well-prepared for analysis and visualization .

Power Query - Power Query is a robust tool that allows you to shape and transform your data through operations such as filtering, grouping, and pivoting. It also supports the creation of calculated columns and measures using the formula bar and a wide range of functions. After the data is cleaned and prepared, you can move on to creating visualizations. Following are the steps to be performed to prepare the data for its visualization.

Step 1: Click on the **"Home"** tab from the top navigation menu.

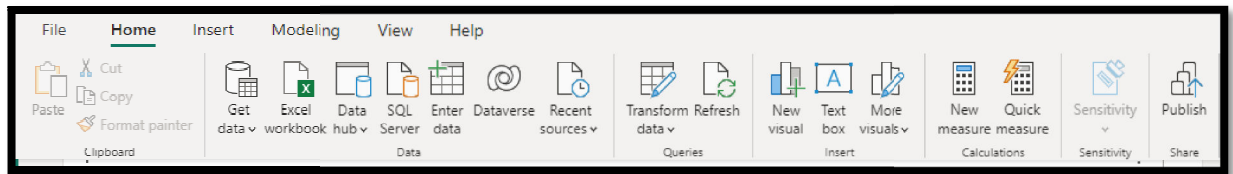


Fig.35: Home Tab Power BI

Step 2: In the **Queries** section, click on **"Transform Data."** (This action will launch the Power Query Editor in a new window. If no data source is connected yet, a blank screen will be displayed.)

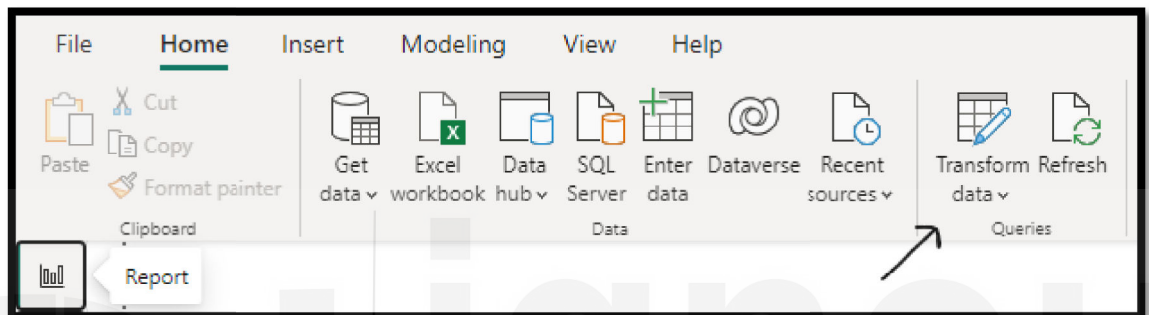


Fig.36: Transform Data Tab Power BI

Step 3: On this screen, you can view, edit, transform, and clean your data as required.

The **Home** tab provides multiple tools for managing your data sources.

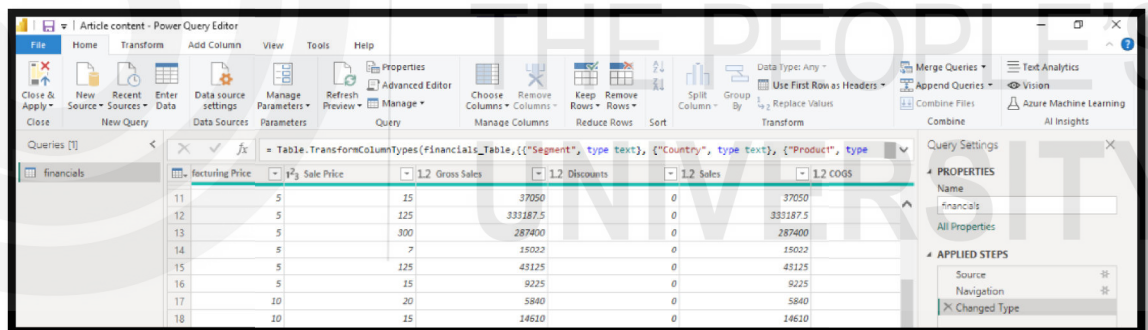


Fig.37: Power Query Editor in Power BI

The **"Transform"** tab contains functions for adding, deleting, splitting, and changing data types.

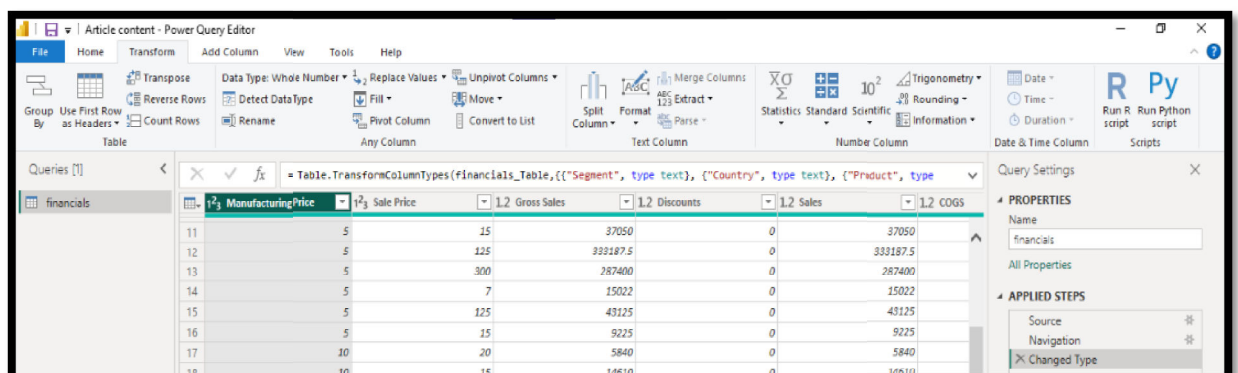


Fig.38: Power Query Editor Transform Tab

"The 'Add Column' tab allows you to add new columns and format them accordingly."

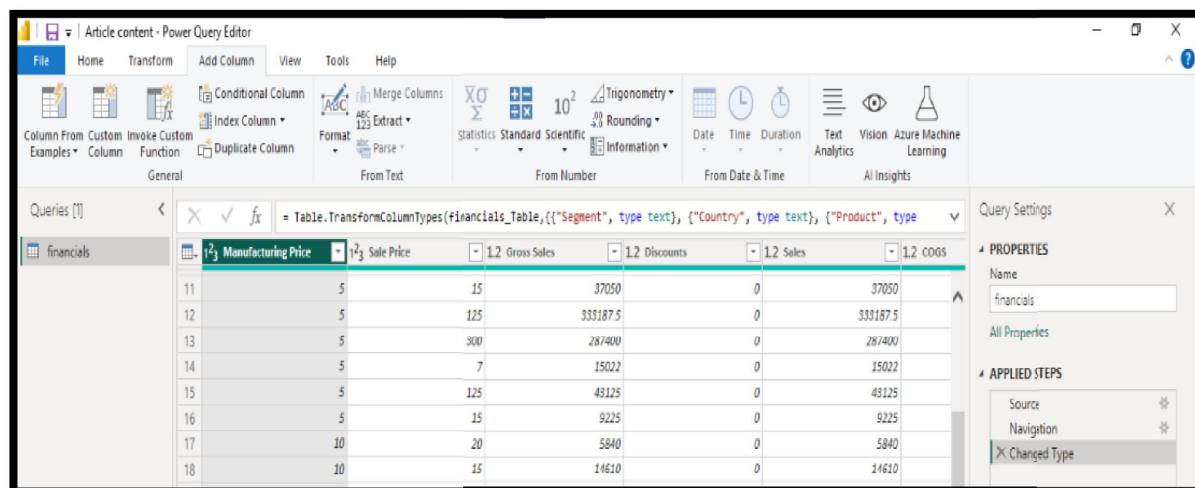


Fig.39: Power Query Editor Adding Column

The 'View' tab manages the way the page is displayed.

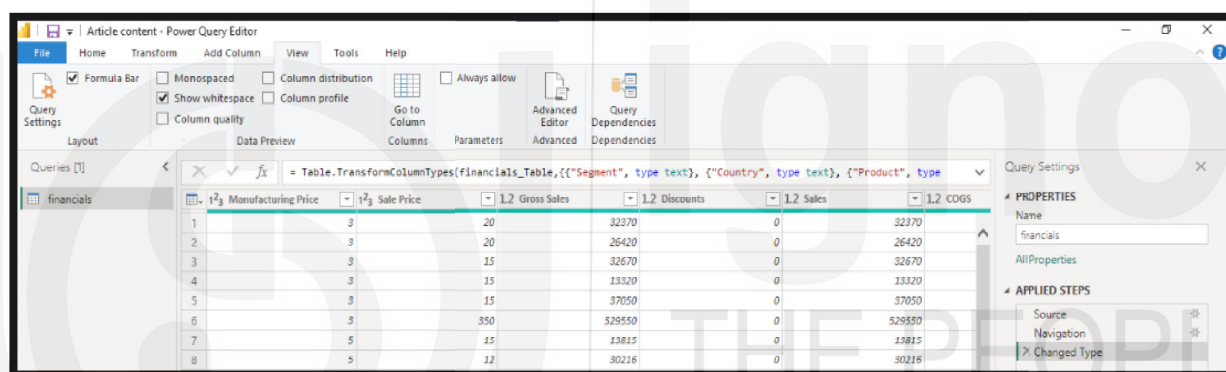


Fig.40: Power Query Editor View Tab

Step 4: By right-clicking on your column, you can apply various modifications.

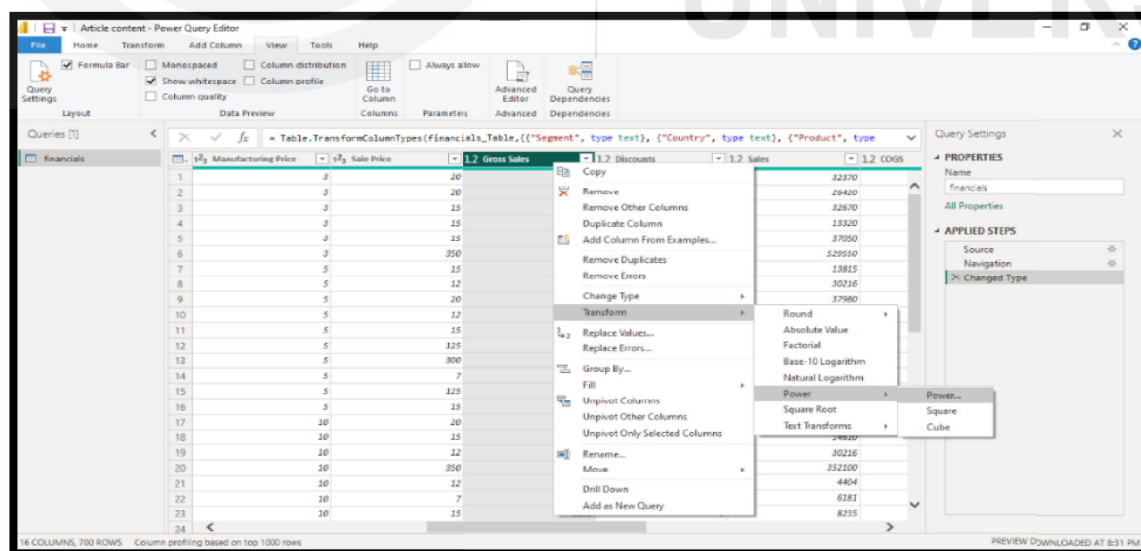


Fig.41: Power Query Editor View Tab

Step 5: After finishing the necessary data transformations, navigate to the 'File' tab and choose 'Close & Apply' to save your changes and return to Power BI Desktop.

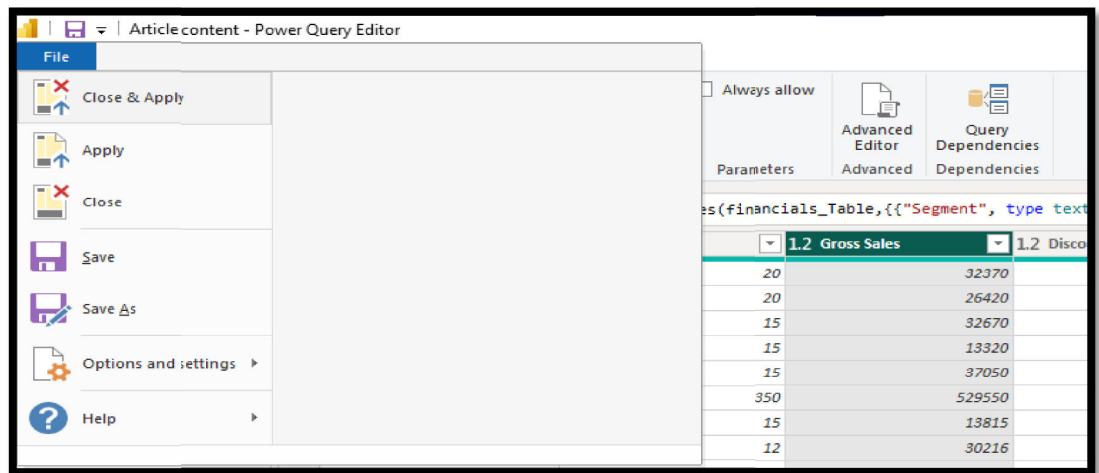


Fig.42: Power Query Editor View Tab

Your data is now ready to be used in creating visualizations and generating reports.

13.6.2 Visualization in Power BI

In Power BI, the Format and Fields panes play a vital role in customizing and enhancing the visual appearance of reports and dashboards. The Fields pane is used to manage and organize the dataset by selecting, adding, or removing fields for different visualizations, while the Format pane allows users to control the visual properties of charts and reports such as colors, titles, labels, backgrounds, data bars, and tooltips. To adjust the overall layout and appearance of a report, users can utilize the View tab, where options such as Themes, Page View, Gridlines, and Snap to Grid are available. The Themes option provides multiple predefined styling formats that help maintain visual consistency and improve report aesthetics according to organizational or presentation requirements. Together, the Format, Fields, and View options in Power BI enable users to design professional, interactive, and visually appealing dashboards and reports.

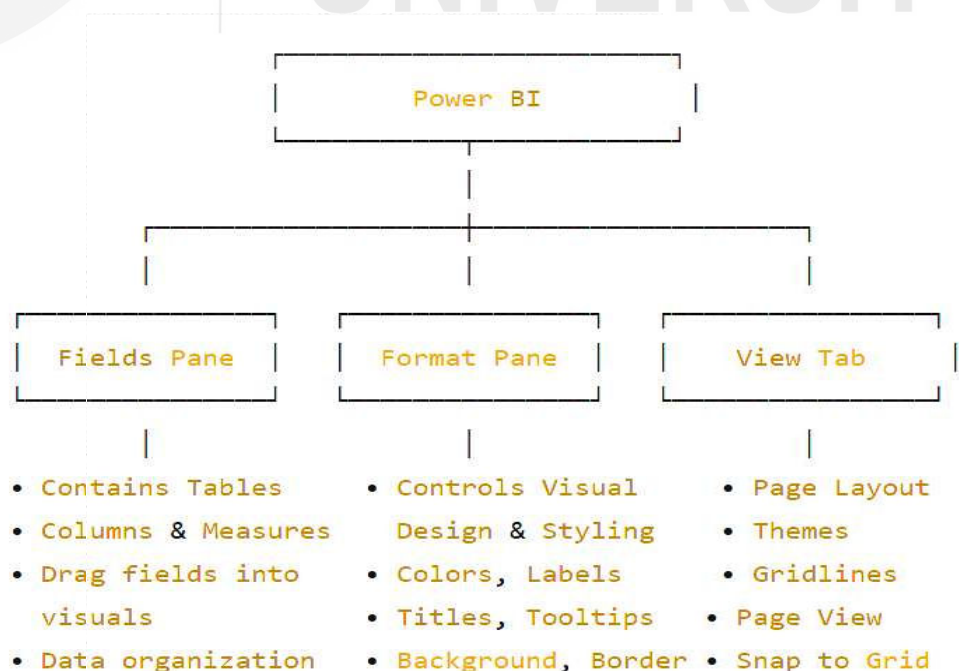


Fig.43: Power BI - Fields pane, Format Pane & View Tab

The **Format** and **Fields** pane is where you customize and improve the appearance of your dashboards and reports. To adjust the layout of your report, use the "**Layout**" option under the **View** tab, which provides various theme options to match your report's design.

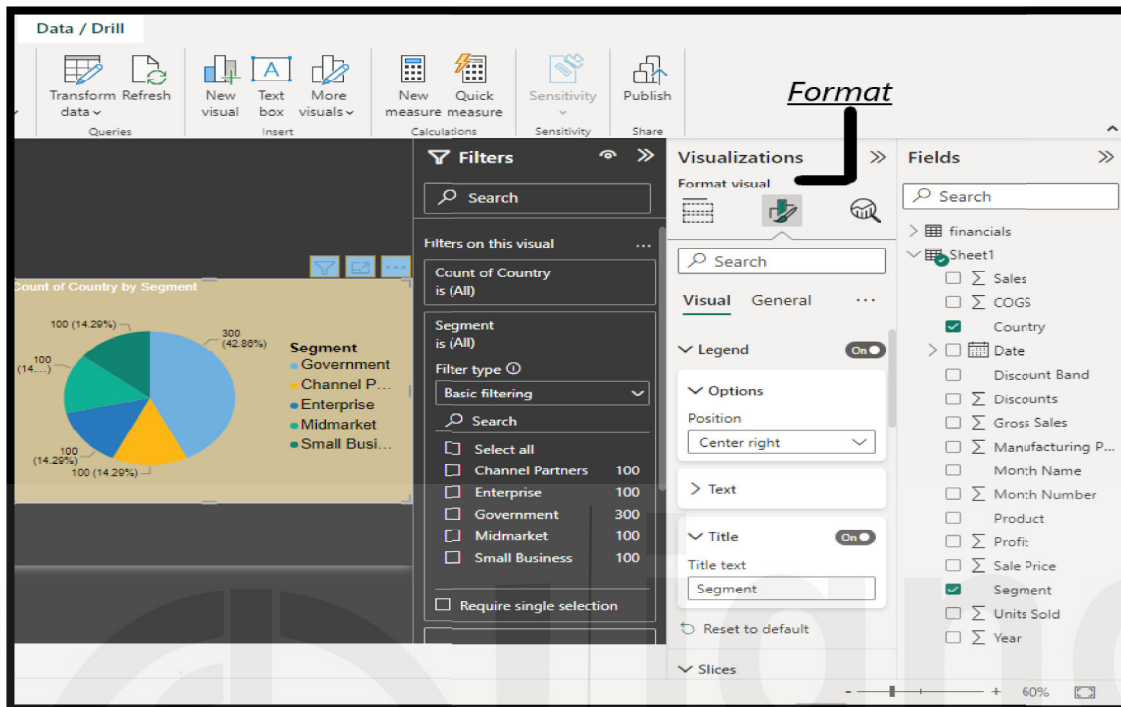


Fig.44: Visualization Tab

To modify a visualization already added to the report, select it and use the "Format" option to customize its appearance.

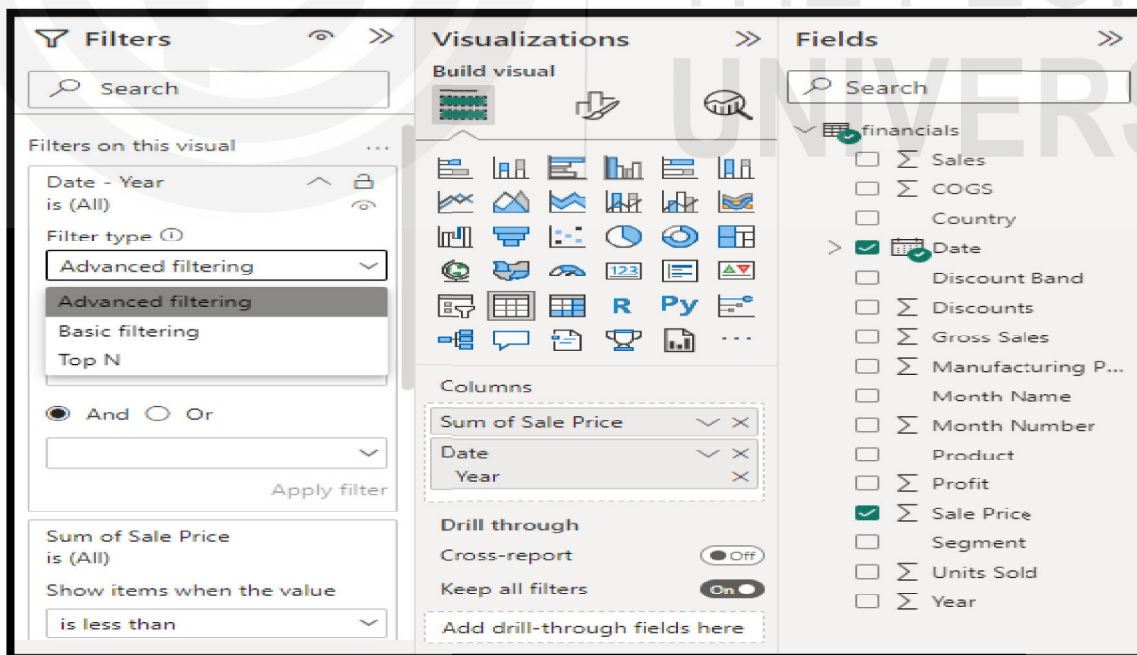


Fig.45: Visualization Tab

For more advanced customization of your visualizations, you can use **DAX formulas** to create calculated columns and measures as shown below in this section.

Data Analysis Expressions (DAX) is a powerful formula language used in **Power BI** (as well as in Excel Power Pivot and SQL Server Analysis Services) to perform **custom calculations on data** after it has been imported into the model. DAX helps users go beyond basic summaries by enabling the creation of **calculated columns, measures, and calculated tables**, which add new analytical meaning to existing data.

A **calculated column** works row by row and is mainly used to add new derived data to a table, such as profit calculated from sales and cost. A **measure**, on the other hand, is used to perform dynamic calculations such as totals, averages, percentages, or ratios that automatically change based on filters and slicers applied in the report. This dynamic behavior is what makes DAX especially valuable for interactive dashboards.

For example, a simple DAX measure like:

Total Sales = SUM(Sales[Amount])

calculates the total revenue by adding all sales values from the Sales table. This measure updates instantly when a user applies filters such as product category, date, or region. In this way, DAX allows reports to become **interactive and responsive**, which is essential for business analysis.

DAX also supports more **advanced analytical functions**, such as **time-intelligence calculations**. These functions make it possible to compare performance across time periods, such as month-to-month growth, year-over-year growth, or running totals. For example, a **Year-over-Year (YoY) Sales Growth** measure compares current year sales with previous year sales and expresses the change as a percentage. Such calculations are extremely useful for trend analysis and performance evaluation.

Basic Steps of Working with DAX in Power BI

Step 1: Load Data into Power BI

1. Open **Power BI Desktop**.
2. Click on **Home** → **Get Data**.
3. Select **Excel / CSV / Text file**.
4. Browse and load your dataset.
5. Click **Load** to import the data into Power BI.

After loading the dataset:

- Use **Data View** to see the table.
- Use **Report View** to create visuals and apply DAX measures.

6. Create a Basic DAX Measure

1. Go to **Report View**.
2. In the **Fields Pane**, right-click on the table name (e.g., *Sales*).
3. Click on **New Measure**.
4. In the formula bar, type the DAX formula:

Example, Total Sales = SUM (Sales [Price])

5. Press Enter

This measure calculates the **total sales amount dynamically**.

In short, **DAX acts as the analytical brain of Power BI**. It allows analysts to create meaningful metrics, perform complex calculations, compare performance over time, and build interactive dashboards that support informed decision-making. Even at an introductory level, understanding basic DAX concepts like measures, filters, and aggregation functions provides a strong foundation for advanced business intelligence and data analysis tasks.

This measure dynamically evaluates sales growth relative to the previous year based on the current filter context. Thus, the use of DAX formulas significantly enhances analytical flexibility and supports the development of meaningful business insights within Power BI.

13.6.3 Generation of Descriptive Statistics in Power BI

Descriptive statistics are used to summarize and describe the basic features of a dataset in a simple and meaningful way. In Power BI, descriptive statistics are generated using a combination of built-in visual tools and DAX (Data Analysis Expressions) measures, allowing users to quickly understand patterns, trends, and overall performance of data.

Power BI provides built-in analytical capabilities to summarize data and derive descriptive statistics such as totals, averages, minimum and maximum values, counts, percentages, and distributions. Once a dataset is imported into Power BI Desktop, visual elements like Cards, Tables, Matrix, and Clustered Bar/Column Charts can be used to present summary statistics. Additionally, DAX measures enhance the analytical scope by enabling dynamic calculations based on user interactions. Common descriptive statistical measures in Power BI can be generated with simple DAX formulas

To create more flexible and dynamic statistics, Power BI allows users to define custom DAX measures. Simple DAX formulas such as SUM, AVERAGE, DISTINCTCOUNT, MIN, and MAX are commonly used to compute descriptive measures. For instance, Total Quantity = SUM(OrderDetails[Quantity]) calculates the total number of items sold, Average Price = AVERAGE(Product[Price]) gives the typical price of products, and Product Count = DISTINCTCOUNT(Product[ProductID]) returns the number of unique products. Similarly, MIN(Product[Price]) and MAX(Product[Price]) identify the lowest and highest product prices in the dataset. These metrics update automatically based on report filters or slicers, helping users gain real-time insights into product performance, customer preferences, and revenue trends.

One of the major advantages of using Power BI for descriptive statistics is that these values are dynamic. This means the results automatically change whenever a user applies filters or slicers, such as selecting a particular product category, time period, or region. As a result, users can instantly observe how key metrics vary across different conditions.

We learned that, Power BI simplifies the generation of descriptive statistics by combining easy-to-use visuals with powerful DAX calculations. This enables beginners to summarize large datasets, uncover basic patterns, and gain quick insights into product performance, customer behavior, and overall business trends in an interactive and user-friendly manner.

A) Creation of Dashboard Steps in Power BI

The dashboard development process in Power BI involves connecting to data, transforming it as needed, and building interactive visuals. The essential steps are:

Step 1: Importing Data

To begin working in Power BI, the first task is to load the required dataset into the application. Power BI enables data import from multiple sources, including Excel, CSV, databases, and online services. Click on the “Get Data” option located on the left side of the Home screen to browse and select your data file. Once the desired dataset is chosen, it can be easily loaded into the workspace for further processing and analysis.

- Open Power BI Desktop → Click Get Data
- Select the data source (Excel/CSV/Database/etc.)
- Load the dataset into the Data Model

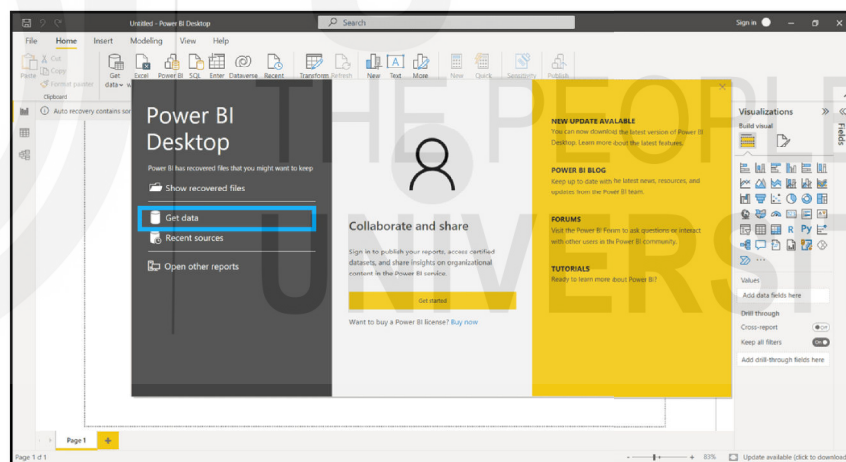


Fig.46: PowerBI Dashboard

EmployeeID	Married	MaritalStatus	GenderID	EmploymentDeptID	PerfPercent	FromOverSalary	Termed	PositionID	Position	State	Zip	DOB	Sex	MaritalStatus	Citizenship	HighestLe	HasChildren	Descriptive	Dataset	Tr	To			
Adkins, VA	10026	0	0	1	1	5	4	0	62506	0	19	Producter	MA	1960	00000000	M	Single	US Citizen	No	White	00000000	NJ		
Ahn, Ted, Ka	10081	1	1	1	5	3	1	0	104437	1	27	Sr. DBA	MA	2148	00000000	M	Married	US Citizen	No	White	3/10/2015	6/16/2006	ca	
Alkassabeh	10196	1	1	0	5	5	1	0	64955	1	20	Producter	MA	1810	00000000	F	Married	US Citizen	No	White	00000000	9/24/2002	h	
Almagne, Triv	10088	1	1	0	1	5	3	0	64991	0	19	Producter	MA	1886	00000000	F	Married	US Citizen	No	White	00000000	NJ		
Anderson, 10009	0	2	0	5	5	3	0	0	50825	1	19	Producter	MA	2168	00000000	F	Married	US Citizen	No	White	00000000	NJ		
Anderson, 10002	0	0	0	1	1	5	4	0	57568	0	19	Producter	MA	1841	00000000	F	Single	US Citizen	No	White	00000000	NJ		
Andersola, I	10194	0	0	0	1	4	3	0	95660	0	24	Software	MA	2110	00000000	F	Single	US Citizen	No	White	00000000	NJ		
Arbuckle, SA	10062	0	4	1	1	5	3	0	59385	0	19	Producter	MA	2109	00000000	M	Widowed	US Citizen	No	White	9/26/2012	NJ		
Barbich, 10114	0	0	0	0	3	5	3	1	47837	0	19	Producter	MA	1900	00000000	F	Single	US Citizen	No	Black or AI	00000000	NJ		
Baron, 10152	1	1	0	5	5	1	1	0	14670	1	19	Producter	MA	1982	00000000	F	Married	US Citizen	Yes	White	00000000	NJ		
Barber, T	10242	1	1	1	5	5	3	1	47211	1	19	Producter	MA	2062	00000000	M	Married	US Citizen	Yes	Black or AI	00000000	9/10/2006	un	
Barbosa, 10152	0	2	1	1	5	3	4	1	92228	0	9	Data Analy	TX	78230	00000000	M	Married	US Citizen	No	Black or AI	00000000	NJ		
Barone, Tr	10265	0	0	1	1	1	5	1	0	58705	0	19	Producter	MA	1810	00000000	M	Single	US Citizen	No	White	00000000	9/24/2002	h
Barton, NJ	10066	0	2	1	5	5	1	0	52505	1	19	Producter	MA	2147	00000000	M	Married	US Citizen	No	White	00000000	NJ		
Bates, Nor	10004	0	0	1	4	5	3	0	51834	1	19	Producter	MA	2050	00000000	M	Single	US Citizen	No	White	2/21/2011	00000000	at	
Bauer, Rnd	10023	1	1	0	2	5	4	0	70131	0	20	Producter	MA	2149	00000000	F	Married	US Citizen	No	White	7/12/2010	NJ		
Beardice, C	10055	0	0	0	1	5	3	0	59026	0	19	Producter	MA	1915	00000000	F	Single	Eligible No	No	White	00000000	NJ		
Bauer, Re	10245	0	0	0	4	5	3	0	110300	1	8	Database	MA	2026	00000000	M	Single	US Citizen	Yes	White	00000000	per		
Becker, No	10277	0	0	1	8	5	1	0	31290	0	19	Producter	MA	2052	00000000	M	Single	US Citizen	No	Asian	00000000	NJ		
Berndt, 10046	0	0	1	1	5	3	0	0	51044	0	19	Producter	MA	2072	00000000	M	Single	US Citizen	Yes	White	00000000	NJ		
Biden, Lon	10236	0	2	0	1	5	3	0	64910	0	19	Producter	MA	2007	00000000	F	Married	US Citizen	No	Asian	8/19/2011	NJ		
Bills, Rich	10003	1	1	0	1	5	4	0	62910	0	19	Producter	MA	2011	00000000	F	Single	US Citizen	No	White	00000000	NJ		
Blount, Div	10294	0	0	0	1	5	2	0	65441	0	20	Producter	MA	2171	00000000	F	Single	US Citizen	No	White	00000000	NJ		
Bonifant, 20267	0	0	0	0	2	5	3	0	37820	1	20	Producter	MA	2138	00000000	F	Single	US Citizen	No	White	00000000	ce		
Bouly, Fra	10199	0	0	1	4	3	3	0	108513	1	30	Enterprise	CT	6633	00000000	M	Single	US Citizen	No	Black or AI	2/17/2014	2/19/2006	Le	
Brownell, I	10081	1	1	0	1	1	1	3	1	106367	0	26	Sr. Account	MA	2468	00000000	F	Married	US Citizen	No	Black or AI	2/16/2015	NJ	
Brown, Cha	10175	0	0	1	5	5	3	0	74112	1	18	Producter	MA	1980	00000000	M	Single	US Citizen	No	Asian	9/30/2013	00000000	re	
Brill, Donn	10277	1	1	0	5	5	3	0	53492	1	19	Producter	MA	1701	00000000	F	Married	US Citizen	No	White	00000000	6/15/2003	Ar	
Brown, Mi	10238	1	1	0	1	1	1	1	83000	0	1	Accountant	MA	1450	00000000	F	Married	US Citizen	No	Black or AI	10/27/2008	NJ		
Buckner, I	10184	0	0	1	1	5	3	0	62038	0	20	Producter	MA	1913	00000000	M	Single	US Citizen	No	White	9/26/2014	NJ		
Bugall, Joe	10203	0	0	0	2	5	1	0	64175	0	19	Producter	MA	2093	00000000	F	Separated	US Citizen	No	Black or AI	00000000	NJ		

Fig.47: Sample Dataset

The Navigation pane provides an option labeled **Files**, which allows users to

access data stored locally. Select **Files** and browse to the folder where the Excel workbook or other supported data format is saved. After selecting the appropriate file, click **Connect** to establish the data import into Power BI.

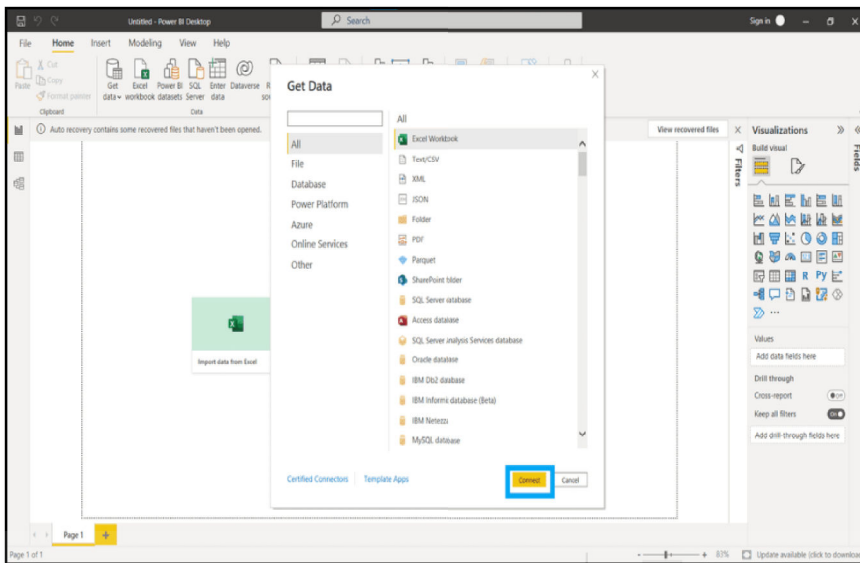


Fig.48: PowerBI Data Upload

Depending on the dataset size, Power BI may require a short processing time to extract the data. Once the preview is displayed, verify that the data has been imported correctly and then click **Load** to bring the dataset into the Power BI workspace.

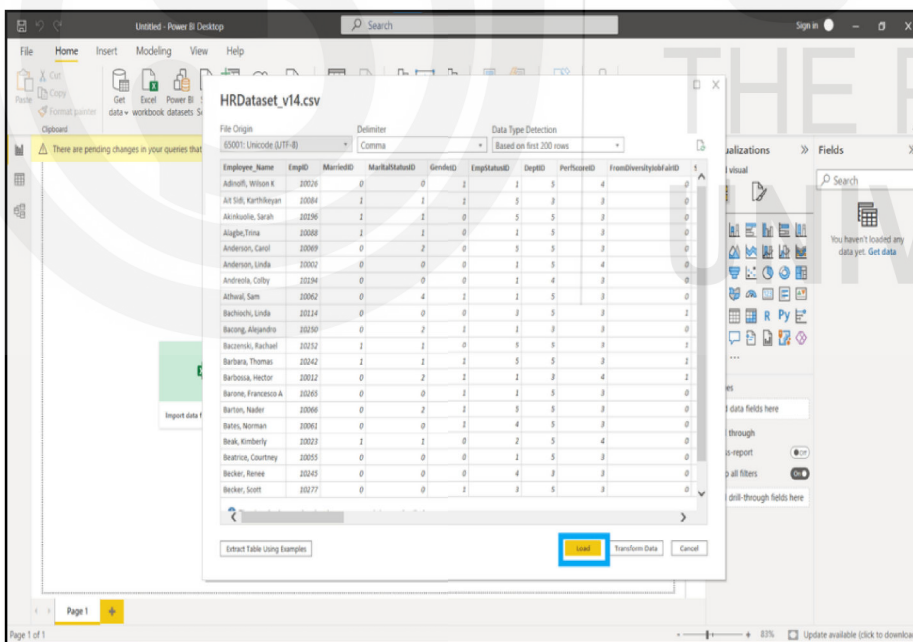


Fig.49: HR Dataset Upload

Step 2: Exploring the Data

After the data is loaded, navigate to the **Data View** to examine the dataset in a structured tabular format. The **Fields Pane** on the right side of the interface displays all available tables along with their respective columns, allowing you to review and understand the attributes present in the imported data.

Employee Name	EmployeeID	MarriedID	MaritalStatusID	GenderID	EmploymentStatusID	DegreeID	PerformanceID	FranchiseID	Salary	Tenure	Profitability	Position	State	Zip
Adams, Wilson K	10005	0	0	1	1	5	4	0	62500	0	15	Production Technician I	MA	2500 01
Alagba, Ima	10088	1	2	0	1	5	3	0	64992	0	19	Production Technician I	MA	2884 06
Anderson, Linda	10002	0	0	0	1	5	4	0	57500	0	19	Production Technician I	MA	1844 05
Atharal, Sam	10062	0	4	1	1	5	3	0	59365	0	19	Production Technician I	MA	2299 02
Bachsch, Linda	10111	0	0	0	1	5	3	0	49627	0	19	Production Technician I	MA	1962 02
Baroni, Francesco A	10205	0	0	1	1	5	3	0	58700	0	19	Production Technician I	MA	2820 07
Beak, Kimberly	10023	1	1	0	1	5	4	0	70212	0	20	Production Technician I	MA	2345 04
Beathric, Courtney	10055	0	0	0	1	5	3	0	59020	0	19	Production Technician I	MA	2521 04
Becker, Scott	10277	0	0	1	1	5	3	0	53250	0	19	Production Technician I	MA	2452 04
Bernsteln, Sean	10040	0	0	1	1	5	3	0	53044	0	19	Production Technician I	MA	2072 12
Biden, Luanan M	10226	0	2	0	1	5	3	0	64029	0	19	Production Technician I	MA	2027 12
Bills, Helen	10003	1	1	0	1	5	4	0	62500	0	19	Production Technician I	MA	2031 05
Buchner, Joseph	10184	0	0	1	1	5	3	0	65280	0	20	Production Technician I	MA	2623 07
Bugali, Josephine	10009	0	3	0	1	5	3	1	64975	0	19	Production Technician I	MA	2043 10
Burke, Iselle	10207	0	0	0	1	5	3	0	62762	0	20	Production Technician I	MA	2348 05
Burkett, Benjamin	10181	1	1	1	1	5	3	0	62162	0	20	Production Technician I	MA	1880 06
Candle, Calvin	10001	0	0	1	1	5	4	0	72040	0	18	Production Manager	MA	2389 08
Carey, Michael	10115	0	0	1	1	5	3	0	62844	0	19	Production Technician I	MA	1761 02
Chaco, Beatrice	10067	0	0	0	1	5	3	0	60856	0	19	Production Technician I	MA	2763 01
Chan, Lin	10110	0	0	0	1	5	3	0	54287	0	19	Production Technician I	MA	2330 02
Chang, Donovan E	10154	0	0	1	1	5	3	0	60180	0	19	Production Technician I	MA	1845 08
Ciepiolowski, Caroline	10168	0	0	0	1	5	3	0	64834	0	19	Production Technician I	MA	2044 05
Claley, Eljan	10029	1	1	1	1	5	4	0	50973	0	19	Production Technician I	MA	2334 04
Cockel, James	10061	0	0	1	1	5	3	0	63820	0	19	Production Technician I	MA	2452 05
Concone, Vito	10019	0	0	1	1	5	4	0	170500	0	10	Director of Operations	MA	2030 01
Corbett, Lisa	10094	1	1	0	1	5	3	0	63882	0	19	Production Technician I	MA	2389 05
Cronmings, Jean	10112	0	0	0	1	5	3	0	56349	0	19	Production Technician I	MA	1821 04

Fig.50: Data View

You can click on any table or individual field to apply formatting options as required. For attributes representing values such as dates, time, geographical locations (city, state), percentages, or currency, the appropriate **data type and formatting** can be assigned using the tools available under the **Modeling** tab.

Step 3: Selecting an Appropriate Visualization

For developing the dashboard, we focus on five key fields: **HiredYear**, **RecruitmentSource**, **Position**, **EmployerID**, and **Gender (Male/Female)**. The first visual to be added is a **Card** visualization. To create it, simply select the **Card** icon from the **Visualizations** pane and place the relevant field onto the canvas.

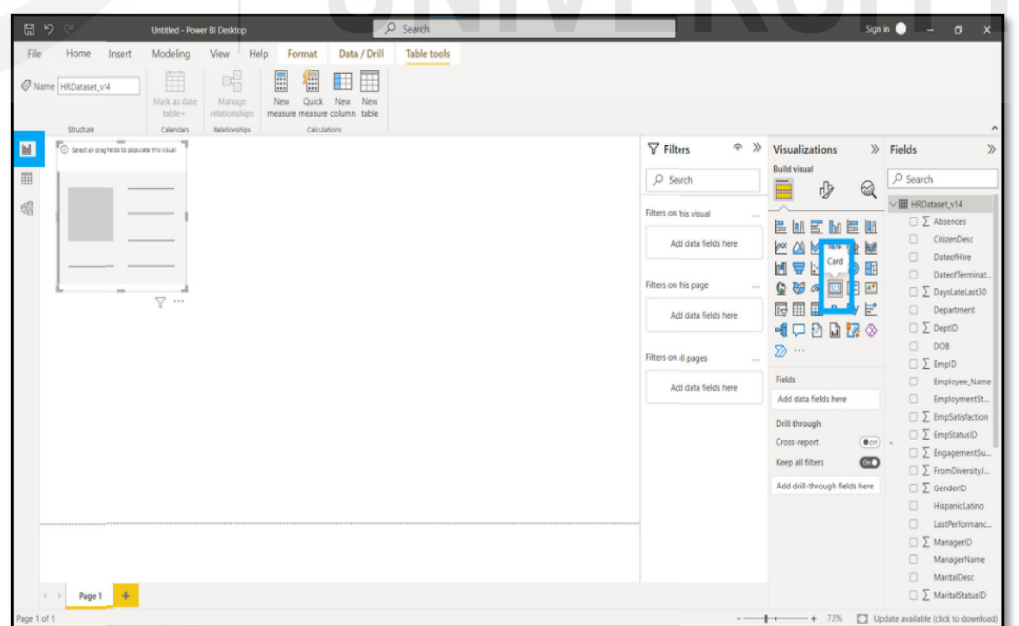


Fig.51: Table Tools

Next, choose the fields that you want to include in the visualization from the **Fields Pane**. You may either click to add them or simply **drag and drop** the selected fields into the corresponding placeholders within the visualization, as indicated in the interface.

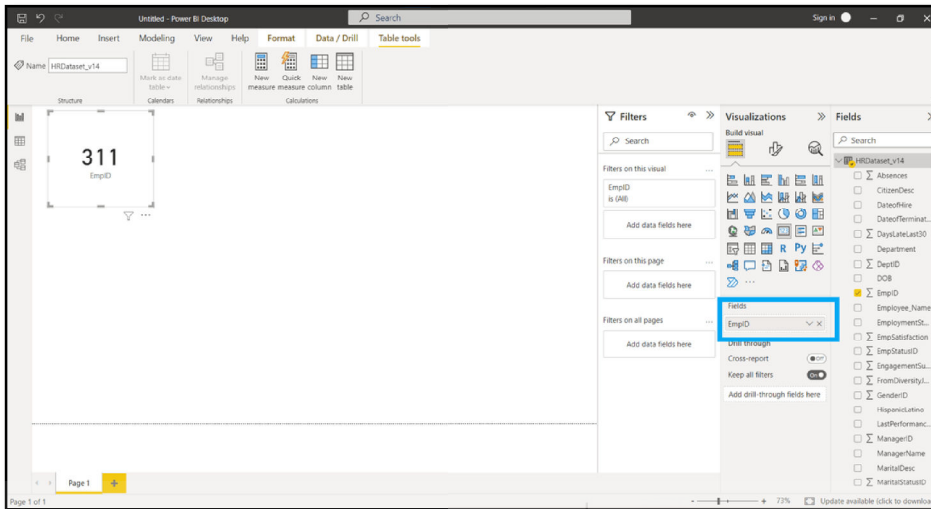


Fig.52: Field Pane

You can customize the visualization further by selecting fields, applying filters, and adjusting appearance settings using the **Format** options. The first Card visualization created displays the **total number of Employers**, providing a quick summary metric for the dashboard.

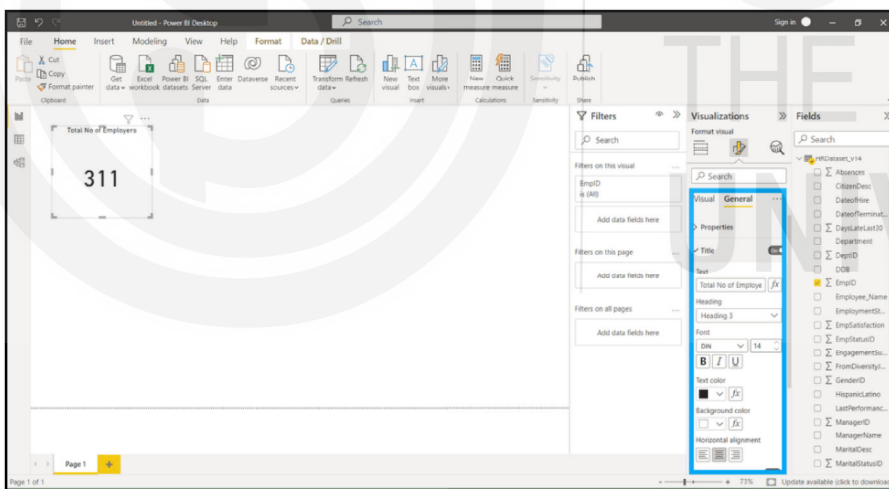


Fig.53: Visualization Pane

The same procedure was repeated to create the remaining Card visuals. These additional cards represent key summary indicators, including the **total number of Recruitment Sources**, **total number of Positions**, and the **Maximum Salary** value present in the dataset. Together, these visuals provide an at-a-glance statistical overview that supports further analysis within the dashboard.

Next, a **Pie Chart** and a **Donut Chart** are created to represent the **distribution of job positions** and the **gender-wise employment ratio**, respectively. These charts can be inserted by selecting the corresponding

icons from the **Visualizations** pane and assigning the required fields to the chart areas.

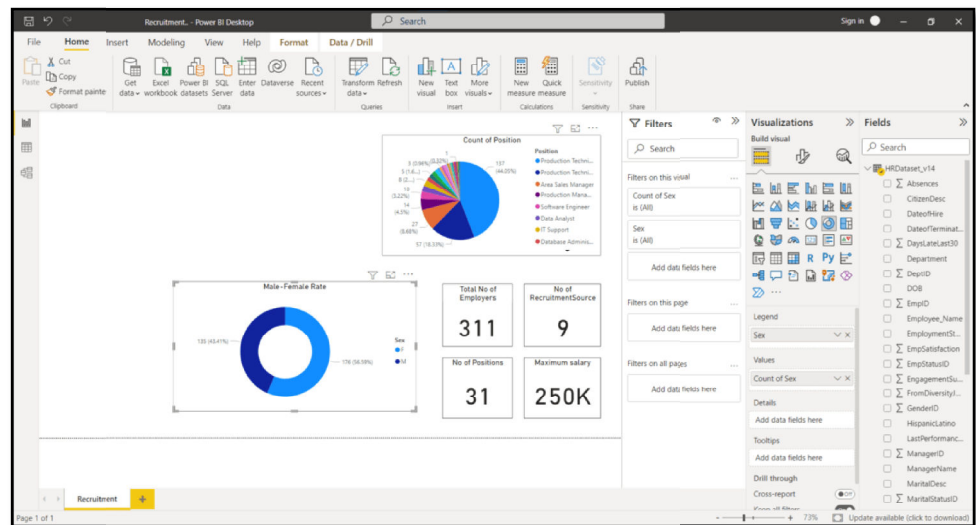


Fig.54: Types of Charts

Finally, a **Funnel Chart** and a **Stacked Bar Chart** are added to visualize the year-wise hiring counts and the proportional contribution of different recruitment sources, respectively. To enhance clarity and presentation quality, formatting options such as titles, data labels, axis properties, legends, plot area settings, and color schemes are applied. These adjustments ensure the dashboard communicates insights effectively, as illustrated in the figure below.

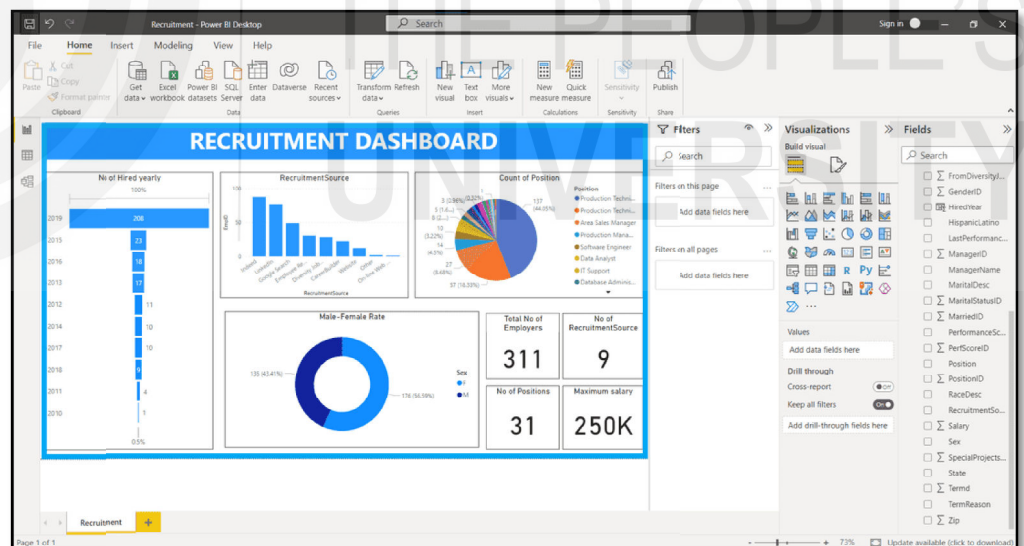


Fig.55: Visual demonstration of Recruitment Dashboard

Step 4: Publish and Share

- Publish report to Power BI Service
- Create dashboards by pinning visuals
- Share insights through secure cloud access

13.6.4 SUM UP: Power BI - User Interface and Data Handling

1. *Navigating Power BI*

- **Home Pane:** Get Data, Transform Data.
- **Fields Pane:** Lists all columns and tables.
- **Visualizations Pane:** Select and customize charts.
- **Report View:** Main canvas for building reports.

2. *Uploading the Dataset*

- Launch Power BI Desktop.
- Select "**Get Data**" and choose your source (Excel, CSV, or Database).
- Use the **Power Query Editor** to load or clean and shape your data as needed.

3. *Handling Data*

- Use Power Query to clean your data by removing rows and adjusting data types.
- Generate new columns and perform calculations with DAX (Data Analysis Expressions).
- Establish **Relationships** between tables.

4. *Plotting Graphs*

- Use drag-and-drop to build visuals on canvas.
- Available charts: bar, line, scatter, map, waterfall, KPI, donut.
- Interactivity via slicers and filters.

5. *Example of a DAX Formula*

Power BI allows users to create custom calculations using DAX (Data Analysis Expressions).

Below is an example formula that calculates the **Total Sales** based on Price × Quantity:

Total Sales = SUMX(*OrderDetails*, *OrderDetails*[Quantity] * *OrderDetails*[Price])

This calculated measure dynamically updates based on filters used in the report and helps in analyzing revenue trends across different dimensions such as products, categories, or properties.

🔧 Check Your Progress-2

1. What is Power Query used for?

.....

.....

.....

.....

.....

2. How can you create a new measure in Power BI?

.....

.....

.....

3. What are calculated columns in Power BI?

.....

.....

.....

4. How do you upload a dataset in Power BI?

.....

.....

.....

5. How do you publish a report in Power BI?

.....

.....

.....

13.7 PYTHON

This section is an Introduction to Python Interface (Anaconda & Google Colab) - **Python IDLE, Jupyter Notebook, and Google Colab** all support Python development but differ in interface and capabilities, a brief introduction to each is given below:

- **Python IDLE:** A simple, built-in IDE with a Python Shell for interactive code and an Editor for scripts. It's lightweight and good for beginners and small projects.
- **Jupyter Notebook:** A web-based tool with a cell-based layout supporting live code, text (Markdown), and visualizations. Ideal for data science, research, and documentation.
- **Google Colab:** A cloud-hosted version of Jupyter with added benefits like free GPU/TPU access, Google Drive integration, and real-time collaboration. Best suited for machine learning tasks.

Before we start diving into Python coding, let's learn how to install Python IDLE.

Installing Python and Accessing IDLE : To work with Python locally (instead of Google Colab), we use **Python IDLE**, which comes bundled with the official Python installation. The steps to be followed for download and installation of Python IDLE are as follows:


```
python  
  
print("Hello, world!")
```

5. Save Your File
 - Navigate to **File > Save** or press **Ctrl+S**.
 - Save the file using a **.py** extension, such as **hello.py**.
6. Run Your Program
 - In the editor window, go to **Run > Run Module** or press **F5**.
 - The output appears in the Python Shell window.
7. Use Basic Features
 - IDLE also includes:
 - ✓ **Syntax highlighting** in the editor
 - ✓ **Auto-indentation**
 - ✓ **Basic debugger** (under **Debug > Debugger**)

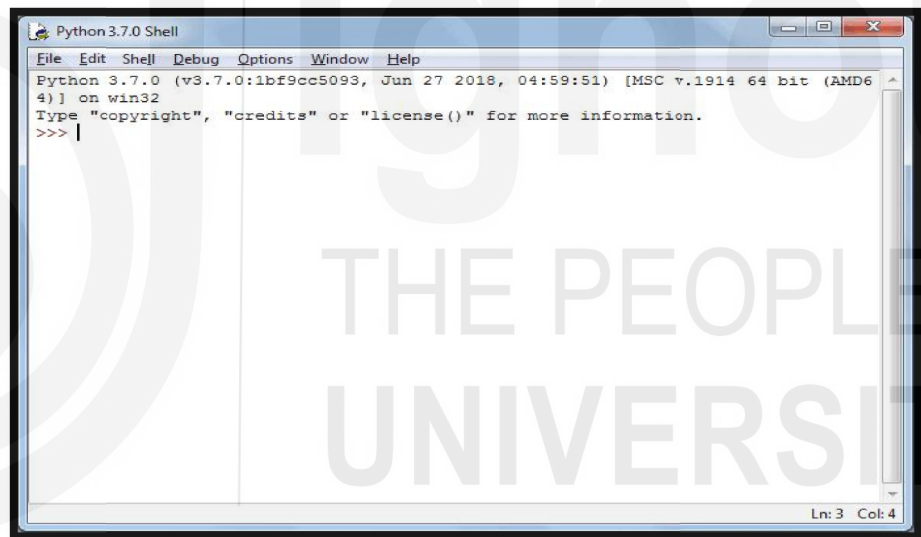


Fig .63: Python IDLE Interface

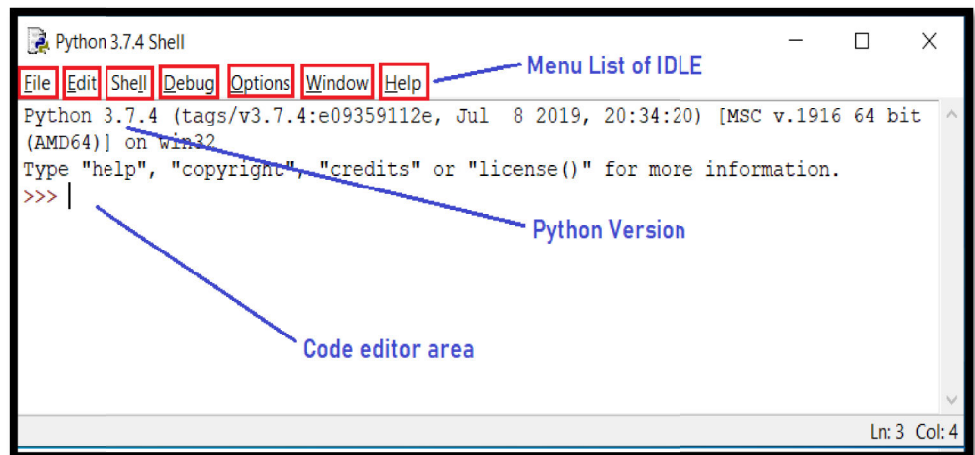


Fig .64: Python IDLE shell Interface

13.7.1 Steps to Use Jupyter Notebook

1. Install Jupyter Notebook (if it isn't already installed)

- You can install it using pip:

```
pip install notebook
```

Or install via Anaconda (which includes Jupyter by default).

2. Start Jupyter Notebook

- Open a terminal or command prompt and enter the following command:

```
jupyter notebook
```

- A web browser will open showing the Jupyter dashboard (usually at <http://localhost:8888>).

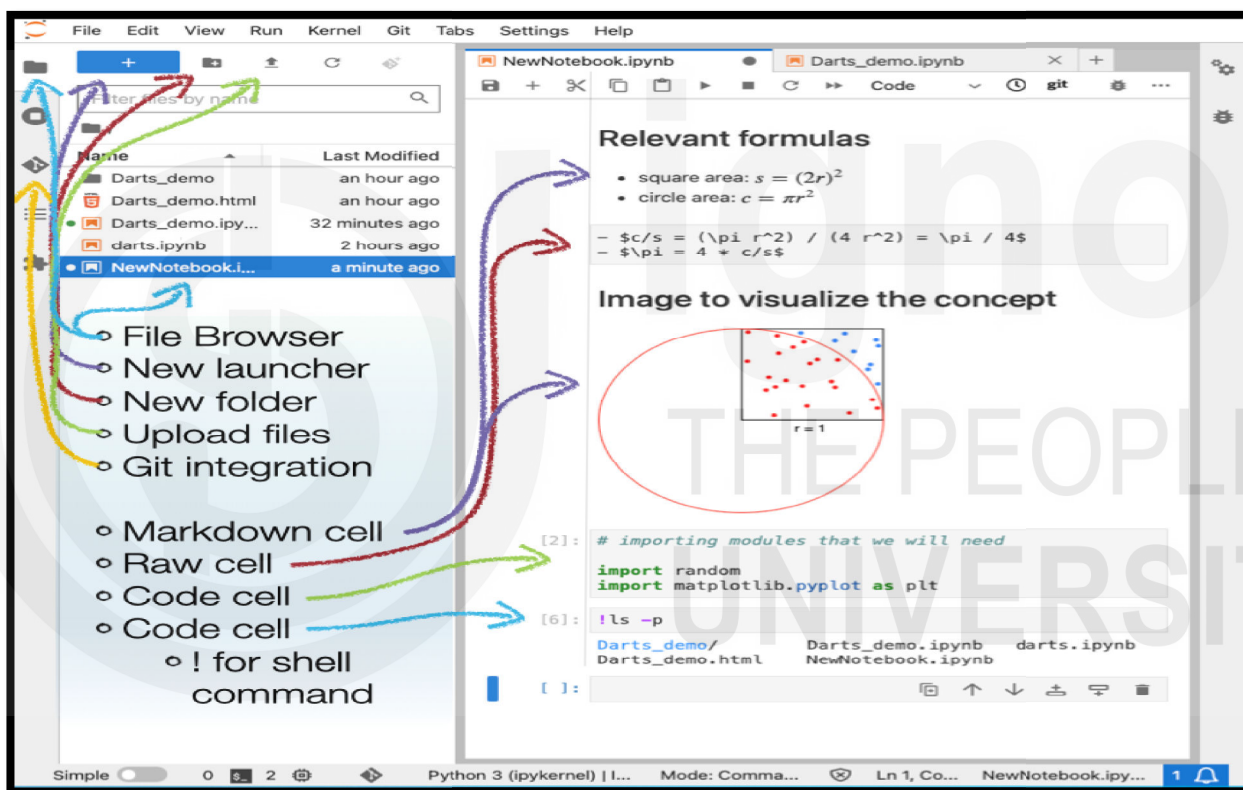


Fig .65: Python JupyterLab Interface

The JupyterLab interface features a toolbar on the left that includes a file browser and a Git tab. The file browser displays the directory from which JupyterLab was launched. On the right side, you can view and work with files you've opened. To enable a split view, simply drag a file tab from the top into another area of the workspace.

7. Create a New Notebook

- Click on “New” and select “Python 3” to launch a fresh notebook.
- A new tab opens with a blank notebook .

8. Write and Run Code

- Write your code in a **cell**.

Example:

```
python  
  
print("Hello, Jupyter!")
```

- Execute the cell by clicking the **Run** button or pressing **Shift + Enter** [5].
9. Use Markdown for Notes
 - Change a cell to **Markdown** (from the dropdown menu).
 - Write formatted text using Markdown syntax.
 10. Save Your Notebook
 - Go to **File > Save and Checkpoint**, or use the shortcut **Ctrl+S**.
 - It saves as a .ipynb file.
 11. Close the Notebook
 - Close the browser tab and shut down the server by pressing **Ctrl+C** in the terminal window.

13.7.2 Steps to Use Google Colab

1. **Open Google Colab**
 - Go to: <https://colab.research.google.com>
2. **Sign In with Google Account**
 - You need a Google account to use Colab.
3. **Create or Open a Notebook**
 - Click **"File > New Notebook"** to start a new one.
 - Or open an existing one from:
 - Google Drive
 - GitHub
 - Upload from your device

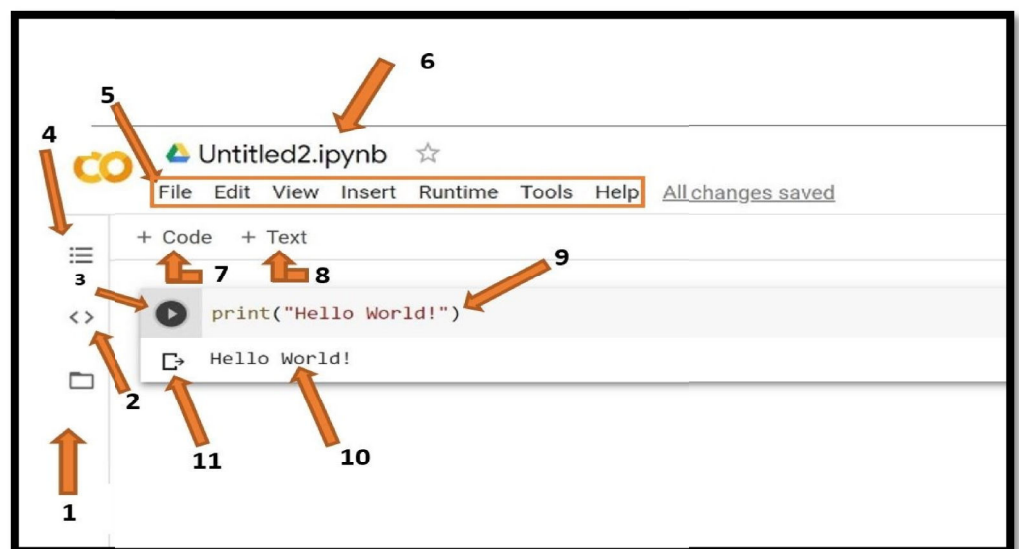


Fig.66: Python Google Colab Notebook Interface

4. Write and Run Code

- Enter Python code into a cell and press **Shift + Enter** to execute it.

5. Add Text Cells

- Click + **Text** to add a Markdown cell for formatted notes or headings.

6. Save the Notebook

- It autosaves to **Google Drive**.
- To download, click **File > Download > .ipynb** or **.py**.

7. Use Extra Features

- You can:
 - Upload files using the sidebar or `files.upload()`
 - Mount Google Drive using:

```
python

from google.colab import drive
drive.mount('/content/drive')
```

- Use GPUs (under **Runtime > Change runtime type > GPU**)

Till now we learned how to work with Google Colab, Jupyter Notebook, IDLE etc., now let's understand how to work with Python using datasets. To understand the working we used MovieLens Small Dataset, you can browse it on internet and download it from the link : <https://grouplens.org/datasets/movielens/> ; We'll use the file: `movies.csv`

13.7.3 Steps to Upload Dataset in Python IDLE

IDLE is a basic IDE and doesn't have a built-in way to upload files — but you can **manually place the file in your script folder**.

Steps to be followed are:

1. **Download** `movies.csv` from MovieLens and save it in the same folder as your `.py` file.
2. Open IDLE > File > New File.
3. Write and run the code:

```
import pandas as pd

# Load the dataset
data = pd.read_csv("movies.csv")

# Display first 5 rows
print(data.head())
```

4. Save and run the script (F5).

Steps to Upload Dataset in Jupyter Notebook are:

1. Open Jupyter Notebook.
2. Place movies.csv in the **same folder** where your notebook is saved.
3. Use this code to load the file:

```
import pandas as pd

# Load CSV
df = pd.read_csv("movies.csv")

# Show first 5 rows
df.head()
```

4. If you want to **browse and upload** via Jupyter:
 - o Click **Upload** in the Jupyter dashboard > Choose movies.csv > Click **Upload** button again.

13.7.4 Steps to Upload Dataset in Google Colab

Method 1: Upload from Local System

- Open a new Colab notebook.
- Use this code:

```
from google.colab import files
uploaded = files.upload()
```

- Select and upload movies.csv.
- Then use:

```
import pandas as pd
import io

df = pd.read_csv(io.BytesIO(uploaded['movies.csv']))
df.head()
```

Method 2: Load from Google Drive

1. Mount Google Drive:

```
from google.colab import drive
drive.mount('/content/drive')
```

2. Place movies.csv in your Drive, then access it like:

```
df = pd.read_csv('/content/drive/MyDrive/movies.csv')
df.head()
```

Till this point we learned “how to interact with the python tools like Google Colab, Jupyter Notebook etc?”, and also we learned “how to upload the dataset?”. Now, we will learn to generate the descriptive statistics in the subsequent section.

13.7.5 Descriptive Statistics in Python

Descriptive statistics summarize and describe the main characteristics of a dataset in a meaningful way. They help us understand the central tendency, spread, and overall distribution of the data. Common descriptive measures include the mean, which represents the average value, and the median, which identifies the middle value when the data is sorted. The mode indicates the most frequently occurring value. Measures such as variance and standard deviation describe how much the data varies from the mean, while minimum, maximum, and range provide information about the spread of the dataset. These statistics allow readers to quickly understand the nature of the data before performing advanced analysis.

So, we learned that Descriptive Statistics, helps us to understand the dataset before modeling, and it helps to detect patterns, outliers, or missing data, further it makes us to summarize large data into key insights quickly.

Let's Learn the process of Generating Descriptive Statistics in Python :

To produce descriptive statistics in Python, the panda's library is used extensively. The following steps show how to compute them:

Step 1: Import pandas

```
import pandas as pd
```

Step 2: Create or Load a Dataset

```
data = {  
'Marks': [65, 70, 85, 90, 75, 88, 92]  
}  
df = pd.DataFrame(data)
```

Step 3: Generate Descriptive Statistics

```
stats = df.describe()  
print(stats)
```

Output will include:

- Count
- Mean
- Standard deviation
- Minimum value
- Maximum value
- 25th, 50th (Median), and 75th percentiles

Now, let's apply the understanding and try to explore, How to Perform Descriptive Statistics in Python?, Let's use the MovieLens ratings.csv (has user ratings of movies) for demonstration.

Step 1: Upload Dataset

(Use the same file upload method as discussed earlier in IDLE, Jupyter, or Colab.)

```
import pandas as pd

# Load ratings.csv (assumed to be uploaded in your current directory)
df = pd.read_csv("ratings.csv")

# Show first 5 rows
print(df.head())
```

Step 2: Apply Descriptive Statistics

```
# Summary of numerical columns
print(df.describe())
```

- **Output Includes:**
 - **count** – total number of entries
 - **mean** – average rating
 - **std** – standard deviation (spread)
 - **min, max** – lowest and highest values
 - **25%, 50%, 75%** – percentiles (quartiles)
- **Additional Analysis**

```
# Count of unique users and movies
print("Unique users:", df['userId'].nunique())
print("Unique movies:", df['movieId'].nunique())

# Most common rating
print("Most frequent rating:", df['rating'].mode()[0])

# Rating distribution
print(df['rating'].value_counts())
```

- **Explanation with Example:**

Assume these are some ratings:

```
4.0, 5.0, 3.0, 4.5, 3.5
```

- Mean = average rating = $(4 + 5 + 3 + 4.5 + 3.5) / 5 = 4.0$

- Median = 4.0 (middle value)
 - Mode = most frequent (if any)
 - Standard **deviation** = how much ratings deviate from mean
- Sample table example:

Term	Meaning	Example
Mean	Average	4.0
Median	Middle value	4.0
Mode	Most common value	4.0
Std. Dev	Spread of data	0.7
Count	Number of ratings	5

After generating descriptive statistics, plotting of graphs is an important step, and we will learn this process of plotting graphs in python in subsequent section.

13.7.6 Plotting Graphs in Python (Using Matplotlib)

To visualize data in Python, the most widely used library is Matplotlib, however many other libraries are available.

Follow the steps below to plot a simple graph:

Step 1: Install Matplotlib (if not already installed)

```
pip install matplotlib
```

Step 2: Import Required Libraries

```
import matplotlib.pyplot as plt
```

Step 3: Prepare Data

```
x = [1, 2, 3, 4, 5]
```

```
y = [10, 20, 15, 30, 25]
```

Step 4: Plot the Graph

```
plt.plot(x, y)
```

Step 5: Add Labels and Title

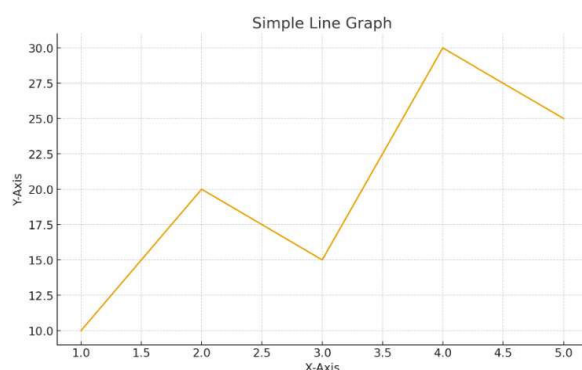
```
plt.xlabel("X-Axis")
```

```
plt.ylabel("Y-Axis")
```

```
plt.title("Simple Line Graph")
```

Step 6: Display the Plot

```
plt.show()
```



Let's understand the complete process by executing a Hands-on Example using mtcars dataset: we learned that Descriptive statistics help

highlight key characteristics of a dataset, often using just one value. Creating these statistics is typically the first step after data has been cleaned and prepared for analysis. We've previously encountered examples like the mean and median. In this section, we'll revisit those and introduce some additional statistical tools.

Firstly we begin with the finding of Central tendency of data i.e. Measures of Center, which are statistical methods used to identify the "middle" or most typical value in a numerical dataset. They provide insight into what a standard or expected value might be. The most common measures of central tendency include the mean, median, and mode. We can compute the mean for each column in a DataFrame using:

```
In [1]: %matplotlib inline

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

In [2]: mtcars = pd.read_csv("../input/mtcars/mtcars.csv")
mtcars = mtcars.rename(columns={'Unnamed: 0': 'model'})
mtcars.index = mtcars.model
del mtcars["model"]

mtcars.head()
```

```
Out[2]:
```

	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
model											
Mazda RX4	21.0	6	160.0	110	3.90	2.620	16.46	0	1	4	4
Mazda RX4 Wag	21.0	6	160.0	110	3.90	2.875	17.02	0	1	4	4
Datsun 710	22.8	4	108.0	93	3.85	2.320	18.61	1	1	4	1
Hornet 4 Drive	21.4	6	258.0	110	3.08	3.215	19.44	1	0	3	1
Hornet Sportabout	18.7	8	360.0	175	3.15	3.440	17.02	0	0	3	2

```
In [3]: mtcars.mean()           # Get the mean of each column

Out[3]:
mpg      20.090625
cyl       6.187500
disp     230.721875
hp       146.687500
drat      3.596563
wt        3.217250
qsec     17.848750
vs        0.437500
am        0.406250
gear      3.687500
carb      2.812500
dtype: float64
```

We can also get the means of each row by supplying an axis argument:

```
In [4]: mtcars.mean(axis=1)           # Get the mean of each row
```

```
Out[4]:
```

model	
Mazda RX4	29.907273
Mazda RX4 Wag	29.981364
Datsun 710	23.598182
Hornet 4 Drive	38.739545
Hornet Sportabout	53.664545
Valiant	35.049091
Duster 360	59.720000
Merc 240D	24.634545
Merc 230	27.233636
Merc 280	31.860000
Merc 280C	31.787273
Merc 450SE	46.430909
Merc 450SL	46.500000
Merc 450SLC	46.350000
Cadillac Fleetwood	66.232727
Lincoln Continental	66.058545
Chrysler Imperial	65.972273
Fiat 128	19.440909
Honda Civic	17.742273

The median of a dataset is the value that splits the data into two equal parts—half the values are lower, and half are higher. It represents the center point when the data is sorted. For this reason, it's also known as the 50th percentile. As shown previously, you can compute the median for each column in a DataFrame using:

```
In [5]: mtcars.median()           # Get the median of each column
```

```
Out[5]:
```

mpg	19.200
cyl	6.000
disp	196.300
hp	123.000
drat	3.695
wt	3.325
qsec	17.710
vs	0.000
am	0.000
gear	4.000
carb	2.000

dtype: float64

We can also calculate the median across each row by passing the argument `axis=1`. Although both the mean and median indicate the central tendency of a dataset, they measure it differently and are not always the same. The median reliably splits the data into two equal halves, while the mean represents the arithmetic average, which can be significantly affected by outliers or extreme values. In a

symmetrical distribution, the mean and median are generally equal. This concept can be better understood by examining a density plot.

```
In [6]: norm_data = pd.DataFrame(np.random.normal(size=100000))

norm_data.plot(kind="density",
               figsize=(10,10));

plt.vlines(norm_data.mean(),      # Plot black line at mean
           ymin=0,
           ymax=0.4,
           linewidth=5.0);

plt.vlines(norm_data.median(),   # Plot red line at median
           ymin=0,
           ymax=0.4,
           linewidth=2.0,
           color="red");
```

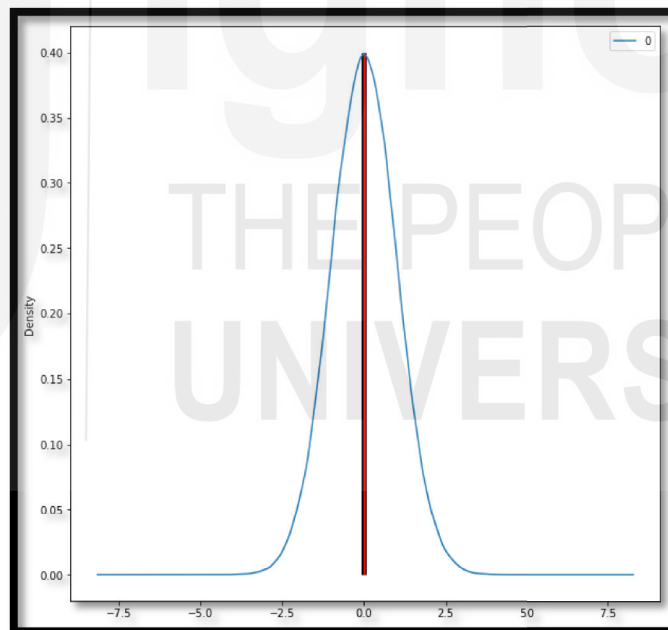


Fig.67: Output to check Normalization in dataset

In the plot shown above, the **mean** and **median** are nearly identical—both are close to zero—so the red line representing the median overlaps the thicker black line that marks the mean.

In skewed distributions, the mean is more affected by the skew and is drawn in the direction of the skewness, while the median is less influenced and tends to stay near the central point of the data.

```
In [7]: skewed_data = pd.DataFrame(np.random.exponential(size=100000))

skewed_data.plot(kind="density",
                  figsize=(10,10),
                  xlim=(-1,5));

plt.vlines(skewed_data.mean(), # Plot black line at mean
           ymin=0,
           ymax=0.8,
           linewidth=5.0);

plt.vlines(skewed_data.median(), # Plot red line at median
           ymin=0,
           ymax=0.8,
           linewidth=2.0,
           color="red");
```

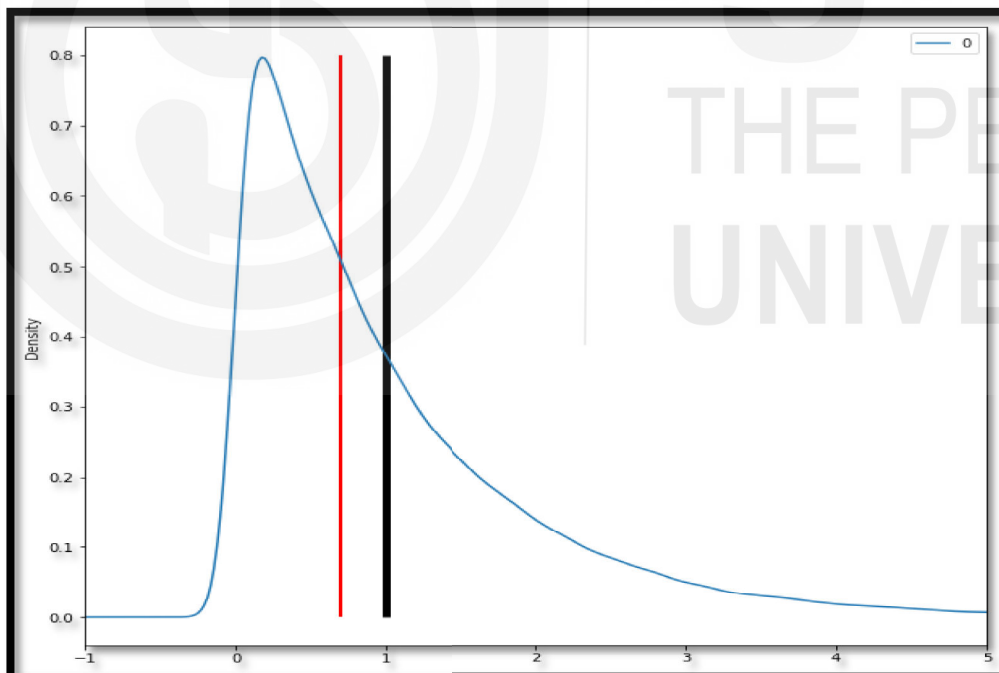


Fig: 68: Mean & Median Output

The mean is greatly influenced by outliers because it includes all values, even the extreme ones, in its calculation. On the other hand, the median is more resistant to outliers and stays fairly consistent, making it a more reliable measure of central tendency when outliers are present.

```
In [8]: norm_data = np.random.normal(size=50)
outliers = np.random.normal(15, size=3)
combined_data = pd.DataFrame(np.concatenate((norm_data, outliers), axis=0))

combined_data.plot(kind="density",
                    figsize=(10,10),
                    xlim=(-5,20));

plt.vlines(combined_data.mean(),      # Plot black line at mean
            ymin=0,
            ymax=0.2,
            linewidth=5.0);

plt.vlines(combined_data.median(),    # Plot red line at median
            ymin=0,
            ymax=0.2,
            linewidth=2.0,
            color="red");
```

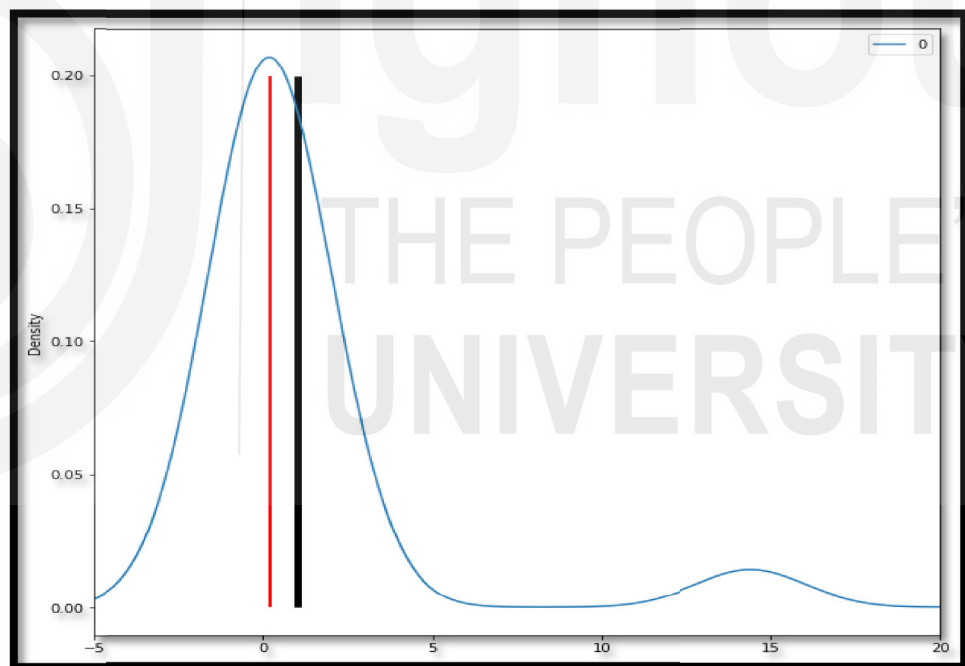


Fig.69: Outliers in dataset

Since the median is less influenced by skewed data and outliers, it is regarded as a "robust" measure. In cases where distributions are noticeably skewed or contain extreme values, the median often gives a more reliable representation of a typical value.

The mode is the value that appears most often in a dataset. Unlike the mean and median, the mode can also be used with categorical data. Additionally, a dataset may have more than one mode if several values share the highest frequency. To identify the mode, you can use:

```
In [9]: mtcars.mode()
```

```
Out[9]:
```

	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
0	10.4	8.0	275.8	110.0	3.07	3.44	17.02	0.0	0.0	3.0	2.0
1	15.2	NaN	NaN	175.0	3.92	NaN	18.90	NaN	NaN	NaN	4.0
2	19.2	NaN	NaN	180.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN
3	21.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
4	21.4	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
5	22.8	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
6	30.4	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN

If a column has **multiple modes**—meaning several values occur with the same highest frequency—then all those values are returned as modes. On the other hand, if a column has **no repeating values**, and thus **no mode**, the result will be **NaN**.

Finding the Measures of Spread is also important in data analysis, lets understand how to determine Measures of spread, also known as dispersion, indicate how much the values in a dataset differ from each other. While measures of center highlight a typical or average value, measures of spread show how widely the data points are scattered around that center.

A simple method for assessing spread is the **range**, calculated by subtracting the minimum value from the maximum value in the dataset.

```
In [10]: max(mtcars["mpg"]) - min(mtcars["mpg"])
```

```
Out[10]: 23.5
```

As noted earlier, the median represents the 50th percentile of a dataset. To gain a clearer picture of a variable's spread, it's helpful to examine several percentiles. These include the minimum (0th percentile), first quartile (25th percentile), median (50th percentile), third quartile (75th percentile), and maximum (100th percentile). You can extract these values using the `quantile()` function in pandas.

```
In [11]: five_num = [mtcars["mpg"].quantile(0),
                    mtcars["mpg"].quantile(0.25),
                    mtcars["mpg"].quantile(0.50),
                    mtcars["mpg"].quantile(0.75),
                    mtcars["mpg"].quantile(1)]

                    five_num

Out[11]: [10.4, 15.425, 19.2, 22.8, 33.9]
```

Because these percentile values are frequently used to summarize a dataset, they are collectively known as the **five-number summary**. The five-number summary consists of the minimum, 25th percentile (Q1), median (Q2), 75th percentile (Q3), and maximum. These key statistics offer a quick overview of the data's distribution and are also generated by the `df.describe()` function in pandas.

```
In [12]: mtcars["mpg"].describe()

Out[12]: count    32.000000
         mean     20.090625
         std       6.026948
         min      10.400000
         25%      15.425000
         50%      19.200000
         75%      22.800000
         max      33.900000
         Name: mpg, dtype: float64
```

The **Interquartile Range (IQR)** is a commonly used measure of variability. It shows the spread of the middle 50% of the data and is determined by subtracting the first quartile (Q1) from the third quartile (Q3):

$$\text{IQR} = \text{Q3} - \text{Q1}.$$

```
In [13]: mtcars["mpg"].quantile(0.75) - mtcars["mpg"].quantile(0.25)

Out[13]: 7.375
```

The boxplots shown in fig 57 representing the summary and the interquartile range (IQR) offers a clear view of the data's distribution, central tendency, spread, and possible outliers.

```
In [14]: mtcars.boxplot(column="mpg",
                      return_type='axes',
                      figsize=(8,8))

plt.text(x=0.74, y=22.25, s="3rd Quartile")
plt.text(x=0.8, y=18.75, s="Median")
plt.text(x=0.75, y=15.5, s="1st Quartile")
plt.text(x=0.9, y=10, s="Min")
plt.text(x=0.9, y=33.5, s="Max")
plt.text(x=0.7, y=19.5, s="IQR", rotation=90, size=25);
```



Fig.70: Boxplot of IQR

Two other commonly used measures of spread are **variance** and **standard deviation**.

- Variance is the average of the squared differences between each data point and the mean. It provides insight into how much the values in a dataset deviate from the average.

You can calculate the variance of each column in a DataFrame using:

```
In [15]: mtcars["mpg"].var()

Out[15]: 36.32410282258065
```

- **Standard deviation** is the square root of the variance. It's generally more intuitive to interpret since it's in the same units as the original data, unlike variance, which is expressed in squared units. To calculate the standard deviation for each column in a DataFrame, use:

```
In [16]: mtcars["mpg"].std()

Out[16]: 6.026948052089105
```

Since variance and standard deviation rely on the mean, they can be heavily influenced by skewed data and outliers. A more robust alternative is the Median Absolute Deviation (MAD). MAD is computed as the median of the absolute differences from the median, making it a dependable measure of spread for datasets with non-normal distributions or extreme values.

```
In [17]: abs_median_devs = abs(mtcars["mpg"] - mtcars["mpg"].median())

abs_median_devs.median() * 1.4826

Out[17]: 5.411490000000001
```

Note: The **Median Absolute Deviation (MAD)** is frequently multiplied by a **scaling factor of 1.4826**. This adjustment makes the MAD comparable to the **standard deviation** when the data follows a normal distribution, allowing for better consistency between the two measures.

Skewness and Kurtosis : In addition to measures of center and spread, descriptive statistics also include metrics that describe the **shape** of a distribution i.e. skewness and kurtosis. A brief introduction to both is given below:

- **Skewness** indicates the **asymmetry** of the distribution—whether the data is skewed to the left or right.
- **Kurtosis** reflects how much of the data lies in the **tails** versus the **center** of the distribution.

While we won't dive into the exact formulas, these metrics build on the concept of variance:

- **Skewness** uses **cubed deviations** from the mean.
- **Kurtosis** uses **fourth power deviations** from the mean.

```
In [18]: mtcars["mpg"].skew() # Check skewness
```

```
Out[18]: 0.6723771376290805
```

```
In [19]: mtcars["mpg"].kurt() # Check kurtosis
```

```
Out[19]: -0.0220062914240855
```

To better understand **skewness** and **kurtosis**, let's generate some **sample (dummy) data** and examine its distribution using these two measures:

```
In [20]: norm_data = np.random.normal(size=100000)
skewed_data = np.concatenate((np.random.normal(size=35000)+2,
                                np.random.exponential(size=65000)),
                                axis=0)
uniform_data = np.random.uniform(0,2, size=100000)
peaked_data = np.concatenate((np.random.exponential(size=50000),
                                np.random.exponential(size=50000)*(-1)),
                                axis=0)

data_df = pd.DataFrame({"norm":norm_data,
                        "skewed":skewed_data,
                        "uniform":uniform_data,
                        "peaked":peaked_data})
```

```
In [21]: data_df.plot(kind="density",
                        figsize=(10,10),
                        xlim=(-5,5));
```

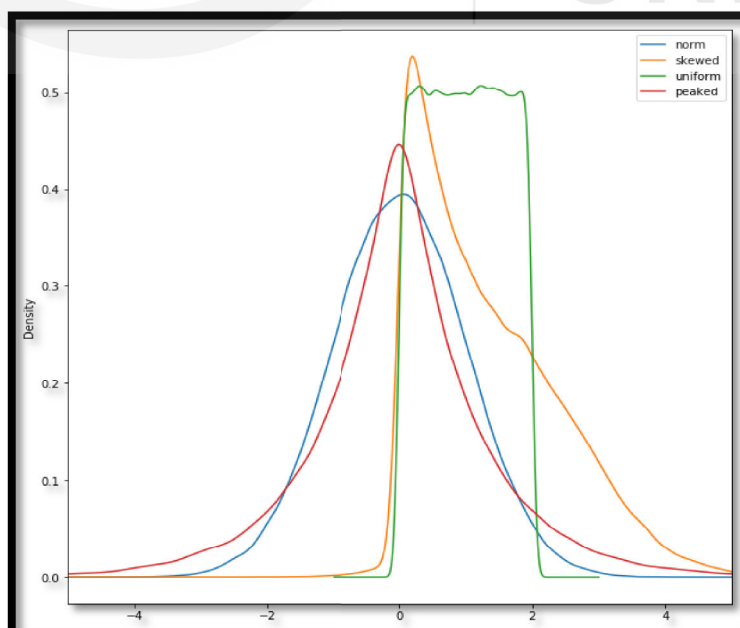


Fig.71: Skewness and Kurtosis

Next, let's evaluate the **skewness** of each dataset. Since skewness reflects the **asymmetry** of a distribution, we expect it to be **low** for most of the distributions that are relatively **symmetric**, and **higher** only for the one that is intentionally **skewed**.

```
In [22]: data_df.skew()

Out[22]:
norm      -0.003337
skewed     1.004511
uniform   -0.001172
peaked     0.001861
dtype: float64
```

Now let's check kurtosis.

```
In [23]: data_df.kurt()

Out[23]:
norm      -0.009062
skewed     1.297040
uniform   -1.200426
peaked     2.952304
dtype: float64
```

As observed from the results, the **normal distribution** shows **kurtosis close to zero**, indicating a typical bell-shaped curve. The **uniform (flat) distribution** has **negative kurtosis**, meaning it has **lighter tails** and a flatter peak. In contrast, the other two distributions—with more data in the **tails than in the center**—exhibit **higher kurtosis**, reflecting heavier tails and more extreme values.

So we learned that Descriptive statistics provide numerical summaries that help you understand key aspects of your data, such as its **center, spread, and shape**. These statistics not only guide the direction of your analysis but also make it easier to communicate findings clearly and efficiently. Moreover, metrics like **mean** and **variance** are foundational in many statistical tests and predictive models.

SUM UP - User Interface and Data Handling in Python (Anaconda/Google Colab)

1. Introduction to Anaconda & Colab

- **Anaconda Navigator:** Launch Jupyter Notebook, Spyder.

- **Jupyter Notebook Interface:** Cell-based execution, Markdown support.
- **Google Colab:** Cloud-based, supports GPU, file uploads.

2. *Uploading the Dataset*

- In Jupyter:

```
pd.read_csv("file.csv")
```

- In Google Colab, import and use the file upload function:

```
from google.colab import files  
uploaded = files.upload()
```

- Then, read the file using Pandas:

```
pd.read_csv("file.csv")
```

3. *Handling Data*

- Use Pandas: `df.head()`, `df.info()`, `df.describe()`
- Clean data: drop missing, rename columns, filter rows.
- Transformation: `df['new'] = df['col1'] + df['col2']`

5. *Creating Visualizations Use libraries like Matplotlib and Seaborn to plot different types of graphs:*

```
import matplotlib.pyplot as plt  
import seaborn as sns  
  
plt.bar(x, y) # Bar chart  
sns.histplot(df['column']) # Histogram  
sns.scatterplot(x='x', y='y', data=df) # Scatter plot
```

- Use Matplotlib and Seaborn for various graphs.

☛ Check Your Progress - 3

1. What function in pandas shows basic statistics of a dataset?

.....
.....

2. How can you read a CSV file in pandas?

.....
.....
.....

3. What library in Python is used for plotting graphs?

.....
.....

4. How do you calculate mean and median in pandas?

.....
.....

5. How can you identify outliers using visualization?

.....
.....

13.8 R-STUDIO

RStudio is a powerful, user-friendly IDE for R programming. It's the preferred environment for most R users. It offers Clean layout for data science tasks, and quite easy access to documentation and plots with integrated version control (Git). However, Other Interfaces (Optional) are **Jupyter Notebook with IRKernel** – Interactive R notebooks, **VS Code with R extension**, **StatET Plugin for Eclipse**, and **Jamovi** also.

Here, we will work on R-Studio only, the user interface of the same is shown below:

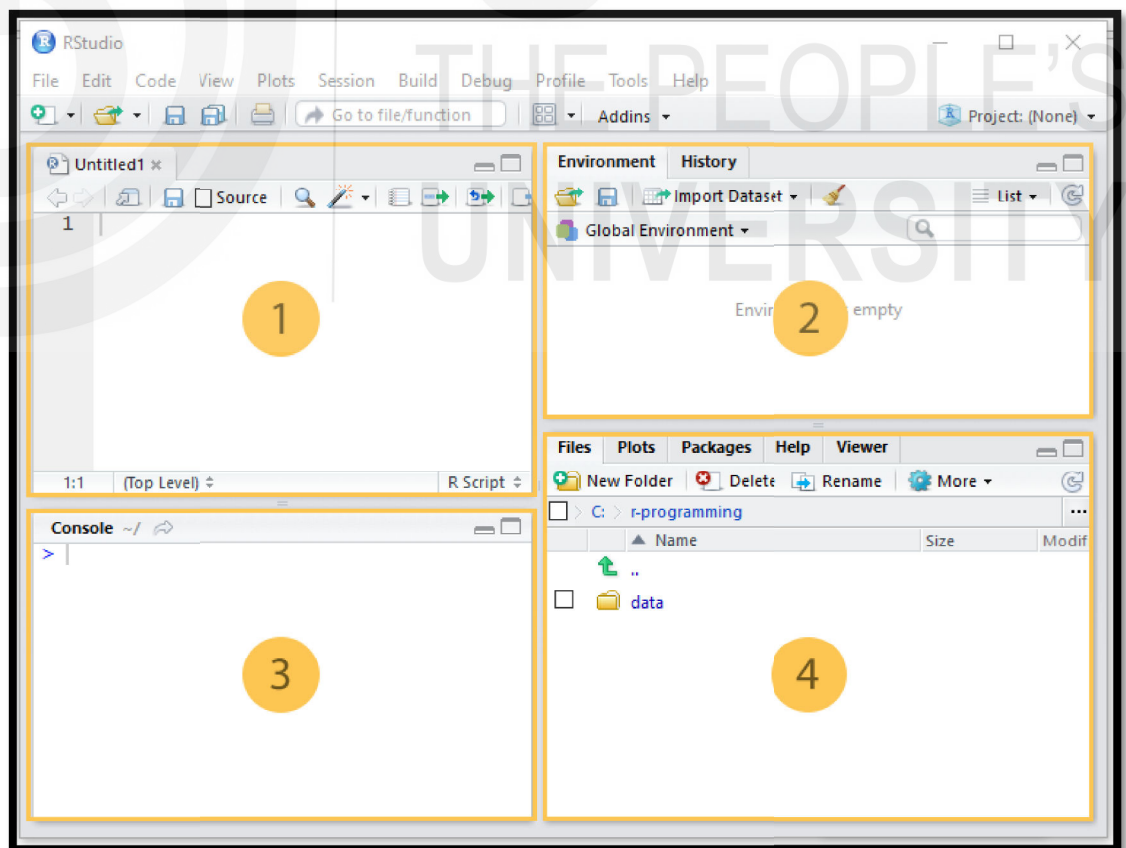


Fig.72: R-Studio: Interface Layout (4 Panes):

The description of the 4-panes shown in the above figure, is as follows:

1. Source Editor (Top-Left)

- Write and edit R scripts (.R), RMarkdown (.Rmd), and notebooks.
- Supports syntax highlighting, code folding, and auto-completion.

Source Editor This is where you **write and edit scripts**, RMarkdown files, and notebooks. You can run code from here directly into the Console (Pane 3). Useful for saving and reusing code.

2. Environment / History Pane

- **Environment Tab:** Shows all the **objects, variables, and datasets** you've created.
- **History Tab:** Keeps a **record of previously executed commands**.
- Helps manage workspace variables and track command history.

3. Console (Bottom-Left)

- Run R commands interactively, and it Displays output, errors, and messages.

4. Files/Plots/Packages/Help/Viewer (Bottom-Right)

- **Files:** Navigate your project directory.
- **Plots:** Display visualizations created in R.
- **Packages:** Manage installed libraries.
- **Help:** Access documentation for functions.

Now, after understanding the utility of all 4-panes of the user interface of R-Studio, let's extend our discussion to learn How to use RStudio's GUI for Importing the Dataset. The steps to be performed are as follows:

Steps:

1. Open **RStudio**.
2. In the **Environment pane (top-right)**, click **"Import Dataset"**.
3. Choose from:
 - **From Text (readr)** or **From Excel**, or **From CSV**
4. Browse and select your file (e.g., data.csv).
5. A preview appears—check the options like separator, header, etc.
6. Click **"Import"**.
7. The dataset is loaded into your environment (usually as a data frame).

Steps for Alternate Method : Using Code (R Console or Script)

1. To Import a CSV File:

```
data <- read.csv("path/to/your/file.csv")
```

2. Example (Windows path):

```
data <- read.csv("C:/Users/YourName/Documents/data.csv")
```

3. To Import Excel File:

```
install.packages("readxl")
library(readxl)
data <- read_excel("path/to/your/file.xlsx")
```

Common Import Functions

Function	Use	Package
read.csv()	Load CSV files	Base R
read.table()	Load general delimited text files	Base R
read_excel()	Load Excel .xlsx files	readxl
read_csv()	Faster CSV loading	readr
fread()	Fast and efficient CSV loader	data.table

Table 4: Functions of R

Note : Use `getwd()` to check the current working directory, and `setwd("path")` to change it:

```
getwd()
setwd("C:/Users/YourName/Documents")
```

After understanding the basic working structure of R-Studio, let's extend our discussion a next level, i.e. how to perform Descriptive analysis in R? the same is addressed in next section.

13.8.2 Descriptive Analysis in R Programming

Descriptive statistics in R involve summarizing and organizing data using various tools such as **charts, graphs, tables, and spreadsheets**. This process helps in presenting the data in a **clear and understandable format**, allowing for meaningful insights. Descriptive analysis aims to **summarize the main features** of a dataset. It is typically performed on **smaller datasets**, providing a foundational understanding that can also guide **future trend predictions** based on existing data patterns.

Descriptive statistics often rely on two major types of measures i.e. Measures of Central Tendency and Measures of Variability (Dispersion), the two are discussed below:

1□. Measures of Central Tendency: Indicates the center or typical value of the dataset and include:

- **Mean (Average):** The arithmetic average of all observations. (R-Syntax:

```
mean(data vector) )
```

- **Median:** The middle value when the data is arranged in ascending order.
(R-syntax: median(data vector))
- **Mode:** The most frequently occurring value. R does not have a built-in mode function, so it can be computed as: (R-Syntax: mode value <- names(sort(table(data vector), decreasing = TRUE))

2. **Measures of Variability (Dispersion):** Reflects spread out of the data and it includes:

- **Range :** Difference between the highest and lowest values.
 - R-Syntax : `range(data vector)`
 - `diff(range(data vector))`
- **Interquartile Range (IQR) :** Measures the spread of the middle 50% of the data.
 - R-Syntax : `IQR(data vector)`
- **Variance:** Average of the squared deviations from the mean.
 - R-Syntax : `var(data vector)`
- **Standard Deviation :** Square root of the variance; shows spread in the same units as the data.
 - R-Syntax : `sd(data vector)`

Example:

```
# Sample dataset
data vector <- c(12, 15, 14, 19, 20, 15, 18, 22, 15)
# Central Tendency
mean(data vector)
median(data vector)
mode value <- names(sort(table(data vector), decreasing = TRUE))
# Variability
range(data vector)
IQR(data vector)
var(data vector)
sd(data vector)
```

This code produces output that clearly shows the descriptive statistics,

Given: data vector <- c(12, 15, 14, 19, 20, 15, 18, 22, 15)

Measure	Output
Mean	16.67
Median	15
Mode	15
Range	12 to 22
Range (Difference)	10
IQR	4

Variance	11.25
Standard Deviation	3.354102

Now, let's revisit the Steps for Performing Descriptive Statistics in R

1. **Load your dataset** using functions like `read.csv()`, `read_excel()`, or `data()` for built-in sets.
2. **Summarize the data:**
 - Use `summary(data)` for a quick overview
 - Use `mean()`, `median()`, `sd()`, `var()`, `range()` for individual measures
3. **Visualize the data:**
 - Create graphs such as histograms (`hist()`), boxplots (`boxplot()`), and bar charts (`barplot()`)
4. **Interpret results** to understand trends, central points, and variability in your data.

To upload a dataset in R, you can use several methods depending on your file type (e.g., CSV, Excel) and your preference (command line, GUI, etc.). The commonly used methods are as follows:

1. **Uploading a CSV File** of dataset can be done by using the `read.csv()` function:

```
# Basic CSV file loading
data <- read.csv("path/to/your/file.csv")
```

- Replace "path/to/your/file.csv" with your actual file path.
 - If using RStudio, you can drag and drop the file into the Files pane, then use the path shown.
2. **Uploading an Excel File** : Use the `readxl` or `openxlsx` package:

```
# First, install the package if not already done
install.packages("readxl")

# Load the package
library(readxl)

# Read Excel file (first sheet by default)
data <- read_excel("path/to/your/file.xlsx")
```

3. **Uploading dataset by Using RStudio GUI (Import Dataset Button)**
 1. Click on **Environment tab > Import Dataset**.
 2. Choose:
 - **From Text (readr)** – for CSV/TSV

- **From Excel** – for Excel files
3. Browse and select your file.
 4. RStudio auto-generates the code and previews the data.
- #### 4. Uploading from the Web

```
data <- read.csv("https://example.com/yourfile.csv")
```

Till the moment we learned how to handle a dataset, now its time to extend our discussion for data visualization and data interpretation. To understand these features we are going to refer to the renowned dataset of mtcars, a built in dataset in R. you can explore the dataset by applying commands, shown in the respective screenshots, given below:

In [2]:

```
df <- mtcars #built-in dataset in R
```

In [3]:

```
head(df) #check the first 6 values
```

A data.frame: 6 × 11

	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
Mazda RX4	21.0	6	160	110	3.90	2.620	16.46	0	1	4	4
Mazda RX4 Wag	21.0	6	160	110	3.90	2.875	17.02	0	1	4	4
Datsun 710	22.8	4	108	93	3.85	2.320	18.61	1	1	4	1
Hornet 4 Drive	21.4	6	258	110	3.08	3.215	19.44	1	0	3	1
Hornet Sportabout	18.7	8	360	175	3.15	3.440	17.02	0	0	3	2
Valiant	18.1	6	225	105	2.76	3.460	20.22	1	0	3	1

In [4]:

```
tail(df) #check the last 6 values
```

A data.frame: 6 × 11

	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
Porsche 914-2	26.0	4	120.3	91	4.43	2.140	16.7	0	1	5	2
Lotus Europa	30.4	4	95.1	113	3.77	1.513	16.9	1	1	5	2
Ford Pantera L	15.8	8	351.0	264	4.22	3.170	14.5	0	1	5	4
Ferrari Dino	19.7	6	145.0	175	3.62	2.770	15.5	0	1	5	6
Maserati Bora	15.0	8	301.0	335	3.54	3.570	14.6	0	1	5	8
Volvo 142E	21.4	4	121.0	109	4.11	2.780	18.6	1	1	4	2

In [5]:

```
dim(df) #check the dimensions of the data, 32 rows and 11 variables
```

32 × 11

```
In [5]: str(df) #check the structure of the data

'data.frame':  32 obs. of  11 variables:
 $ mpg : num  21 21 22.8 21.4 18.7 18.1 14.3 24.4 22.8 19.2 ...
 $ cyl : num  6 6 4 6 8 6 8 4 4 6 ...
 $ disp: num  160 160 108 258 360 ...
 $ hp  : num  110 110 93 110 175 105 245 62 95 123 ...
 $ drat: num  3.9 3.9 3.85 3.08 3.15 2.76 3.21 3.69 3.92 3.92 ...
 $ wt  : num  2.62 2.88 2.32 3.21 3.44 ...
 $ qsec: num  16.5 17 18.6 19.4 17 ...
 $ vs  : num  0 0 1 1 0 1 0 1 1 1 ...
 $ am  : num  1 1 1 0 0 0 0 0 0 0 ...
 $ gear: num  4 4 4 3 3 3 3 4 4 4 ...
 $ carb: num  4 4 1 1 2 1 4 2 2 4 ...
```

```
In [7]: #The Details of the dataset can be found by using:
?mtcars
```

For the purpose of Data Visualization and interpretation we are going to use the data visualization library ggplot() over the built-in dataset “mtcars”.

```
In [8]: library(ggplot2)
```

Here's how you can systematically *perform data visualization and data interpretation, using R*

Task-1: *What is the Relation between MPG (Mileage) and Number of Cylinders ?*

We know that MPG, or *miles per gallon*, indicates how far a car can travel using one gallon of fuel. It's a standard measure of **fuel efficiency**—higher MPG means better fuel economy and less fuel consumption per mile. So, by plotting the graph between *MPG (Mileage) and Number of Cylinders*, using *command shown below*:

```
In [9]: ggplot(df, aes(x= factor(cyl), y=mpg)) + geom_point()
```

We got following output as graph or visualization, showing the relationship between MPG (Mileage) and Number of Cylinders

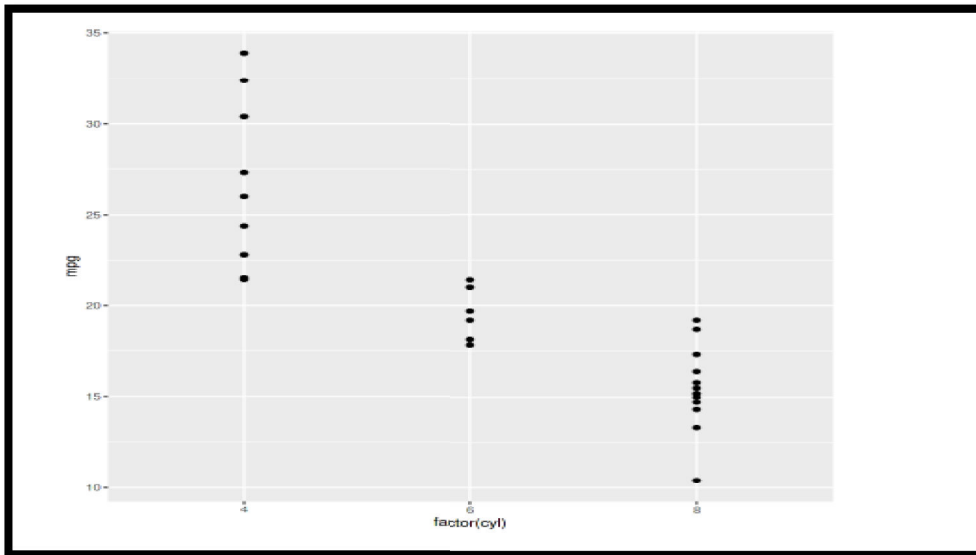


Fig.73: Data Interpretation

Interpretation: The scatter plot shows that as the number of cylinders increases, the mileage (mpg) decreases.

Task-2: Explore the relationship between mileage (mpg) and engine displacement: Again, a graph is plotted between the two i.e. between mileage (mpg) and engine displacement, and the same is shown below:

```
In [10]: ggplot(df, aes(x=mpg, y= disp)) +geom_point()
```

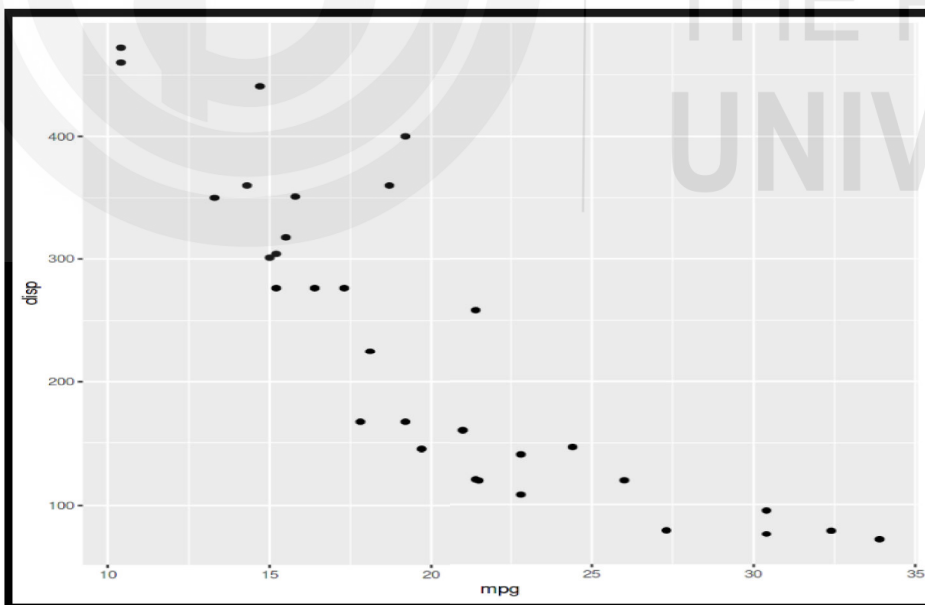


Fig.74: Scatter plot

Interpretation - Greater engine displacement generally results in lower fuel efficiency.

NOTE – (Understanding Displacement): Engine displacement refers to the total volume that all cylinders in an engine can move. A larger displacement allows the

engine to intake more air and fuel, which increases its potential to generate power. However, the actual power output also depends on other internal components and the engine's design. Displacement is usually measured in liters. For example, many modern vehicles have a 2.0-liter four-cylinder engine, meaning each cylinder has a capacity of about 0.5 liters or 500 cc.

Task-3: *How does cylinder effects mpg and displacement?* Again, a graph is plotted between the variables and the same is shown below:

```
In [11]: ggplot(df, aes(x=mpg, y= disp, color =factor(cyl))) + geom_point()
```

```
In [12]: ggplot(df, aes(x=mpg, y= disp, color =factor(cyl))) + geom_point(aes(shape= factor(cyl)))
```

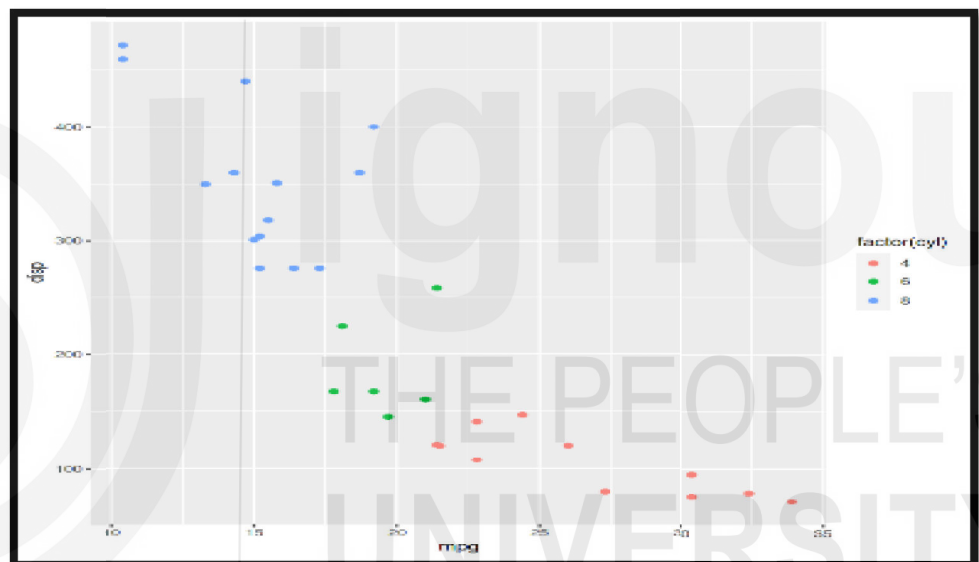


Fig.75: Colored scatter plot

Interpretation - Cars with fewer cylinders tend to have higher mileage and better fuel efficiency.

On the other hand, vehicles with more cylinders usually have larger engine displacement, meaning the engine consumes more air and fuel. As a result, they are generally less fuel-efficient.

Task-4: *Understanding the Relation between mpg, disp and hp, to interpret this a graph is plotted between the variables using the command, shown below:*

```
In [13]: ggplot(df, aes(x=mpg, y= disp, size=hp )) + geom_point()
```

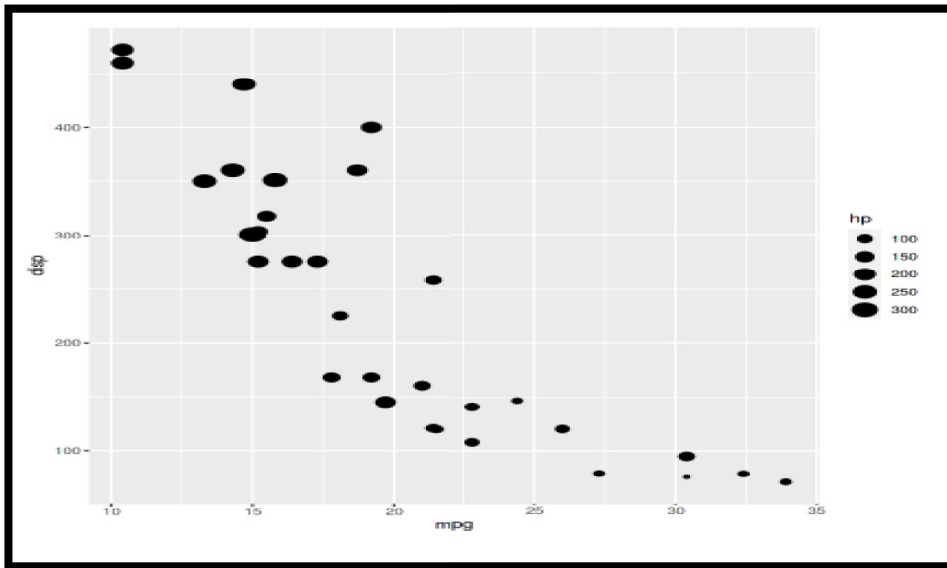


Fig.76: Interpretation of horse power

Interpretation - As horsepower increases, engine displacement also tends to increase. This indicates a more powerful engine, which typically results in lower fuel efficiency — meaning the vehicle delivers less mileage.

Task-5: Understanding the Relation between hp, disp, mpg, and number of cylinders, to interpret this a graph is plotted between the variables using the command, shown below:

```
In [14]: ggplot(df, aes(x=mpg, y= disp, size=hp, color=factor(cyl))) + geom_point()
```

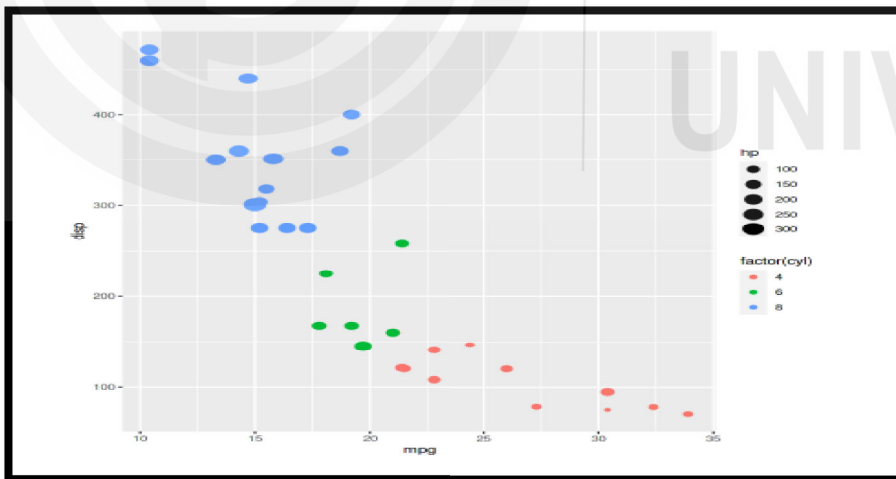


Fig.77: Interpretation of horse power

Interpretation - As horsepower (hp) increases, there is a corresponding increase in engine displacement and usually the number of cylinders as well. This combination points to a more powerful engine. However, such engines tend to be less fuel-efficient, which means cars with higher hp, larger displacement, and more cylinders generally offer lower mileage (mpg).

Addition of shape and color makes it easier to interpret the plot and perform better analysis. This helps in understanding how various features like **displacement**,

horsepower, and **mileage** interact based on engine configuration.

The **Scatter plot** can be generated by using the following syntax in R

```
In [15]: ggplot(df, aes(x=mpg, y= disp, size=hp, color=factor(cyl))) + geom_point(aes(shape=factor(cyl))) +  
labs(title='Motor Cars Scatter Plot', subtitle='showing Mpg,hp,disp,& cyl relation',  
caption='In-Built Dataset', tag='Chart1')
```

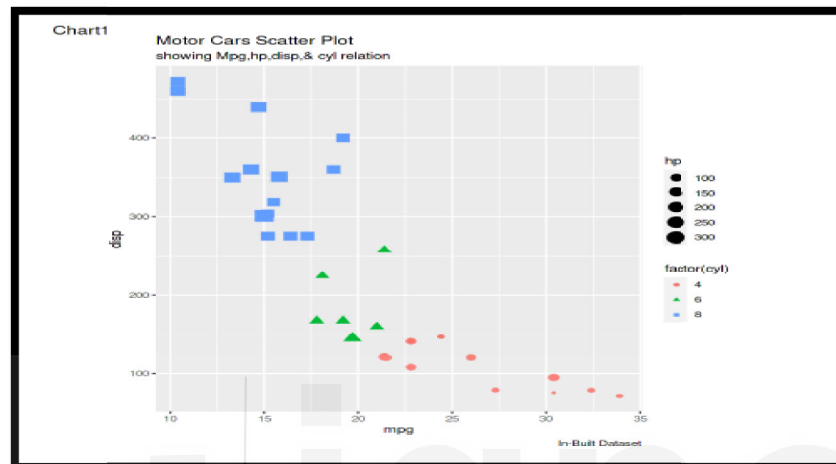


Fig.78: Interpretation of horse power

Generating Bar Charts: Bar charts are useful for summarizing and comparing **categorical variables** or **aggregated values** (like average mileage per cylinder type).

```
In [16]: #simple Bar Chart showing count of cylinders  
ggplot(df, aes(x=factor(cyl)))+geom_bar()
```

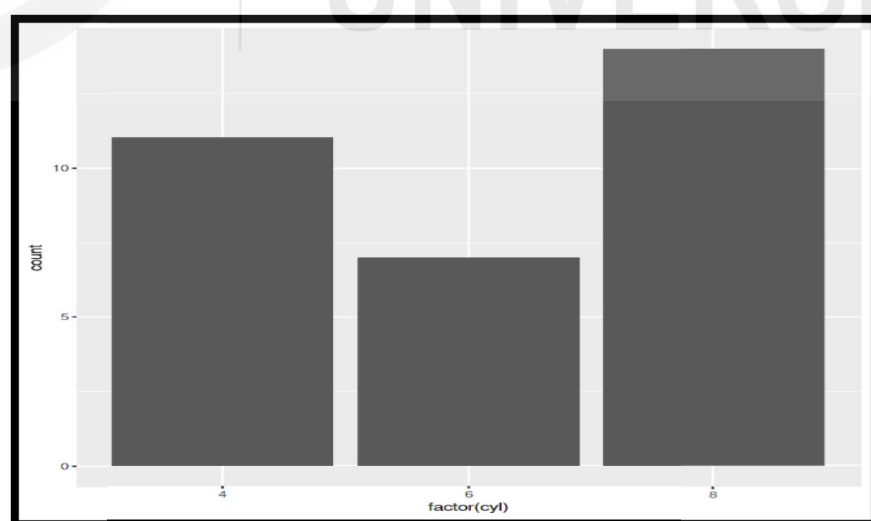


Fig.79: Bar graph

```
In [17]: #using color parameter
ggplot(df, aes(x=factor(cyl), fill=factor(cyl)))+geom_bar()
```

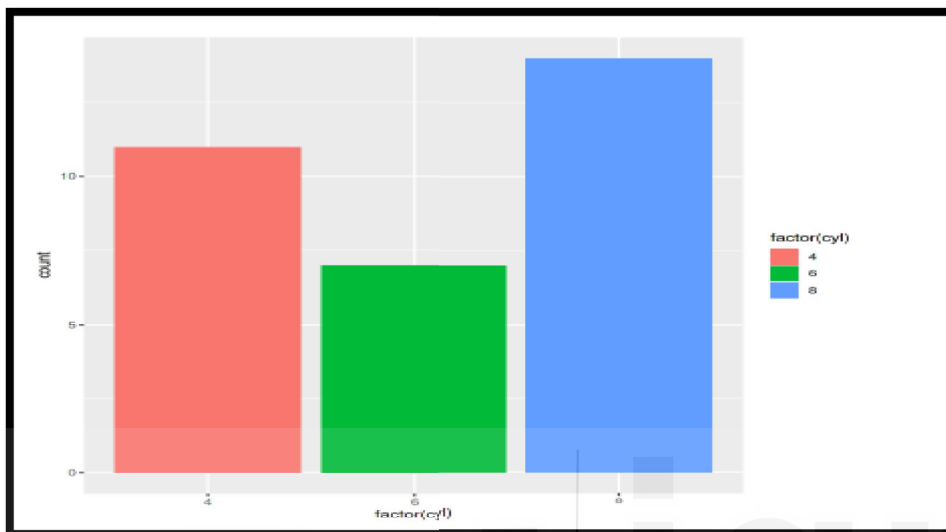


Fig.80: Colored Bar graph

```
In [18]: #add title and subtitle
ggplot(df, aes(x=factor(cyl), fill=factor(cyl)))+geom_bar()+
labs(title = 'Motor Cars Cyl & Count', subtitle='Bar Plot in R')
```

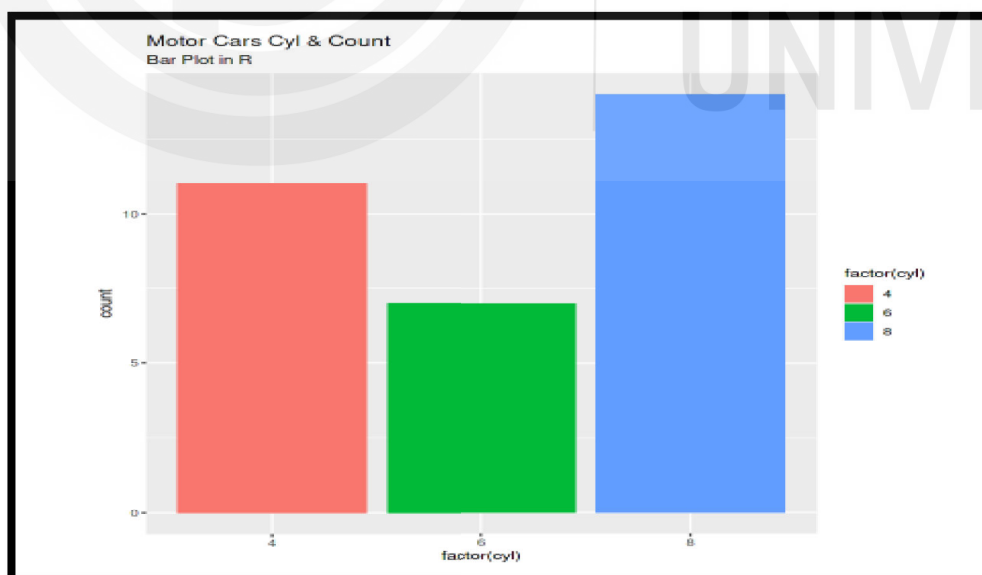


Fig.81: Colored Bar graph (Motor Cars Cyl & Count)

```
In [19]: #using 2 variables
ggplot(df, aes(x=factor(cyl)))+ geom_bar(aes(fill=factor(vs)))
```

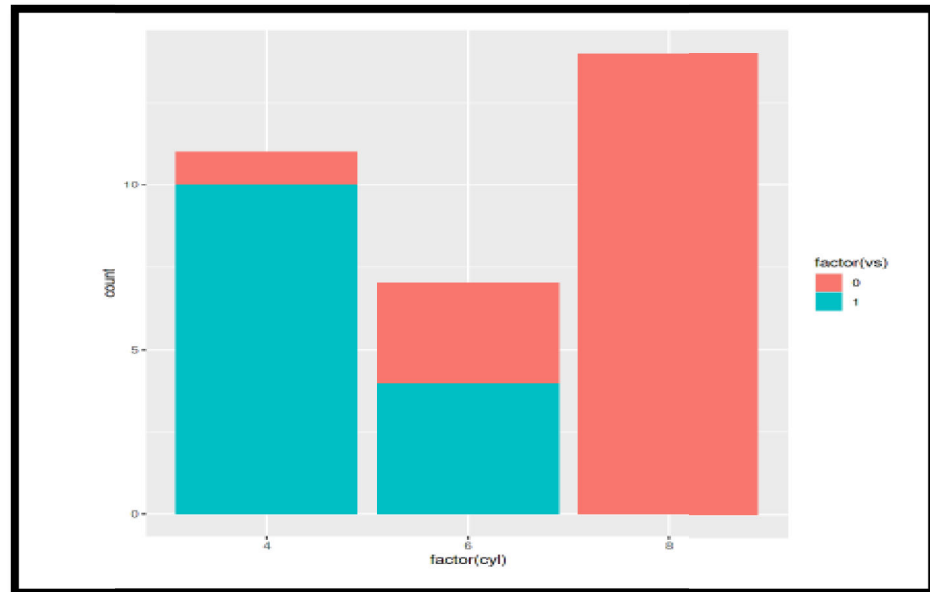


Fig.82 : Factor vs. Count Bar graph

- From the Analysis of the Bar charts produced above, following can be interpreted:
1. Most 8-cylinder engines are V-shaped.
 2. 4-cylinder engines are mainly straight (inline).
 3. 6-cylinder engines have a nearly equal mix of V and straight configurations.

In [20]: `ggplot(df, aes(x=factor(cyl), fill=factor(carb)))+geom_bar()`

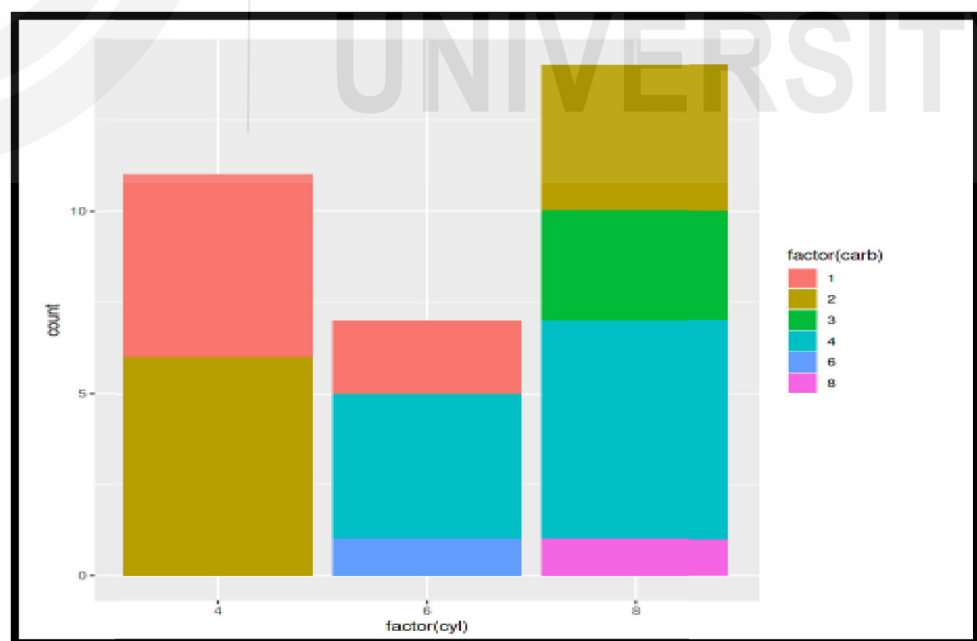


Fig.83: Factor vs. Count Bar graph

- From the Analysis of the Bar charts produced above, following can be interpreted:

- 1) No of Cylinders are 4; carb used are 1 and 2
- 2) No of Cylinders are 6; carb used are 1,4, and 6
- 3) No of Cylinders are 8; carb used are 2,3,4, and 8

➤ **SUM UP - User Interface and Data Handling in R (RStudio)**

1. *Introduction to RStudio Interface*

- **Source Pane:** Script writing.
- **Console Pane:** Code output.
- **Environment/History Pane:** Variables and past commands.
- **Files/Plots/Packages Pane:** File management, visual output.

2. *Uploading the Dataset : `df <- read.csv("filename.csv")`*

- Use "Import Dataset" GUI in RStudio.

3. *Handling Data*

- Use functions like `summary(df)`, `str(df)`, `head(df)`
- Clean using `na.omit()`, `subset()`, `mutate()` (from `dplyr`).

4. *Use Base R plotting functions:*

```
plot(df$var1, df$var2) # Creates a scatter plot
hist(df$var1)          # Draws a histogram of var1
```

For advanced and customizable visuals, use the **ggplot2** package:

```
library(ggplot2)
ggplot(df, aes(x = var1, y = var2)) + geom_point() # Scatter plot using ggplot2
```

- Use base R or ggplot2 for advanced visuals.

🚩 **Check Your Progress - 4**

1. How do you import a CSV file in R?

.....

.....

.....

2. What is the function of `summary()` in R?

.....

.....

.....

3. Which R function is used to generate a histogram?

.....

.....

.....

4. How can you check the structure of a dataset in R?

.....

.....

.....

5. How does cylinder count affect MPG in mtcars dataset?

.....

.....

.....

This structured approach introduces students to the basic interfaces and functionalities of Tableau, Power BI, Python (Anaconda/Colab), and R (RStudio) while ensuring they gain hands-on exposure to uploading datasets, preparing data, and generating useful visual insights.

13.9 SUMMARY

This unit-13 titled “*Tools for Data Science – An Introduction*” emphasizes that in today’s digital age, data is a critical resource, but its value depends on how effectively it is processed and interpreted. Data science, combining mathematics, statistics, computing, and domain expertise, transforms raw data into actionable knowledge. To do this, professionals rely on powerful tools that enable collection, cleaning, analysis, and visualization of data. This unit introduces four essential tools—**Tableau, Power BI, Python, and R**—which together form the foundation of modern data science practice.

Tableau is presented as an intuitive visualization platform requiring little or no coding. It allows learners to connect datasets from various sources, create interactive dashboards, and generate descriptive statistics. Through drag-and-drop features and built-in analytics, Tableau helps communicate insights to both technical and non-technical audiences. Power BI, developed by Microsoft, is introduced as a robust business intelligence tool, closely integrated with Excel and cloud services. Learners are guided through steps of connecting to data, cleaning it using Power Query, building relationships, applying calculations with DAX, and creating dashboards that can be shared via the Power BI service.

The unit then shifts to **Python**, a versatile programming language widely used in data science for its rich libraries such as Pandas, NumPy, Matplotlib, and Seaborn. Learners are introduced to different interfaces like IDLE, Jupyter Notebook, and Google Colab, with hands-on steps for uploading datasets, performing descriptive statistics (mean, median, mode, standard deviation, skewness, kurtosis), and visualizing data through plots. Practical emphasis is laid on using Python for data preparation and exploratory analysis.

Finally, **R (via RStudio)** is introduced as a statistical powerhouse favored in research environments. The unit explains the RStudio interface and dataset import methods, followed by performing descriptive analysis using summary statistics, measures of central tendency, dispersion, and visualization techniques such as histograms, scatter plots, and boxplots. Learners are encouraged to explore

relationships between variables using examples like the *mtcars* dataset.

Overall, the unit equips learners with a comparative understanding of four foundational tools in data science. Tableau and Power BI provide user-friendly, no-code solutions for visualization and business intelligence, while Python and R offer greater flexibility and analytical depth through coding. By the end of the unit, learners gain the ability to import, clean, analyze, and visualize datasets, thereby building confidence to apply these tools in academic, research, or professional contexts.

13.10 SOLUTIONS/ANSWERS

🔧 Check Your Progress - 1

1. What are the key components of the Tableau workspace?

Answer: The key components include Data Pane, Shelves (Rows, Columns), Cards, Toolbar, Worksheet Tabs, and the View Area.

2. How do you connect to a Microsoft Excel file in Tableau?

Answer: Go to the 'Connect' section, select 'To a File', and then choose 'Microsoft Excel' to upload the dataset.

3. What is the purpose of the 'Show Me' panel in Tableau?

Answer: It suggests the most suitable visualizations based on selected data fields.

4. How can you calculate the median in Tableau?

Answer: You can use the median aggregation function or drag the Median option from the Analytics pane.

5. How are outliers identified using Tableau?

Answer: Outliers are identified using distribution bands in visualizations like box plots or line charts.

🔧 Check Your Progress-2

1. What is Power Query used for?

Answer: Power Query is used to clean, transform, and shape data before analysis.

2. How can you create a new measure in Power BI?

Answer: Use the 'New Measure' option under the 'Table Tools' tab and define logic using DAX.

3. What are calculated columns in Power BI?

Answer: Calculated columns are fields created with DAX that become part of the data model and update with refreshes.

4. How do you upload a dataset in Power BI?

Answer: Click on 'Get Data' under Home tab, select the source (e.g., Excel), and load the file.

5. How do you publish a report in Power BI?

Answer: Use the 'Publish' button to upload your report to the Power BI Service for sharing.

☛ Check Your Progress - 3

1. What function in pandas shows basic statistics of a dataset?

Answer: `df.describe()` provides count, mean, std deviation, min, max, and quartiles.

2. How can you read a CSV file in pandas?

Answer: Use `pd.read_csv('filename.csv')` to read the file into a DataFrame.

3. What library in Python is used for plotting graphs?

Answer: Matplotlib and Seaborn are widely used for data visualization.

4. How do you calculate mean and median in pandas?

Answer: Use `df.mean()` for mean and `df.median()` for median.

5. How can you identify outliers using visualization?

Answer: Use boxplots from Seaborn or matplotlib to highlight outliers.

☛ Check Your Progress - 4

1. How do you import a CSV file in R?

Answer: Use `read.csv('filename.csv')` or use the Import Dataset GUI in RStudio.

2. What is the function of summary() in R?

Answer: `summary(data)` gives min, max, mean, median, and quartile information.

3. Which R function is used to generate a histogram?

Answer: `hist()` is used to create histograms in R.

4. How can you check the structure of a dataset in R?

Answer: Use `str(data)` to see the data types and structure.

5. How does cylinder count affect MPG in mtcars dataset?

Answer: More cylinders typically result in lower MPG due to higher fuel consumption.

13.11 FURTHER READINGS

- **Official Tableau Learning Resources** <https://www.tableau.com/learn/training>
- **Tableau Your Data!** – Daniel G. Murray A comprehensive book covering Tableau techniques and real-world applications.
- **Visual Analytics with Tableau** – Alexander Loth Focuses on building dashboards and interpreting data effectively.
- **Microsoft Power BI Documentation** <https://learn.microsoft.com/en-us/power-bi/>
- **The Definitive Guide to DAX** – Marco Russo & Alberto Ferrari, Excellent for learning data modeling and DAX functions in Power BI.
- **Power BI for the Excel Analyst** – Wyn Hopkins, Helpful for Excel users transitioning to Power BI.

- **Python Official Documentation**, <https://docs.python.org/3/>
- **Automate the Boring Stuff with Python** – Al Sweigart (free to read online), Great beginner book with hands-on examples.
- **Python for Data Analysis** – Wes McKinney, Focuses on NumPy, pandas, and data wrangling in Python.
- **Google’s Python Class**, <https://developers.google.com/edu/python>
- **R for Data Science** – Hadley Wickham & Garrett Grolemund, <https://r4ds.had.co.nz/>
A great introductory book for learning R in the context of data analysis.
- **The Art of R Programming** – Norman Matloff, Ideal for students who want to understand R at a deeper programming level.
- **Hands-On Programming with R** – Garrett Grolemund, Interactive and beginner-friendly for learning R syntax and logic.
- **CRAN R Manuals and Tutorials**, <https://cran.r-project.org/manuals.html>
- Tableau Software. (2024). *Tableau Training and Tutorials*. Retrieved from <https://www.tableau.com/learn/training>
- Loth, A. (2019). *Visual Analytics with Tableau*. Wiley.
- Murray, D. G. (2016). *Tableau Your Data! Fast and Easy Visual Analysis with Tableau Software*. Wiley.
- Microsoft Corporation. (2024). *Power BI Documentation*. Retrieved from <https://learn.microsoft.com/en-us/power-bi/>
- Russo, M., & Ferrari, A. (2020). *The Definitive Guide to DAX: Business Intelligence for Microsoft Power BI, SQL Server Analysis Services, and Excel*. Microsoft Press.
- Hopkins, W. (2021). *Power BI for the Excel Analyst: A Step-by-Step Guide to Effective Data Analysis*. Apress.
- Python Software Foundation. (2024). *Python 3 Documentation*. Retrieved from <https://docs.python.org/3/>
- Sweigart, A. (2019). *Automate the Boring Stuff with Python*. No Starch Press.
- McKinney, W. (2022). *Python for Data Analysis: Data Wrangling with pandas, NumPy, and Jupyter*. O'Reilly Media.
- Google Developers. (2024). *Google's Python Class*. Retrieved from <https://developers.google.com/edu/python>
- Wickham, H., & Grolemund, G. (2016). *R for Data Science*. O'Reilly Media. Retrieved from <https://r4ds.had.co.nz/>
- Matloff, N. (2011). *The Art of R Programming*. No Starch Press.
- Grolemund, G. (2014). *Hands-On Programming with R*. O'Reilly Media.
- R Core Team. (2024). *R: The R Project for Statistical Computing*. Retrieved from <https://cran.r-project.org/>